
MicroPython Documentation

Release v1.26.0

MicroPython authors and contributors

Aug 09, 2025

CONTENTS

1	MicroPython libraries	1
1.1	Python standard libraries and micro-libraries	1
1.1.1	array – arrays of numeric data	2
1.1.2	asyncio — asynchronous I/O scheduler	3
1.1.3	binascii – binary/ASCII conversions	7
1.1.4	builtins – builtin functions and exceptions	8
1.1.5	cmath – mathematical functions for complex numbers	11
1.1.6	collections – collection and container types	12
1.1.7	errno – system error codes	13
1.1.8	gc – control the garbage collector	14
1.1.9	gzip – gzip compression & decompression	15
1.1.10	hashlib – hashing algorithms	16
1.1.11	heapq – heap queue algorithm	17
1.1.12	io – input/output streams	18
1.1.13	json – JSON encoding and decoding	19
1.1.14	marshal – Python object serialization	20
1.1.15	math – mathematical functions	20
1.1.16	os – basic “operating system” services	22
1.1.17	platform – access to underlying platform’s identifying data	25
1.1.18	random – generate random numbers	25
1.1.19	re – simple regular expressions	26
1.1.20	select – wait for events on a set of streams	29
1.1.21	socket – socket module	30
1.1.22	ssl – SSL/TLS module	36
1.1.23	struct – pack and unpack primitive data types	38
1.1.24	sys – system specific functions	40
1.1.25	time – time related functions	42
1.1.26	zlib – zlib compression & decompression	46
1.1.27	_thread – multithreading support	47
1.2	MicroPython-specific libraries	47
1.2.1	bluetooth — low-level Bluetooth	47
1.2.2	btree – simple BTree database	59
1.2.3	cryptolib – cryptographic ciphers	61
1.2.4	deflate – deflate compression & decompression	62
1.2.5	framebuf — frame buffer manipulation	64
1.2.6	machine — functions related to the hardware	67
1.2.7	micropython – access and control MicroPython internals	109
1.2.8	neopixel — control of WS2812 / NeoPixel LEDs	112
1.2.9	network — network configuration	113
1.2.10	openamp – provides standard Asymmetric Multiprocessing (AMP) support	126

1.2.11	uctypes – access binary data in a structured way	127
1.2.12	vfs – virtual filesystem control	132
1.2.13	WM8960 – Driver for the WM8960 codec	135
1.3	Port-specific libraries	141
1.3.1	Libraries specific to the pyboard	141
1.3.2	Libraries specific to the WiPy	190
1.3.3	Libraries specific to the ESP8266 and ESP32	194
1.3.4	Libraries specific to the RP2040	218
1.3.5	Libraries specific to Zephyr	230
1.4	Extending built-in libraries from Python	233
2	MicroPython language and implementation	235
2.1	Glossary	235
2.2	The MicroPython Interactive Interpreter Mode (aka REPL)	238
2.2.1	Auto-indent	238
2.2.2	Auto-completion	239
2.2.3	Interrupting a running program	239
2.2.4	Paste mode	240
2.2.5	Soft reset	240
2.2.6	The special variable _ (underscore)	241
2.2.7	Raw mode and raw-paste mode	241
2.3	Reset and Boot Sequence	243
2.3.1	Hard reset	243
2.3.2	Soft Reset	243
2.3.3	Boot Sequence	244
2.3.4	Soft Bricking (failure to boot)	245
2.4	MicroPython remote control: mpremote	246
2.4.1	Commands	247
2.4.2	Auto connection and soft-reset	252
2.4.3	Shortcuts	252
2.4.4	Examples	253
2.5	MicroPython .mpy files	256
2.5.1	Versioning and compatibility of .mpy files	256
2.5.2	Binary encoding of .mpy files	257
2.6	Writing interrupt handlers	259
2.6.1	Tips and recommended practices	259
2.6.2	MicroPython issues	259
2.6.3	Exceptions	262
2.6.4	Interfacing to asyncio	262
2.6.5	General issues	263
2.7	Maximising MicroPython speed	265
2.7.1	Designing for speed	266
2.7.2	Identifying the slowest section of code	268
2.7.3	MicroPython code improvements	268
2.7.4	The Native code emitter	269
2.7.5	The Viper code emitter	269
2.7.6	Accessing hardware directly	271
2.8	MicroPython on microcontrollers	271
2.8.1	Flash memory	271
2.8.2	RAM	272
2.8.3	The heap	275
2.8.4	String operations	277
2.8.5	Postscript	277
2.9	MicroPython manifest files	277

2.9.1	Summary	277
2.9.2	Writing manifest files	278
2.9.3	Examples	280
2.10	Package management	281
2.10.1	Installing packages with <code>mip</code>	281
2.10.2	Installing packages with <code>mpremote</code>	282
2.10.3	Installing packages manually	283
2.10.4	Writing & publishing packages	283
2.10.5	Freezing packages	284
2.11	Inline assembler for Thumb2 architectures	284
2.11.1	Document conventions	284
2.11.2	Instruction categories	285
2.11.3	Usage examples	293
2.11.4	References	296
2.12	Working with filesystems	297
2.12.1	VFS	297
2.12.2	Block devices	297
2.12.3	Filesystems	300
2.13	The <code>pyboard.py</code> tool	302
2.13.1	Running a command on the device	303
2.13.2	Running a script on the device	303
2.13.3	Filesystem access	303
2.13.4	Using the <code>pyboard</code> library	304
2.14	MicroPython 2.0 Migration Guide	304
2.14.1	Hardware and peripherals	304
2.14.2	OS & filesystem	305
2.14.3	CPython compatibility	305
3	MicroPython differences from CPython	307
3.1	Python 3.5	307
3.2	Python 3.6	309
3.3	Python 3.7	311
3.4	Python 3.8	313
3.5	Python 3.9	315
3.6	Python 3.10	317
3.7	Syntax	320
3.7.1	Literals	321
3.7.2	Operators	322
3.7.3	Unicode	322
3.7.4	Unpacking	323
3.8	Core language	323
3.8.1	Classes	323
3.8.2	Functions	327
3.8.3	Generator	328
3.8.4	Runtime	329
3.8.5	f-strings	330
3.8.6	import	331
3.9	Builtin types	332
3.9.1	Exception	332
3.9.2	bytearray	335
3.9.3	bytes	335
3.9.4	complex	336
3.9.5	dict	337
3.9.6	float	337

3.9.7	int	337
3.9.8	list	339
3.9.9	memoryview	340
3.9.10	str	340
3.9.11	tuple	342
3.10	Modules	343
3.10.1	Positional-only Parameters	343
3.10.2	array	344
3.10.3	json	345
3.10.4	os	346
3.10.5	random	347
3.10.6	struct	348
3.10.7	sys	349
4	MicroPython Internals	351
4.1	Getting Started	351
4.1.1	Source control with git	351
4.1.2	Get the code	351
4.1.3	Compile and build the code	352
4.1.4	Building the documentation	354
4.1.5	Running the tests	355
4.1.6	Folder structure	355
4.2	Writing tests	356
4.3	The Compiler	357
4.3.1	Adding a grammar rule	357
4.3.2	Adding a lexical token	358
4.3.3	Parsing	359
4.3.4	Compiler passes	359
4.3.5	Emitting bytecode	361
4.3.6	Emitting native code	361
4.4	Memory Management	362
4.4.1	The object model	362
4.4.2	Allocation of objects	363
4.5	Implementing a Module	364
4.5.1	Implementing a core module	365
4.6	Optimizations	366
4.6.1	Frozen bytecode	366
4.6.2	Variables	366
4.6.3	Allocation of memory	367
4.7	MicroPython string interning	367
4.7.1	Compile-time QSTR generation	367
4.7.2	Run-time QSTR generation	368
4.8	Maps and Dictionaries	368
4.8.1	Open addressing	368
4.8.2	Linear probing	369
4.9	The public C API	370
4.10	Extending MicroPython in C	370
4.10.1	MicroPython external C modules	370
4.10.2	Native machine code in .mpy files	374
4.11	Porting MicroPython	378
4.11.1	Minimal MicroPython firmware	379
4.11.2	MicroPython Configurations	380
4.11.3	Support for standard input/output	381
4.11.4	Building and running	382

4.11.5	Adding a module to the port	382
5	MicroPython license information	385
6	Quick reference for the pyboard	387
6.1	General information about the pyboard	387
6.1.1	Local filesystem and SD card	388
6.1.2	Boot modes	388
6.1.3	Errors: flashing LEDs	389
6.1.4	Guide for using the pyboard with Windows	389
6.1.5	The pyboard hardware	389
6.1.6	Datasheets for the components on the pyboard	389
6.1.7	Datasheets for other components	389
6.2	MicroPython tutorial for the pyboard	390
6.2.1	Introduction to the pyboard	390
6.2.2	Running your first script	390
6.2.3	Getting a MicroPython REPL prompt	393
6.2.4	Turning on LEDs and basic Python concepts	394
6.2.5	Switches, callbacks and interrupts	396
6.2.6	The accelerometer	398
6.2.7	Safe mode and factory reset	399
6.2.8	Making the pyboard act as a USB mouse	400
6.2.9	The Timers	402
6.2.10	Inline assembler	404
6.2.11	Power control	406
6.2.12	Tutorials requiring extra components	406
6.2.13	Tips, tricks and useful things to know	418
6.3	General board control	419
6.4	Delay and timing	419
6.5	Internal LEDs	419
6.6	Internal switch	420
6.7	Pins and GPIO	420
6.8	Servo control	420
6.9	External interrupts	421
6.10	Timers	421
6.11	RTC (real time clock)	421
6.12	PWM (pulse width modulation)	421
6.13	ADC (analog to digital conversion)	421
6.14	DAC (digital to analog conversion)	422
6.15	UART (serial bus)	422
6.16	SPI bus	422
6.17	I2C bus	422
6.18	I2S bus	423
6.19	CAN bus (controller area network)	423
6.20	Internal accelerometer	423
7	Quick reference for the ESP8266	425
7.1	General information about the ESP8266 port	425
7.1.1	Multitude of boards	425
7.1.2	Technical specifications and SoC datasheets	426
7.1.3	Scarcity of runtime resources	426
7.1.4	Boot process	426
7.1.5	Known Issues	426
7.2	MicroPython tutorial for ESP8266	428

7.2.1	Getting started with MicroPython on the ESP8266	428
7.2.2	Getting a MicroPython REPL prompt	431
7.2.3	The internal filesystem	434
7.2.4	Network basics	435
7.2.5	Network - TCP sockets	436
7.2.6	GPIO Pins	439
7.2.7	Pulse Width Modulation	440
7.2.8	Analog to Digital Conversion	441
7.2.9	Power control	441
7.2.10	Controlling 1-wire devices	442
7.2.11	Controlling NeoPixels	443
7.2.12	Controlling APA102 LEDs	444
7.2.13	Temperature and Humidity	446
7.2.14	Using a SSD1306 OLED display	447
7.2.15	Next steps	449
7.3	Installing MicroPython	449
7.4	General board control	449
7.5	Networking	450
7.6	Delay and timing	450
7.7	Timers	450
7.8	Pins and GPIO	451
7.9	UART (serial bus)	451
7.10	PWM (pulse width modulation)	452
7.11	ADC (analog to digital conversion)	452
7.12	Software SPI bus	452
7.13	Hardware SPI bus	453
7.14	I2C bus	453
7.15	Real time clock (RTC)	453
7.16	WDT (Watchdog timer)	454
7.17	Deep-sleep mode	454
7.18	OneWire driver	455
7.19	NeoPixel driver	455
7.20	APA102 driver	456
7.21	DHT driver	456
7.22	SSD1306 driver	456
7.23	WebREPL (web browser interactive prompt)	457
8	Quick reference for the ESP32	459
8.1	General information about the ESP32 port	459
8.1.1	Multitude of boards	460
8.1.2	Technical specifications and SoC datasheets	460
8.2	MicroPython tutorial for ESP32	461
8.2.1	Getting started with MicroPython on the ESP32	461
8.2.2	Pulse Width Modulation	463
8.2.3	Accessing peripherals directly via registers	467
8.2.4	Factory reset	470
8.3	Installing MicroPython	471
8.4	General board control	471
8.5	Networking	471
8.5.1	WLAN	471
8.5.2	LAN	472
8.6	Delay and timing	474
8.7	Timers	475
8.8	Pins and GPIO	475

8.9	UART (serial bus)	476
8.10	PWM (pulse width modulation)	476
8.11	DAC (digital to analog conversion)	477
8.12	ADC (analog to digital conversion)	478
8.13	Pulse Counter (pin pulse/edge counting)	479
8.14	Software SPI bus	480
8.15	Hardware SPI bus	481
8.16	Software I2C bus	481
8.17	Hardware I2C bus	481
8.18	I2S bus	482
8.19	Real time clock (RTC)	482
8.20	WDT (Watchdog timer)	482
8.21	Deep-sleep mode	483
8.22	SD card	484
8.23	RMT	484
8.24	OneWire driver	484
8.25	NeoPixel and APA106 driver	485
8.26	Capacitive touch	485
8.27	DHT driver	486
8.28	WebREPL (web browser interactive prompt)	487
9	Quick reference for the RP2	489
9.1	General information about the RP2xxx port	489
9.1.1	Technical specifications and SoC datasheets	490
9.2	Getting started with MicroPython on the RP2xxx	490
9.2.1	Factory reset	490
9.2.2	Programmable IO	490
9.3	Installing MicroPython	493
9.4	General board control	493
9.5	Networking	493
9.5.1	WLAN	493
9.6	Delay and timing	494
9.7	Timers	494
9.8	Pins and GPIO	495
9.9	Programmable IO (PIO)	495
9.10	UART (serial bus)	496
9.11	PWM (pulse width modulation)	496
9.12	ADC (analog to digital conversion)	496
9.13	Software SPI bus	497
9.14	Hardware SPI bus	497
9.15	Software I2C bus	498
9.16	Hardware I2C bus	498
9.17	I2S bus	498
9.18	Real time clock (RTC)	499
9.19	WDT (Watchdog timer)	499
9.20	OneWire driver	499
9.21	NeoPixel and APA106 driver	500
10	Quick reference for the i.MXRT family	501
10.1	General information about the MIMXRT port	501
10.1.1	Multitude of boards	501
10.1.2	Supported MCUs	502
10.1.3	Technical specifications and SoC datasheets	502
10.2	Getting started with MicroPython on the i.MXRT	503

10.2.1	Requirements	503
10.2.2	Powering the board	503
10.2.3	Getting the firmware	503
10.2.4	Deploying the firmware	503
10.2.5	Serial prompt	504
10.2.6	Troubleshooting installation problems	504
10.3	Pinout for the i.MXRT machine modules	505
10.3.1	UART pin assignment	505
10.3.2	PWM pin assignment	506
10.3.3	Hardware SPI pin assignment	512
10.3.4	Hardware I2C pin assignment	513
10.3.5	Hardware I2S pin assignment	514
10.4	Installing MicroPython	515
10.5	General board control	515
10.6	Delay and timing	515
10.7	Timers	516
10.8	Pins and GPIO	516
10.9	UART (serial bus)	517
10.10	PWM (pulse width modulation)	517
10.10.1	PWM Constructor	518
10.10.2	PWM Methods	519
10.10.3	PWM Pin Assignment	519
10.11	ADC (analog to digital conversion)	519
10.12	Software SPI bus	519
10.13	Hardware SPI bus	520
10.14	Software I2C bus	520
10.15	Hardware I2C bus	521
10.16	I2S bus	521
10.17	Real time clock (RTC)	522
10.18	SD card	522
10.19	OneWire driver	523
10.20	DHT driver	523
10.21	Ethernet driver	524
10.22	Transferring files	524
11	Quick reference for the WiPy	525
11.1	General information about the WiPy	525
11.1.1	No floating point support	525
11.1.2	Before applying power	525
11.1.3	WLAN default behaviour	526
11.1.4	Telnet REPL	526
11.1.5	Local file system and FTP access	526
11.1.6	FileZilla settings	526
11.1.7	Upgrading the firmware Over The Air	526
11.1.8	Boot modes and safe boot	527
11.1.9	The heartbeat LED	527
11.1.10	Details on sleep modes	528
11.1.11	Additional details for machine.Pin	528
11.1.12	Additional details for machine.I2C	529
11.1.13	Known issues	529
11.2	WiPy tutorials and examples	531
11.2.1	Introduction to the WiPy	531
11.2.2	Getting a MicroPython REPL prompt	532
11.2.3	Getting started with Blynk and the WiPy	534

11.2.4	WLAN step by step	534
11.2.5	Hardware timers	536
11.2.6	Reset and boot modes	537
11.3	General board control (including sleep modes)	538
11.4	Pins and GPIO	538
11.5	Timers	538
11.6	PWM (pulse width modulation)	539
11.7	ADC (analog to digital conversion)	539
11.8	UART (serial bus)	539
11.9	SPI bus	539
11.10	I2C bus	540
11.11	Watchdog timer (WDT)	540
11.12	Real time clock (RTC)	540
11.13	SD card	541
11.14	WLAN (WiFi)	541
11.15	Telnet and FTP server	541
11.16	Heart beat LED	542
12	Quick reference for the UNIX and Windows ports	543
12.1	Command line options	543
12.2	Environment variables	544
13	Quick reference for the Zephyr port	545
13.1	General information about the Zephyr port	545
13.1.1	Multitude of boards	545
13.2	MicroPython tutorial for the Zephyr port	545
13.2.1	Getting started with MicroPython on the Zephyr port	545
13.2.2	Getting a MicroPython REPL prompt	546
13.2.3	Filesystems and Storage	547
13.2.4	GPIO Pins	548
13.3	Running MicroPython	549
13.4	Delay and timing	549
13.5	Pins and GPIO	549
13.6	PWM	549
13.7	Hardware I2C bus	550
13.8	Hardware SPI bus	550
13.9	Disk Access	551
13.10	Flash Area	551
13.11	Sensor	551
14	Quick reference for the Renesas RA	553
14.1	General information about Renesas RA port	553
14.1.1	Overview	553
14.2	MicroPython tutorial for Renesas RA port	554
14.2.1	Getting started with MicroPython on the Renesas RA	554
14.2.2	Using peripherals	555
14.2.3	Write a program in internal file system	556
14.2.4	Reset and boot mode	557
14.2.5	Trouble Shooting	558
14.3	Installing MicroPython	558
14.4	General board control	558
14.5	Delay and timing	559
14.6	Timers	559
14.7	Pins and GPIO	559

14.8	UART (serial bus)	560
14.9	Real time clock (RTC)	561
14.10	ADC (analog to digital conversion)	562
14.11	SPI bus	562
14.12	I2C bus	563
14.13	PWM (pulse width modulation)	564
14.14	WDT (Watchdog timer)	564
14.15	SDCard	564
14.16	OneWire driver	565
14.17	NeoPixel and APA106 driver	565
15	Quick reference for the SAMD21/SAMD51 family	567
15.1	General information about the SAMD port	567
15.1.1	Multitude of boards	568
15.1.2	Technical specifications and SoC data sheets	568
15.2	Getting started with MicroPython on the SAMD	569
15.2.1	Requirements	569
15.2.2	Powering the board	569
15.2.3	Getting the firmware	569
15.2.4	Deploying the firmware	570
15.2.5	Serial prompt	570
15.2.6	Troubleshooting installation problems	570
15.3	Pinout for the SAMD machine modules	570
15.3.1	Adafruit ItsyBitsy M0 Express pin assignment table	570
15.3.2	Adafruit ItsyBitsy M4 Express pin assignment table	574
15.3.3	Adafruit Feather M4 Express pin assignment table	577
15.3.4	Adafruit Metro M4 Airlift pin assignment table	580
15.3.5	SEEED XIAO pin assignment table	583
15.3.6	Adafruit Feather M0 Express pin assignment table	585
15.3.7	Adafruit Trinket M0 pin assignment table	587
15.3.8	Adafruit QT PY pin assignment table	590
15.3.9	Adafruit NeoKey Trinkey pin assignment table	591
15.3.10	SparkFun Redboard Turbo assignment table	591
15.3.11	SparkFun SAMD21 Dev Breakout assignment table	593
15.3.12	SAMD21 Xplained PRO pin assignment table	596
15.3.13	Minisam M4 pin assignment table	598
15.3.14	Seeed WIO Terminal pin assignment table	600
15.3.15	SparkFun SAMD51 Thing Plus pin assignment table	606
15.3.16	Generic SAMD21x18 pin assignment table	608
15.3.17	Generic SAMD51x19 and SAM51x20 pin assignment table	611
15.3.18	Scripts for creating the pin assignment tables	616
15.4	Installing MicroPython	617
15.5	General board control	617
15.6	Delay and timing	618
15.7	Clock and time	618
15.8	Timers	618
15.9	Pins and GPIO	619
15.10	UART (serial bus)	619
15.11	PWM (pulse width modulation)	620
15.11.1	PWM Constructor	620
15.11.2	PWM Methods	621
15.11.3	PWM Pin Assignment	621
15.12	ADC (analog to digital conversion)	621
15.12.1	ADC Constructor	621

15.12.2 ADC Methods	622
15.13 DAC (digital to analog conversion)	622
15.13.1 DAC Constructor	622
15.13.2 DAC Methods	622
15.14 Software SPI bus	622
15.15 Hardware SPI bus	623
15.16 Software I2C bus	623
15.17 Hardware I2C bus	624
15.18 OneWire driver	624
15.19 DHT driver	625
15.20 Driving an APA102 LED	625
15.21 Driving a Neopixel LED	625
15.22 Transferring files	626
Python Module Index	627

MICROPYTHON LIBRARIES

Warning

Important summary of this section

- MicroPython provides built-in modules that mirror the functionality of the *Python standard library* (e.g. *os*, *time*), as well as *MicroPython-specific modules* (e.g. *bluetooth*, *machine*).
- Most Python standard library modules implement a subset of the functionality of the equivalent Python module, and in a few cases provide some MicroPython-specific extensions (e.g. *array*, *os*)
- Due to resource constraints or other limitations, some ports or firmware versions may not include all the functionality documented here.
- To allow for extensibility, some built-in modules can be *extended from Python code* loaded onto the device filesystem.

This chapter describes modules (function and class libraries) which are built into MicroPython. This documentation in general aspires to describe all modules and functions/classes which are implemented in the MicroPython project. However, MicroPython is highly configurable, and each port to a particular board/embedded system may include only a subset of the available MicroPython libraries.

With that in mind, please be warned that some functions/classes in a module (or even the entire module) described in this documentation **may be unavailable** in a particular build of MicroPython on a particular system. The best place to find general information of the availability/non-availability of a particular feature is the “General Information” section which contains information pertaining to a specific *MicroPython port*.

On some ports you are able to discover the available, built-in libraries that can be imported by entering the following at the *REPL*:

```
help('modules')
```

Beyond the built-in libraries described in this documentation, many more modules from the Python standard library, as well as further MicroPython extensions to it, can be found in *micropython-lib*.

1.1 Python standard libraries and micro-libraries

The following standard Python libraries have been “micro-ified” to fit in with the philosophy of MicroPython. They provide the core functionality of that module and are intended to be a drop-in replacement for the standard Python library.

1.1.1 array – arrays of numeric data

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [array](#).

Supported format codes: b, B, h, H, i, I, l, L, q, Q, f, d (the latter 2 depending on the floating-point support).

Classes

class `array.array` (*typecode*`[, iterable]`)

Create array with elements of given type. Initial contents of the array are given by *iterable*. If it is not provided, an empty array is created.

In addition to the methods below, array objects also implement the buffer protocol. This means the contents of the entire array can be accessed as raw bytes via a [memoryview](#) or other interfaces which use this protocol.

append (*val*)

Append new element *val* to the end of array, growing it.

extend (*iterable*)

Append new elements as contained in *iterable* to the end of array, growing it.

__getitem__ (*index*)

Indexed read of the array, called as `a[index]` (where *a* is an array). Returns a value if *index* is an `int` and an array if *index* is a slice. Negative indices count from the end and `IndexError` is thrown if the index is out of range.

Note: `__getitem__` cannot be called directly (`a.__getitem__(index)` fails) and is not present in `__dict__`, however `a[index]` does work.

__setitem__ (*index, value*)

Indexed write into the array, called as `a[index] = value` (where *a* is an array). *value* is a single value if *index* is an `int` and an array if *index* is a slice. Negative indices count from the end and `IndexError` is thrown if the index is out of range.

Note: `__setitem__` cannot be called directly (`a.__setitem__(index, value)` fails) and is not present in `__dict__`, however `a[index] = value` does work.

__len__ ()

Returns the number of items in the array, called as `len(a)` (where *a* is an array).

Note: `__len__` cannot be called directly (`a.__len__()` fails) and the method is not present in `__dict__`, however `len(a)` does work.

__add__ (*other*)

Return a new array that is the concatenation of the array with *other*, called as `a + other` (where *a* and *other* are both arrays).

Note: `__add__` cannot be called directly (`a.__add__(other)` fails) and is not present in `__dict__`, however `a + other` does work.

__iadd__ (*other*)

Concatenates the array with *other* in-place, called as `a += other` (where *a* and *other* are both arrays). Equivalent to `extend(other)`.

Note: `__iadd__` cannot be called directly (`a.__iadd__(other)` fails) and is not present in `__dict__`, however `a += other` does work.

`__repr__()`

Returns the string representation of the array, called as `str(a)` or `repr(a)` (where `a` is an array). Returns the string `"array(<type>, [<elements>])"`, where `<type>` is the type code letter for the array and `<elements>` is a comma separated list of the elements of the array.

Note: `__repr__` cannot be called directly (`a.__repr__()` fails) and is not present in `__dict__`, however `str(a)` and `repr(a)` both work.

1.1.2 `asyncio` — asynchronous I/O scheduler

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [asyncio](#)

Example:

```
import asyncio

async def blink(led, period_ms):
    while True:
        led.on()
        await asyncio.sleep_ms(5)
        led.off()
        await asyncio.sleep_ms(period_ms)

async def main(led1, led2):
    asyncio.create_task(blink(led1, 700))
    asyncio.create_task(blink(led2, 400))
    await asyncio.sleep_ms(10_000)

# Running on a pyboard
from pyb import LED
asyncio.run(main(LED(1), LED(2)))

# Running on a generic board
from machine import Pin
asyncio.run(main(Pin(1), Pin(2)))
```

Core functions

`asyncio.create_task(coro)`

Create a new task from the given coroutine and schedule it to run.

Returns the corresponding `Task` object.

`asyncio.current_task()`

Return the `Task` object associated with the currently running task.

`asyncio.run(coro)`

Create a new task from the given coroutine and run it until it completes.

Returns the value returned by `coro`.

`asyncio.sleep(t)`

Sleep for `t` seconds (can be a float).

This is a coroutine.

`asyncio.sleep_ms(t)`

Sleep for *t* milliseconds.

This is a coroutine, and a MicroPython extension.

Additional functions

`asyncio.wait_for(awaitable, timeout)`

Wait for the *awaitable* to complete, but cancel it if it takes longer than *timeout* seconds. If *awaitable* is not a task then a task will be created from it.

If a timeout occurs, it cancels the task and raises `asyncio.TimeoutError`: this should be trapped by the caller. The task receives `asyncio.CancelledError` which may be ignored or trapped using `try...except` or `try...finally` to run cleanup code.

Returns the return value of *awaitable*.

This is a coroutine.

`asyncio.wait_for_ms(awaitable, timeout)`

Similar to *wait_for* but *timeout* is an integer in milliseconds.

This is a coroutine, and a MicroPython extension.

`asyncio.gather(*awaitables, return_exceptions=False)`

Run all *awaitables* concurrently. Any *awaitables* that are not tasks are promoted to tasks.

Returns a list of return values of all *awaitables*.

This is a coroutine.

class Task

class `asyncio.Task`

This object wraps a coroutine into a running task. Tasks can be waited on using `await task`, which will wait for the task to complete and return the return value of the task.

Tasks should not be created directly, rather use *create_task* to create them.

`Task.cancel()`

Cancel the task by injecting `asyncio.CancelledError` into it. The task may ignore this exception. Cleanup code may be run by trapping it, or via `try...finally`.

class Event

class `asyncio.Event`

Create a new event which can be used to synchronise tasks. Events start in the cleared state.

`Event.is_set()`

Returns `True` if the event is set, `False` otherwise.

`Event.set()`

Set the event. Any tasks waiting on the event will be scheduled to run.

Note: This must be called from within a task. It is not safe to call this from an IRQ, scheduler callback, or other thread. See *ThreadSafeFlag*.

`Event.clear()`

Clear the event.

`Event.wait()`

Wait for the event to be set. If the event is already set then it returns immediately.

This is a coroutine.

class ThreadSafeFlag

class `asyncio.ThreadSafeFlag`

Create a new flag which can be used to synchronise a task with code running outside the asyncio loop, such as other threads, IRQs, or scheduler callbacks. Flags start in the cleared state.

`ThreadSafeFlag.set()`

Set the flag. If there is a task waiting on the flag, it will be scheduled to run.

`ThreadSafeFlag.clear()`

Clear the flag. This may be used to ensure that a possibly previously-set flag is clear before waiting for it.

`ThreadSafeFlag.wait()`

Wait for the flag to be set. If the flag is already set then it returns immediately. The flag is automatically reset upon return from `wait`.

A flag may only be waited on by a single task at a time.

This is a coroutine.

class Lock

class `asyncio.Lock`

Create a new lock which can be used to coordinate tasks. Locks start in the unlocked state.

In addition to the methods below, locks can be used in an `async with` statement.

`Lock.locked()`

Returns `True` if the lock is locked, otherwise `False`.

`Lock.acquire()`

Wait for the lock to be in the unlocked state and then lock it in an atomic way. Only one task can acquire the lock at any one time.

This is a coroutine.

`Lock.release()`

Release the lock. If any tasks are waiting on the lock then the next one in the queue is scheduled to run and the lock remains locked. Otherwise, no tasks are waiting and the lock becomes unlocked.

TCP stream connections

`asyncio.open_connection(host, port, ssl=None)`

Open a TCP connection to the given *host* and *port*. The *host* address will be resolved using `socket.getaddrinfo`, which is currently a blocking call. If *ssl* is a `ssl.SSLContext` object, this context is used to create the transport; if *ssl* is `True`, a default context is used.

Returns a pair of streams: a reader and a writer stream. Will raise a socket-specific `OSError` if the host could not be resolved or if the connection could not be made.

This is a coroutine.

`asyncio.start_server` (*callback*, *host*, *port*, *backlog*=5, *ssl*=None)

Start a TCP server on the given *host* and *port*. The *callback* will be called with incoming, accepted connections, and be passed 2 arguments: reader and writer streams for the connection.

If *ssl* is a `ssl.SSLContext` object, this context is used to create the transport.

Returns a `Server` object.

This is a coroutine.

class `asyncio.Stream`

This represents a TCP stream connection. To minimise code this class implements both a reader and a writer, and both `StreamReader` and `StreamWriter` alias to this class.

`Stream.get_extra_info` (*v*)

Get extra information about the stream, given by *v*. The valid values for *v* are: `peername`.

`Stream.close` ()

Close the stream.

`Stream.wait_closed` ()

Wait for the stream to close.

This is a coroutine.

`Stream.read` (*n*=-1)

Read up to *n* bytes and return them. If *n* is not provided or -1 then read all bytes until EOF. The returned value will be an empty bytes object if EOF is encountered before any bytes are read.

This is a coroutine.

`Stream.readinto` (*buf*)

Read up to *n* bytes into *buf* with *n* being equal to the length of *buf*.

Return the number of bytes read into *buf*.

This is a coroutine, and a MicroPython extension.

`Stream.readexactly` (*n*)

Read exactly *n* bytes and return them as a bytes object.

Raises an `EOFError` exception if the stream ends before reading *n* bytes.

This is a coroutine.

`Stream.readline` ()

Read a line and return it.

This is a coroutine.

`Stream.write` (*buf*)

Accumulated *buf* to the output buffer. The data is only flushed when `Stream.drain` is called. It is recommended to call `Stream.drain` immediately after calling this function.

`Stream.drain` ()

Drain (write) all buffered output data out to the stream.

This is a coroutine.

class `asyncio.Server`

This represents the server class returned from `start_server`. It can be used in an `async with` statement to close the server upon exit.

`Server.close()`

Close the server.

`Server.wait_closed()`

Wait for the server to close.

This is a coroutine.

Event Loop

`asyncio.get_event_loop()`

Return the event loop used to schedule and run tasks. See [Loop](#).

`asyncio.new_event_loop()`

Reset the event loop and return it.

Note: since MicroPython only has a single event loop this function just resets the loop's state, it does not create a new one.

class `asyncio.Loop`

This represents the object which schedules and runs tasks. It cannot be created, use [get_event_loop](#) instead.

`Loop.create_task(coro)`

Create a task from the given *coro* and return the new *Task* object.

`Loop.run_forever()`

Run the event loop until `stop()` is called.

`Loop.run_until_complete(awaitable)`

Run the given *awaitable* until it completes. If *awaitable* is not a task then it will be promoted to one.

`Loop.stop()`

Stop the event loop.

`Loop.close()`

Close the event loop.

`Loop.set_exception_handler(handler)`

Set the exception handler to call when a Task raises an exception that is not caught. The *handler* should accept two arguments: (loop, context).

`Loop.get_exception_handler()`

Get the current exception handler. Returns the handler, or `None` if no custom handler is set.

`Loop.default_exception_handler(context)`

The default exception handler that is called.

`Loop.call_exception_handler(context)`

Call the current exception handler. The argument *context* is passed through and is a dictionary containing keys: 'message', 'exception', 'future'.

1.1.3 binascii – binary/ASCII conversions

This module implements a subset of the corresponding *CPython* module, as described below. For more information, refer to the original *CPython* documentation: [binascii](#).

This module implements conversions between binary data and various encodings of it in ASCII form (in both directions).

Functions

`binascii.hexlify(data[, sep])`

Convert the bytes in the *data* object to a hexadecimal representation. Returns a bytes object.

If the additional argument *sep* is supplied it is used as a separator between hexadecimal values.

`binascii.unhexlify(data)`

Convert hexadecimal data to binary representation. Returns bytes string. (i.e. inverse of `hexlify`)

`binascii.a2b_base64(data)`

Decode base64-encoded data, ignoring invalid characters in the input. Conforms to [RFC 2045 s.6.8](#). Returns a bytes object.

`binascii.b2a_base64(data, *, newline=True)`

Encode binary data in base64 format, as in [RFC 3548](#). Returns the encoded data followed by a newline character if *newline* is true, as a bytes object.

`binascii.crc32(data[, value])`

Compute CRC-32, the 32-bit checksum of *data*, starting with an initial CRC of *value*. The default initial CRC is zero. The algorithm is consistent with the ZIP file checksum.

1.1.4 builtins – builtin functions and exceptions

All builtin functions and exceptions are described here. They are also available via `builtins` module.

Functions and types

`abs()`

`all()`

`any()`

`bin()`

`class bool`

`class bytearray`

See CPython documentation: [bytearray](#).

`class bytes`

See CPython documentation: [bytes](#).

`callable()`

`chr()`

`classmethod()`

`compile()`

`class complex`

`delattr(obj, name)`

The argument *name* should be a string, and this function deletes the named attribute from the object given by *obj*.

`class dict`

`dir()`
`divmod()`
`enumerate()`
`eval()`
`exec()`
`filter()`
`class float`
`class frozenset`
`getattr()`
`globals()`
`hasattr()`
`hash()`
`hex()`
`id()`
`input()`
`class int`

`classmethod from_bytes (bytes, byteorder)`

In MicroPython, *byteorder* parameter must be positional (this is compatible with CPython).

`to_bytes (size, byteorder)`

In MicroPython, *byteorder* parameter must be positional (this is compatible with CPython).

Note

The optional `signed` kwarg from CPython is not supported. MicroPython currently converts negative integers as signed, and positive as unsigned. (*Details.*)

`isinstance()`
`issubclass()`
`iter()`
`len()`
`class list`
`locals()`
`map()`
`max()`

`class memoryview`

See CPython documentation: `memoryview`.

`min()`

`next()`

`class object`

`oct()`

`open()`

`ord()`

`pow()`

`print()`

`property()`

`range()`

`repr()`

`reversed()`

`round()`

`class set`

`setattr()`

`class slice`

The *slice* builtin is the type that slice objects have.

`sorted()`

`staticmethod()`

`class str`

`sum()`

`super()`

`class tuple`

`type()`

`zip()`

Exceptions

`exception AssertionError`

`exception AttributeError`

`exception Exception`

exception ImportError

exception IndexError

exception KeyboardInterrupt

See CPython documentation: [KeyboardInterrupt](#).

See also in the context of *Soft Bricking (failure to boot)*.

exception KeyError

exception MemoryError

exception NameError

exception NotImplementedError

exception OSError

exception RuntimeError

exception StopIteration

exception SyntaxError

exception SystemExit

See CPython documentation: [SystemExit](#).

On non-embedded ports (i.e. Windows and Unix), an unhandled `SystemExit` exits the MicroPython process in a similar way to CPython.

On embedded ports, an unhandled `SystemExit` currently causes a *Soft Reset* of MicroPython.

exception TypeError

See CPython documentation: [TypeError](#).

exception ValueError

exception ZeroDivisionError

1.1.5 `cmath` – mathematical functions for complex numbers

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [cmath](#).

The `cmath` module provides some basic mathematical functions for working with complex numbers.

Availability: not available on WiPy and ESP8266. Floating point support required for this module.

Functions

`cmath.cos(z)`

Return the cosine of `z`.

`cmath.exp(z)`

Return the exponential of `z`.

`cmath.log(z)`

Return the natural logarithm of `z`. The branch cut is along the negative real axis.

`cmath.log10(z)`

Return the base-10 logarithm of *z*. The branch cut is along the negative real axis.

`cmath.phase(z)`

Returns the phase of the number *z*, in the range $(-\pi, +\pi]$.

`cmath.polar(z)`

Returns, as a tuple, the polar form of *z*.

`cmath.rect(r, phi)`

Returns the complex number with modulus *r* and phase *phi*.

`cmath.sin(z)`

Return the sine of *z*.

`cmath.sqrt(z)`

Return the square-root of *z*.

Constants

`cmath.e`

base of the natural logarithm

`cmath.pi`

the ratio of a circle's circumference to its diameter

1.1.6 collections – collection and container types

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [collections](#).

This module implements advanced collection and container types to hold/accumulate various objects.

Classes

class `collections.deque(iterable, maxlen[, flags])`

Deques (double-ended queues) are a list-like container that support $O(1)$ appends and pops from either side of the deque. New deques are created using the following arguments:

- *iterable* is an iterable used to populate the deque when it is created. It can be an empty tuple or list to create a deque that is initially empty.
- *maxlen* must be specified and the deque will be bounded to this maximum length. Once the deque is full, any new items added will discard items from the opposite end.
- The optional *flags* can be 1 to check for overflow when adding items.

Deque objects support *bool*, *len*, iteration and subscript load and store. They also have the following methods:

append(*x*)

Add *x* to the right side of the deque. Raises `IndexError` if overflow checking is enabled and there is no more room in the queue.

appendleft(*x*)

Add *x* to the left side of the deque. Raises `IndexError` if overflow checking is enabled and there is no more room in the queue.

pop()

Remove and return an item from the right side of the deque. Raises `IndexError` if no items are present.

popleft()

Remove and return an item from the left side of the deque. Raises `IndexError` if no items are present.

extend(iterable)

Extend the deque by appending all the items from *iterable* to the right of the deque. Raises `IndexError` if overflow checking is enabled and there is no more room in the deque.

collections.namedtuple(name, fields)

This is factory function to create a new `namedtuple` type with a specific name and set of fields. A `namedtuple` is a subclass of `tuple` which allows to access its fields not just by numeric index, but also with an attribute access syntax using symbolic field names. *Fields* is a sequence of strings specifying field names. For compatibility with CPython it can also be a string with space-separated field named (but this is less efficient). Example of use:

```
from collections import namedtuple

MyTuple = namedtuple("MyTuple", ("id", "name"))
t1 = MyTuple(1, "foo")
t2 = MyTuple(2, "bar")
print(t1.name)
assert t2.name == t2[1]
```

class collections.OrderedDict(...)

`dict` type subclass which remembers and preserves the order of keys added. When ordered dict is iterated over, keys/items are returned in the order they were added:

```
from collections import OrderedDict

# To make benefit of ordered keys, OrderedDict should be initialized
# from sequence of (key, value) pairs.
d = OrderedDict([("z", 1), ("a", 2)])
# More items can be added as usual
d["w"] = 5
d["b"] = 3
for k, v in d.items():
    print(k, v)
```

Output:

```
z 1
a 2
w 5
b 3
```

1.1.7 `errno` – system error codes

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `errno`.

This module provides access to symbolic error codes for `OSError` exception. A particular inventory of codes depends on *MicroPython port*.

Constants

EEXIST, EAGAIN, etc.

Error codes, based on ANSI C/POSIX standard. All error codes start with “E”. As mentioned above, inventory of the codes depends on *MicroPython port*. Errors are usually accessible as `exc.errno` where `exc` is an instance of *OSError*. Usage example:

```
try:
    os.mkdir("my_dir")
except OSError as exc:
    if exc.errno == errno.EEXIST:
        print("Directory already exists")
```

errno.errorcode

Dictionary mapping numeric error codes to strings with symbolic error code (see above):

```
>>> print(errno.errorcode[errno.EEXIST])
EEXIST
```

1.1.8 gc – control the garbage collector

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [gc](#).

Functions

gc.enable()

Enable automatic garbage collection.

gc.disable()

Disable automatic garbage collection. Heap memory can still be allocated, and garbage collection can still be initiated manually using `gc.collect()`.

gc.collect()

Run a garbage collection.

gc.mem_alloc()

Return the number of bytes of heap RAM that are allocated by Python code.

Difference to CPython

This function is MicroPython extension.

gc.mem_free()

Return the number of bytes of heap RAM that is available for Python code to allocate, or -1 if this amount is not known.

Difference to CPython

This function is MicroPython extension.

```
gc.threshold([amount])
```

Set or query the additional GC allocation threshold. Normally, a collection is triggered only when a new allocation cannot be satisfied, i.e. on an out-of-memory (OOM) condition. If this function is called, in addition to OOM, a collection will be triggered each time after *amount* bytes have been allocated (in total, since the previous time such an amount of bytes have been allocated). *amount* is usually specified as less than the full heap size, with the intention to trigger a collection earlier than when the heap becomes exhausted, and in the hope that an early collection will prevent excessive memory fragmentation. This is a heuristic measure, the effect of which will vary from application to application, as well as the optimal value of the *amount* parameter.

Calling the function without argument will return the current value of the threshold. A value of -1 means a disabled allocation threshold.

Difference to CPython

This function is a MicroPython extension. CPython has a similar function - `set_threshold()`, but due to different GC implementations, its signature and semantics are different.

1.1.9 gzip – gzip compression & decompression

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [gzip](#).

This module allows compression and decompression of binary data with the [DEFLATE algorithm](#) used by the gzip file format.

Note

Prefer to use `deflate.DeflateIO` instead of the functions in this module as it provides a streaming interface to compression and decompression which is convenient and more memory efficient when working with reading or writing compressed data to a file, socket, or stream.

Availability:

- This module is **not present by default** in official MicroPython firmware releases as it duplicates functionality available in the `deflate` module.
- A copy of this module can be installed (or frozen) from [micropython-lib](#) (source). See [Package management](#) for more information. This documentation describes that module.
- Compression support will only be available if compression support is enabled in the built-in `deflate` module.

Functions

```
gzip.open(filename, mode, /)
```

Wrapper around built-in `open()` returning a `GzipFile` instance.

```
gzip.decompress(data, /)
```

Decompresses *data* into a bytes object.

```
gzip.compress(data, /)
```

Compresses *data* into a bytes object.

Classes

`class gzip.GzipFile(*, fileobj, mode)`

This class can be used to wrap a *fileobj* which is any *stream-like* object such as a file, socket, or stream (including `io.BytesIO`). It is itself a stream and implements the standard `read/readinto/write/close` methods.

When the *mode* argument is `"rb"`, reads from the `GzipFile` instance will decompress the data in the underlying stream and return decompressed data.

If compression support is enabled then the *mode* argument can be set to `"wb"`, and writes to the `GzipFile` instance will be compressed and written to the underlying stream.

By default the `GzipFile` class will read and write data using the gzip file format, including a header and footer with checksum and a window size of 512 bytes.

The **file**, **compresslevel**, and **mtime** arguments are not supported. **fileobj** and **mode** must always be specified as keyword arguments.

Examples

A typical use case for `gzip.GzipFile` is to read or write a compressed file from storage:

```
import gzip

# Reading:
with open("data.gz", "rb") as f:
    with gzip.GzipFile(fileobj=f, mode="rb") as g:
        # Use g.read(), g.readinto(), etc.

# Same, but using gzip.open:
with gzip.open("data.gz", "rb") as f:
    # Use f.read(), f.readinto(), etc.

# Writing:
with open("data.gz", "wb") as f:
    with gzip.GzipFile(fileobj=f, mode="wb") as g:
        # Use g.write(...) etc

# Same, but using gzip.open:
with gzip.open("data.gz", "wb") as f:
    # Use f.write(...) etc

# Write a dictionary as JSON in gzip format, with a
# small (64 byte) window size.
config = { ... }
with gzip.open("config.gz", "wb") as f:
    json.dump(config, f)
```

For guidance on working with gzip sources and choosing the window size see the note at the *end of the deflate documentation*.

1.1.10 hashlib – hashing algorithms

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `hashlib`.

This module implements binary data hashing algorithms. The exact inventory of available algorithms depends on a board. Among the algorithms which may be implemented:

- SHA256 - The current generation, modern hashing algorithm (of SHA2 series). It is suitable for cryptographically-secure purposes. Included in the MicroPython core and any board is recommended to provide this, unless it has particular code size constraints.
- SHA1 - A previous generation algorithm. Not recommended for new usages, but SHA1 is a part of number of Internet standards and existing applications, so boards targeting network connectivity and interoperability will try to provide this.
- MD5 - A legacy algorithm, not considered cryptographically secure. Only selected boards, targeting interoperability with legacy applications, will offer this.

Constructors

`class hashlib.sha256([data])`

Create an SHA256 hasher object and optionally feed `data` into it.

`class hashlib.sha1([data])`

Create an SHA1 hasher object and optionally feed `data` into it.

`class hashlib.md5([data])`

Create an MD5 hasher object and optionally feed `data` into it.

Methods

`hash.update(data)`

Feed more binary data into hash.

`hash.digest()`

Return hash for all data passed through hash, as a bytes object. After this method is called, more data cannot be fed into the hash any longer.

`hash.hexdigest()`

This method is NOT implemented. Use `binascii.hexlify(hash.digest())` to achieve a similar effect.

1.1.11 heapq – heap queue algorithm

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [heapq](#).

This module implements the [min heap queue algorithm](#).

A heap queue is essentially a list that has its elements stored in such a way that the first item of the list is always the smallest.

Functions

`heapq.heappush(heap, item)`

Push the `item` onto the heap.

`heapq.heappop(heap)`

Pop the first item from the heap, and return it. Raise `IndexError` if heap is empty.

The returned item will be the smallest item in the heap.

`heapq.heapify(x)`

Convert the list `x` into a heap. This is an in-place operation.

1.1.12 `io` – input/output streams

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `io`.

This module contains additional types of `stream` (file-like) objects and helper functions.

Conceptual hierarchy

Difference to CPython

Conceptual hierarchy of stream base classes is simplified in MicroPython, as described in this section.

(Abstract) base stream classes, which serve as a foundation for behaviour of all the concrete classes, adhere to few dichotomies (pair-wise classifications) in CPython. In MicroPython, they are somewhat simplified and made implicit to achieve higher efficiencies and save resources.

An important dichotomy in CPython is unbuffered vs buffered streams. In MicroPython, all streams are currently unbuffered. This is because all modern OSes, and even many RTOSes and filesystem drivers already perform buffering on their side. Adding another layer of buffering is counter-productive (an issue known as “bufferbloat”) and takes precious memory. Note that there still cases where buffering may be useful, so we may introduce optional buffering support at a later time.

But in CPython, another important dichotomy is tied with “bufferedness” - it’s whether a stream may incur short read/writes or not. A short read is when a user asks e.g. 10 bytes from a stream, but gets less, similarly for writes. In CPython, unbuffered streams are automatically short operation susceptible, while buffered are guarantee against them. The no short read/writes is an important trait, as it allows to develop more concise and efficient programs - something which is highly desirable for MicroPython. So, while MicroPython doesn’t support buffered streams, it still provides for no-short-operations streams. Whether there will be short operations or not depends on each particular class’ needs, but developers are strongly advised to favour no-short-operations behaviour for the reasons stated above. For example, MicroPython sockets are guaranteed to avoid short read/writes. Actually, at this time, there is no example of a short-operations stream class in the core, and one would be a port-specific class, where such a need is governed by hardware peculiarities.

The no-short-operations behaviour gets tricky in case of non-blocking streams, blocking vs non-blocking behaviour being another CPython dichotomy, fully supported by MicroPython. Non-blocking streams never wait for data either to arrive or be written - they read/write whatever possible, or signal lack of data (or ability to write data). Clearly, this conflicts with “no-short-operations” policy, and indeed, a case of non-blocking buffered (and this no-short-ops) streams is convoluted in CPython - in some places, such combination is prohibited, in some it’s undefined or just not documented, in some cases it raises verbose exceptions. The matter is much simpler in MicroPython: non-blocking stream are important for efficient asynchronous operations, so this property prevails on the “no-short-ops” one. So, while blocking streams will avoid short reads/writes whenever possible (the only case to get a short read is if end of file is reached, or in case of error (but errors don’t return short data, but raise exceptions)), non-blocking streams may produce short data to avoid blocking the operation.

The final dichotomy is binary vs text streams. MicroPython of course supports these, but while in CPython text streams are inherently buffered, they aren’t in MicroPython. (Indeed, that’s one of the cases for which we may introduce buffering support.)

Note that for efficiency, MicroPython doesn’t provide abstract base classes corresponding to the hierarchy above, and it’s not possible to implement, or subclass, a stream class in pure Python.

Functions

`io.open(name, mode='r', **kwargs)`

Open a file. Builtin `open()` function is aliased to this function. All ports (which provide access to file system) are required to support `mode` parameter, but support for other arguments vary by port.

Classes

`class io.StringIO([string])`

`class io.BytesIO([string])`

In-memory file-like objects for input/output. *StringIO* is used for text-mode I/O (similar to a normal file opened with “t” modifier). *BytesIO* is used for binary-mode I/O (similar to a normal file opened with “b” modifier). Initial contents of file-like objects can be specified with *string* parameter (should be normal string for *StringIO* or bytes object for *BytesIO*). All the usual file methods like `read()`, `write()`, `seek()`, `flush()`, `close()` are available on these objects, and additionally, a following method:

`getvalue()`

Get the current contents of the underlying buffer which holds data.

`class io.StringIO(alloc_size)`

`class io.BytesIO(alloc_size)`

Create an empty *StringIO/BytesIO* object, preallocated to hold up to *alloc_size* number of bytes. That means that writing that amount of bytes won’t lead to reallocation of the buffer, and thus won’t hit out-of-memory situation or lead to memory fragmentation. These constructors are a MicroPython extension and are recommended for usage only in special cases and in system-level libraries, not for end-user applications.

Difference to CPython

These constructors are a MicroPython extension.

1.1.13 json – JSON encoding and decoding

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `json`.

This modules allows to convert between Python objects and the JSON data format.

Functions

`json.dump(obj, stream, separators=None)`

Serialise *obj* to a JSON string, writing it to the given *stream*.

If specified, *separators* should be an (*item_separator*, *key_separator*) tuple. The default is (`' , '`, `' : '`). To get the most compact JSON representation, you should specify (`' , '`, `' : '`) to eliminate whitespace.

`json.dumps(obj, separators=None)`

Return *obj* represented as a JSON string.

The arguments have the same meaning as in *dump*.

`json.load(stream)`

Parse the given *stream*, interpreting it as a JSON string and deserialising the data to a Python object. The resulting object is returned.

Parsing continues until end-of-file is encountered. A `ValueError` is raised if the data in *stream* is not correctly formed.

`json.loads(str)`

Parse the JSON *str* and return an object. Raises `ValueError` if the string is not correctly formed.

1.1.14 `marshal` – Python object serialization

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `marshal`.

This module implements conversion between Python objects and a binary format. The format is specific to MicroPython but does not depend on the machine architecture, so the data can be transferred and used on a different MicroPython instance, as long as the version of the binary data matches (it's currently versioned as the mpy file version, see *MicroPython .mpy files*).

Functions

`marshal.dumps(value, /)`

Convert the given *value* to binary format and return a corresponding `bytes` object.

Currently, code objects are the only supported values that can be converted.

`marshal.loads(data, /)`

Convert the given bytes-like *data* to its corresponding Python object, and return it.

1.1.15 `math` – mathematical functions

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `math`.

The `math` module provides some basic mathematical functions for working with floating-point numbers.

Note: On the pyboard, floating-point numbers have 32-bit precision.

Availability: not available on WiPy. Floating point support required for this module.

Functions

`math.acos(x)`

Return the inverse cosine of *x*.

`math.acosh(x)`

Return the inverse hyperbolic cosine of *x*.

`math.asin(x)`

Return the inverse sine of *x*.

`math.asinh(x)`

Return the inverse hyperbolic sine of *x*.

`math.atan(x)`

Return the inverse tangent of *x*.

`math.atan2(y, x)`

Return the principal value of the inverse tangent of *y/x*.

`math.atanh(x)`

Return the inverse hyperbolic tangent of x .

`math.ceil(x)`

Return an integer, being x rounded towards positive infinity.

`math.copysign(x, y)`

Return x with the sign of y .

`math.cos(x)`

Return the cosine of x .

`math.cosh(x)`

Return the hyperbolic cosine of x .

`math.degrees(x)`

Return radians x converted to degrees.

`math.erf(x)`

Return the error function of x .

`math.erfc(x)`

Return the complementary error function of x .

`math.exp(x)`

Return the exponential of x .

`math.expm1(x)`

Return $\exp(x) - 1$.

`math.fabs(x)`

Return the absolute value of x .

`math.floor(x)`

Return an integer, being x rounded towards negative infinity.

`math.fmod(x, y)`

Return the remainder of x/y .

`math.frexp(x)`

Decomposes a floating-point number into its mantissa and exponent. The returned value is the tuple (m, e) such that $x == m * 2^{**}e$ exactly. If $x == 0$ then the function returns $(0.0, 0)$, otherwise the relation $0.5 <= \text{abs}(m) < 1$ holds.

`math.gamma(x)`

Return the gamma function of x .

`math.isfinite(x)`

Return `True` if x is finite.

`math.isinf(x)`

Return `True` if x is infinite.

`math.isnan(x)`

Return `True` if x is not-a-number

`math.ldexp(x, exp)`

Return $x * (2^{**}exp)$.

`math.lgamma(x)`

Return the natural logarithm of the gamma function of x .

`math.log(x)`

`math.log(x, base)`

With one argument, return the natural logarithm of x .

With two arguments, return the logarithm of x to the given *base*.

`math.log10(x)`

Return the base-10 logarithm of x .

`math.log2(x)`

Return the base-2 logarithm of x .

`math.modf(x)`

Return a tuple of two floats, being the fractional and integral parts of x . Both return values have the same sign as x .

`math.pow(x, y)`

Returns x to the power of y .

`math.radians(x)`

Return degrees x converted to radians.

`math.sin(x)`

Return the sine of x .

`math.sinh(x)`

Return the hyperbolic sine of x .

`math.sqrt(x)`

Return the square root of x .

`math.tan(x)`

Return the tangent of x .

`math.tanh(x)`

Return the hyperbolic tangent of x .

`math.trunc(x)`

Return an integer, being x rounded towards 0.

Constants

`math.e`

base of the natural logarithm

`math.pi`

the ratio of a circle's circumference to its diameter

1.1.16 `os` – basic “operating system” services

This module implements a subset of the corresponding [CPython](#) module, as described below. For more information, refer to the original [CPython](#) documentation: `os`.

The `os` module contains functions for filesystem access and mounting, terminal redirection and duplication, and the `uname` and `urandom` functions.

General functions

`os.uname()`

Return a tuple (possibly a named tuple) containing information about the underlying machine and/or its operating system. The tuple has five fields in the following order, each of them being a string:

- `sysname` – the name of the underlying system
- `nodename` – the network name (can be the same as `sysname`)
- `release` – the version of the underlying system
- `version` – the MicroPython version and build date
- `machine` – an identifier for the underlying hardware (eg board, CPU)

`os.urandom(n)`

Return a bytes object with *n* random bytes. Whenever possible, it is generated by the hardware random number generator.

Filesystem access

`os.chdir(path)`

Change current directory.

`os.getcwd()`

Get the current directory.

`os.listdir([dir])`

This function returns an iterator which then yields tuples corresponding to the entries in the directory that it is listing. With no argument it lists the current directory, otherwise it lists the directory given by *dir*.

The tuples have the form *(name, type, inode[, size])*:

- *name* is a string (or bytes if *dir* is a bytes object) and is the name of the entry;
- *type* is an integer that specifies the type of the entry, with 0x4000 for directories and 0x8000 for regular files;
- *inode* is an integer corresponding to the inode of the file, and may be 0 for filesystems that don't have such a notion.
- Some platforms may return a 4-tuple that includes the entry's *size*. For file entries, *size* is an integer representing the size of the file or -1 if unknown. Its meaning is currently undefined for directory entries.

`os.listdir([dir])`

With no argument, list the current directory. Otherwise list the given directory.

`os.mkdir(path)`

Create a new directory.

`os.remove(path)`

Remove a file.

`os.rmdir(path)`

Remove a directory.

`os.rename(old_path, new_path)`

Rename a file.

`os.stat(path)`

Get the status of a file or directory.

`os.statvfs(path)`

Get the status of a filesystem.

Returns a tuple with the filesystem information in the following order:

- `f_bsize` – file system block size
- `f_frsize` – fragment size
- `f_blocks` – size of fs in `f_frsize` units
- `f_bfree` – number of free blocks
- `f_bavail` – number of free blocks for unprivileged users
- `f_files` – number of inodes
- `f_ffree` – number of free inodes
- `f_favail` – number of free inodes for unprivileged users
- `f_flag` – mount flags
- `f_namemax` – maximum filename length

Parameters related to inodes: `f_files`, `f_ffree`, `f_avail` and the `f_flags` parameter may return 0 as they can be unavailable in a port-specific implementation.

`os.sync()`

Sync all filesystems.

Terminal redirection and duplication

`os.dupterm(stream_object, index=0, /)`

Duplicate or switch the MicroPython terminal (the REPL) on the given *stream*-like object. The *stream_object* argument must be a native stream object, or derive from `io.IOBase` and implement the `readinto()` and `write()` methods. The stream should be in non-blocking mode and `readinto()` should return `None` if there is no data available for reading.

After calling this function all terminal output is repeated on this stream, and any input that is available on the stream is passed on to the terminal input.

The *index* parameter should be a non-negative integer and specifies which duplication slot is set. A given port may implement more than one slot (slot 0 will always be available) and in that case terminal input and output is duplicated on all the slots that are set.

If `None` is passed as the *stream_object* then duplication is cancelled on the slot given by *index*.

The function returns the previous stream-like object in the given slot.

Filesystem mounting

The following functions and classes have been moved to the `vfs` module. They are provided in this module only for backwards compatibility and will be removed in version 2 of MicroPython.

`os.mount(fsobj, mount_point, *, readonly)`

See `vfs.mount`.

`os.umount(mount_point)`

See `vfs.umount`.

`class os.VfsFat(block_dev)`

See `vfs.VfsFat`.

```
class os.VfsLfs1 (block_dev, readsize=32, progsz=32, lookahead=32)
```

See `vfs.VfsLfs1`.

```
class os.VfsLfs2 (block_dev, readsize=32, progsz=32, lookahead=32, mtime=True)
```

See `vfs.VfsLfs2`.

```
class os.VfsPosix (root=None)
```

See `vfs.VfsPosix`.

1.1.17 platform – access to underlying platform’s identifying data

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `platform`.

This module tries to retrieve as much platform-identifying data as possible. It makes this information available via function APIs.

Functions

```
platform.platform()
```

Returns a string identifying the underlying platform. This string is composed of several substrings in the following order, delimited by dashes (-):

- the name of the platform system (e.g. Unix, Windows or MicroPython)
- the MicroPython version
- the architecture of the platform
- the version of the underlying platform
- the concatenation of the name of the libc that MicroPython is linked to and its corresponding version.

For example, this could be "MicroPython-1.20.0-xtensa-IDFv4.2.4-with-newlib3.0.0".

```
platform.python_compiler()
```

Returns a string identifying the compiler used for compiling MicroPython.

```
platform.libc_ver()
```

Returns a tuple of strings (*lib*, *version*), where *lib* is the name of the libc that MicroPython is linked to, and *version* the corresponding version of this libc.

1.1.18 random – generate random numbers

This module implements a pseudo-random number generator (PRNG).

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `random`.

Note

The following notation is used for intervals:

- `()` are open interval brackets and do not include their endpoints. For example, `(0, 1)` means greater than 0 and less than 1. In set notation: `(0, 1) = {x | 0 < x < 1}`.
- `[]` are closed interval brackets which include all their limit points. For example, `[0, 1]` means greater than or equal to 0 and less than or equal to 1. In set notation: `[0, 1] = {x | 0 <= x <= 1}`.

Note

The `randrange()`, `randint()` and `choice()` functions are only available if the `MICROPY_PY_RANDOM_EXTRA_FUNCS` configuration option is enabled.

Functions for integers

`random.getrandbits(n)`

Return an integer with *n* random bits ($0 \leq n \leq 32$).

`random.randint(a, b)`

Return a random integer in the range [*a*, *b*].

`random.randrange(stop)`

`random.randrange(start, stop)`

`random.randrange(start, stop [, step ])`

The first form returns a random integer from the range $[0, stop)$. The second form returns a random integer from the range $[start, stop)$. The third form returns a random integer from the range $[start, stop)$ in steps of *step*. For instance, calling `randrange(1, 10, 2)` will return odd numbers between 1 and 9 inclusive.

Functions for floats

`random.random()`

Return a random floating point number in the range $[0.0, 1.0)$.

`random.uniform(a, b)`

Return a random floating point number *N* such that $a \leq N \leq b$ for $a \leq b$, and $b \leq N \leq a$ for $b < a$.

Other Functions

`random.seed(n=None, /)`

Initialise the random number generator module with the seed *n* which should be an integer. When no argument (or `None`) is passed in it will (if supported by the port) initialise the PRNG with a true random number (usually a hardware generated random number).

The `None` case only works if `MICROPY_PY_RANDOM_SEED_INIT_FUNC` is enabled by the port, otherwise it raises `ValueError`.

`random.choice(sequence)`

Chooses and returns one item at random from *sequence* (tuple, list or any object that supports the subscript operation).

1.1.19 `re` – simple regular expressions

This module implements a subset of the corresponding *CPython* module, as described below. For more information, refer to the original *CPython* documentation: `re`.

This module implements regular expression operations. Regular expression syntax supported is a subset of *CPython* `re` module (and actually is a subset of POSIX extended regular expressions).

Supported operators and special sequences are:

.

Match any character.

- [...] Match set of characters. Individual characters and ranges are supported, including negated sets (e.g. `[^a-c]`).
- ^ Match the start of the string.
- \$ Match the end of the string.
- ? Match zero or one of the previous sub-pattern.
- * Match zero or more of the previous sub-pattern.
- + Match one or more of the previous sub-pattern.
- ?? Non-greedy version of `?`, match zero or one, with the preference for zero.
- *? Non-greedy version of `*`, match zero or more, with the preference for the shortest match.
- +? Non-greedy version of `+`, match one or more, with the preference for the shortest match.
- | Match either the left-hand side or the right-hand side sub-patterns of this operator.
- (...) Grouping. Each group is capturing (a substring it captures can be accessed with `match.group()` method).
- \d Matches digit. Equivalent to `[0-9]`.
- \D Matches non-digit. Equivalent to `[^0-9]`.
- \s Matches whitespace. Equivalent to `[\t-\r]`.
- \S Matches non-whitespace. Equivalent to `[^\t-\r]`.
- \w Matches “word characters” (ASCII only). Equivalent to `[A-Za-z0-9_]`.
- \W Matches non “word characters” (ASCII only). Equivalent to `[^A-Za-z0-9_]`.
- \ Escape character. Any other character following the backslash, except for those listed above, is taken literally. For example, `\*` is equivalent to literal `*` (not treated as the `*` operator). Note that `\r`, `\n`, etc. are not handled specially, and will be equivalent to literal letters `r`, `n`, etc. Due to this, it’s not recommended to use raw Python strings (`r"..."`) for regular expressions. For example, `r"\r\n"` when used as the regular expression is equivalent to `"rn"`. To match CR character followed by LF, use `"\r\n"`.

NOT SUPPORTED:

- counted repetitions (`{m,n}`)
- named groups (`(?P<name>...)`)

- non-capturing groups (`(?:...)`)
- more advanced assertions (`\b`, `\B`)
- special character escapes like `\r`, `\n` - use Python's own escaping instead
- etc.

Example:

```
import re

# As re doesn't support escapes itself, use of r"" strings is not
# recommended.
regex = re.compile("[\r\n]")

regex.split("line1\rline2\nline3\r\n")

# Result:
# ['line1', 'line2', 'line3', '', '']
```

Functions

`re.compile(regex_str[, flags])`

Compile regular expression, return *regex* object.

`re.match(regex_str, string)`

Compile *regex_str* and match against *string*. Match always happens from starting position in a string.

`re.search(regex_str, string)`

Compile *regex_str* and search it in a *string*. Unlike *match*, this will search string for first position which matches *regex* (which still may be 0 if *regex* is anchored).

`re.sub(regex_str, replace, string, count=0, flags=0, /)`

Compile *regex_str* and search for it in *string*, replacing all matches with *replace*, and returning the new string.

replace can be a string or a function. If it is a string then escape sequences of the form `\<number>` and `\g<number>` can be used to expand to the corresponding group (or an empty string for unmatched groups). If *replace* is a function then it must take a single argument (the match) and should return a replacement string.

If *count* is specified and non-zero then substitution will stop after this many substitutions are made. The *flags* argument is ignored.

Note: availability of this function depends on *MicroPython port*.

`re.DEBUG`

Flag value, display debug information about compiled expression. (Availability depends on *MicroPython port*.)

Regex objects

Compiled regular expression. Instances of this class are created using `re.compile()`.

`regex.match(string)`

`regex.search(string)`

`regex.sub(replace, string, count=0, flags=0, /)`

Similar to the module-level functions `match()`, `search()` and `sub()`. Using methods is (much) more efficient if the same *regex* is applied to multiple strings.

`regex.split(string, max_split=-1, /)`

Split a *string* using `regex`. If `max_split` is given, it specifies maximum number of splits to perform. Returns list of strings (there may be up to `max_split+1` elements if it's specified).

Match objects

Match objects as returned by `match()` and `search()` methods, and passed to the replacement function in `sub()`.

`match.group(index)`

Return matching (sub)string. `index` is 0 for entire match, 1 and above for each capturing group. Only numeric groups are supported.

`match.groups()`

Return a tuple containing all the substrings of the groups of the match.

Note: availability of this method depends on *MicroPython port*.

`match.start([index])`

`match.end([index])`

Return the index in the original string of the start or end of the substring group that was matched. `index` defaults to the entire group, otherwise it will select a group.

Note: availability of these methods depends on *MicroPython port*.

`match.span([index])`

Returns the 2-tuple `(match.start(index), match.end(index))`.

Note: availability of this method depends on *MicroPython port*.

1.1.20 select – wait for events on a set of streams

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `select`.

This module provides functions to efficiently wait for events on multiple *streams* (select streams which are ready for operations).

Functions

`select.poll()`

Create an instance of the Poll class.

`select.select(rlist, wlist, xlist[, timeout])`

Wait for activity on a set of objects.

This function is provided by some MicroPython ports for compatibility and is not efficient. Usage of `Poll` is recommended instead.

class Poll

Methods

`poll.register(obj[, eventmask])`

Register *stream obj* for polling. *eventmask* is logical OR of:

- `select.POLLIN` - data available for reading
- `select.POLLOUT` - more data can be written

Note that flags like `select.POLLHUP` and `select.POLLERR` are *not* valid as input eventmask (these are unsolicited events which will be returned from `poll()` regardless of whether they are asked for). This semantics is per POSIX.

`eventmask` defaults to `select.POLLIN | select.POLLOUT`.

It is OK to call this function multiple times for the same `obj`. Successive calls will update `obj`'s eventmask to the value of `eventmask` (i.e. will behave as `modify()`).

`poll.unregister(obj)`

Unregister `obj` from polling.

`poll.modify(obj, eventmask)`

Modify the `eventmask` for `obj`. If `obj` is not registered, `OSError` is raised with error of `ENOENT`.

`poll.poll(timeout=-1, /)`

Wait for at least one of the registered objects to become ready or have an exceptional condition, with optional timeout in milliseconds (if `timeout` arg is not specified or -1, there is no timeout).

Returns list of `(obj, event, ...)` tuples. There may be other elements in tuple, depending on a platform and version, so don't assume that its size is 2. The `event` element specifies which events happened with a stream and is a combination of `select.POLL*` constants described above. Note that flags `select.POLLHUP` and `select.POLLERR` can be returned at any time (even if were not asked for), and must be acted on accordingly (the corresponding stream unregistered from poll and likely closed), because otherwise all further invocations of `poll()` may return immediately with these flags set for this stream again.

In case of timeout, an empty list is returned.

Difference to CPython

Tuples returned may contain more than 2 elements as described above.

`poll.ipoll(timeout=-1, flags=0, /)`

Like `poll.poll()`, but instead returns an iterator which yields a *callee-owned tuple*. This function provides an efficient, allocation-free way to poll on streams.

If `flags` is 1, one-shot behaviour for events is employed: streams for which events happened will have their event masks automatically reset (equivalent to `poll.modify(obj, 0)`), so new events for such a stream won't be processed until new mask is set with `poll.modify()`. This behaviour is useful for asynchronous I/O schedulers.

Difference to CPython

This function is a MicroPython extension.

1.1.21 `socket` – `socket` module

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `socket`.

This module provides access to the BSD socket interface.

Difference to CPython

For efficiency and consistency, socket objects in MicroPython implement a *stream* (file-like) interface directly. In CPython, you need to convert a socket to a file-like object using `makefile()` method. This method is still supported by MicroPython (but is a no-op), so where compatibility with CPython matters, be sure to use it.

Socket address format(s)

The native socket address format of the `socket` module is an opaque data type returned by `getaddrinfo` function, which must be used to resolve textual address (including numeric addresses):

```
sockaddr = socket.getaddrinfo('www.micropython.org', 80)[0][-1]
# You must use getaddrinfo() even for numeric addresses
sockaddr = socket.getaddrinfo('127.0.0.1', 80)[0][-1]
# Now you can use that address
sock.connect(sockaddr)
```

Using `getaddrinfo` is the most efficient (both in terms of memory and processing power) and portable way to work with addresses.

However, `socket` module (note the difference with native MicroPython `socket` module described here) provides CPython-compatible way to specify addresses using tuples, as described below. Note that depending on a *MicroPython port*, `socket` module can be builtin or need to be installed from *micropython-lib* (as in the case of *MicroPython Unix port*), and some ports still accept only numeric addresses in the tuple format, and require to use `getaddrinfo` function to resolve domain names.

Summing up:

- Always use `getaddrinfo` when writing portable applications.
- Tuple addresses described below can be used as a shortcut for quick hacks and interactive use, if your port supports them.

Tuple address format for `socket` module:

- IPv4: (*ipv4_address*, *port*), where *ipv4_address* is a string with dot-notation numeric IPv4 address, e.g. "8.8.8.8", and *port* is an integer port number in the range 1-65535. Note the domain names are not accepted as *ipv4_address*, they should be resolved first using `socket.getaddrinfo()`.
- IPv6: (*ipv6_address*, *port*, *flowinfo*, *scopeid*), where *ipv6_address* is a string with colon-notation numeric IPv6 address, e.g. "2001:db8::1", and *port* is an integer port number in the range 1-65535. *flowinfo* must be 0. *scopeid* is the interface scope identifier for link-local addresses. Note the domain names are not accepted as *ipv6_address*, they should be resolved first using `socket.getaddrinfo()`. Availability of IPv6 support depends on a *MicroPython port*.

Functions

`socket.getaddrinfo` (*host*, *port*, *af=0*, *type=0*, *proto=0*, *flags=0*, /)

Translate the host/port argument into a sequence of 5-tuples that contain all the necessary arguments for creating a socket connected to that service. Arguments *af*, *type*, and *proto* (which have the same meaning as for the `socket()` function) can be used to filter which kind of addresses are returned. If a parameter is not specified or zero, all combinations of addresses can be returned (requiring filtering on the user side).

The resulting list of 5-tuples has the following structure:

```
(family, type, proto, canonname, sockaddr)
```

The following example shows how to connect to a given url:

```
s = socket.socket()
# This assumes that if "type" is not specified, an address for
# SOCK_STREAM will be returned, which may be not true
s.connect(socket.getaddrinfo('www.micropython.org', 80)[0][-1])
```

Recommended use of filtering params:

```
s = socket.socket()
# Guaranteed to return an address which can be connect'ed to for
# stream operation.
s.connect(socket.getaddrinfo('www.micropython.org', 80, 0, SOCK_STREAM)[0][-1])
```

Difference to CPython

CPython raises a `socket.gaierror` exception (`OSError` subclass) in case of error in this function. MicroPython doesn't have `socket.gaierror` and raises `OSError` directly. Note that error numbers of `getaddrinfo()` form a separate namespace and may not match error numbers from the `errno` module. To distinguish `getaddrinfo()` errors, they are represented by negative numbers, whereas standard system errors are positive numbers (error numbers are accessible using `e.args[0]` property from an exception object). The use of negative values is a provisional detail which may change in the future.

`socket.inet_ntop(af, bin_addr)`

Convert a binary network address *bin_addr* of the given address family *af* to a textual representation:

```
>>> socket.inet_ntop(socket.AF_INET, b"\x7f\x00\x00\x01")
'127.0.0.1'
```

`socket.inet_pton(af, txt_addr)`

Convert a textual network address *txt_addr* of the given address family *af* to a binary representation:

```
>>> socket.inet_pton(socket.AF_INET, "1.2.3.4")
b'\x01\x02\x03\x04'
```

Constants

`socket.AF_INET`

`socket.AF_INET6`

Address family types. Availability depends on a particular *MicroPython port*.

`socket.SOCK_STREAM`

`socket.SOCK_DGRAM`

Socket types.

`socket.IPPROTO_UDP`

`socket.IPPROTO_TCP`

IP protocol numbers. Availability depends on a particular *MicroPython port*. Note that you don't need to specify these in a call to `socket.socket()`, because `SOCK_STREAM` socket type automatically selects `IPPROTO_TCP`, and `SOCK_DGRAM` - `IPPROTO_UDP`. Thus, the only real use of these constants is as an argument to `setsockopt()`.

`socket.SOL_*`

Socket option levels (an argument to `setsockopt()`). The exact inventory depends on a *MicroPython port*.

socket.SO_*

Socket options (an argument to `setsockopt()`). The exact inventory depends on a *MicroPython port*.

Constants specific to WiPy:

socket.IPPROTO_SEC

Special protocol value to create SSL-compatible socket.

class socket

class `socket.socket` (*af*=`AF_INET`, *type*=`SOCK_STREAM`, *proto*=`IPPROTO_TCP`, /)

Create a new socket using the given address family, socket type and protocol number. Note that specifying *proto* in most cases is not required (and not recommended, as some MicroPython ports may omit `IPPROTO_*` constants). Instead, *type* argument will select needed protocol automatically:

```
# Create STREAM TCP socket
socket(AF_INET, SOCK_STREAM)
# Create DGRAM UDP socket
socket(AF_INET, SOCK_DGRAM)
```

Methods

`socket.close()`

Mark the socket closed and release all resources. Once that happens, all future operations on the socket object will fail. The remote end will receive EOF indication if supported by protocol.

Sockets are automatically closed when they are garbage-collected, but it is recommended to `close()` them explicitly as soon you finished working with them.

`socket.bind(address)`

Bind the socket to *address*. The socket must not already be bound.

`socket.listen([backlog])`

Enable a server to accept connections. If *backlog* is specified, it must be at least 0 (if it's lower, it will be set to 0); and specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen.

`socket.accept()`

Accept a connection. The socket must be bound to an address and listening for connections. The return value is a pair (conn, address) where conn is a new socket object usable to send and receive data on the connection, and address is the address bound to the socket on the other end of the connection.

`socket.connect(address)`

Connect to a remote socket at *address*.

`socket.send(bytes)`

Send data to the socket. The socket must be connected to a remote socket. Returns number of bytes sent, which may be smaller than the length of data ("short write").

`socket.sendall(bytes)`

Send all data to the socket. The socket must be connected to a remote socket. Unlike `send()`, this method will try to send all of data, by sending data chunk by chunk consecutively.

The behaviour of this method on non-blocking sockets is undefined. Due to this, on MicroPython, it's recommended to use `write()` method instead, which has the same "no short writes" policy for blocking sockets, and will return number of bytes sent on non-blocking sockets.

`socket.recv(bufsize[, flags])`

Receive data from the socket. The return value is a bytes object representing the data received. The maximum amount of data to be received at once is specified by `bufsize`.

Most ports support the optional `flags` argument. Available `flags` are defined as constants in the `socket` module and have the same meaning as in CPython. `MSG_PEEK` and `MSG_DONTWAIT` are supported on all ports which accept the `flags` argument.

`socket.sendto(bytes, address)`

Send data to the socket. The socket should not be connected to a remote socket, since the destination socket is specified by `address`.

`socket.recvfrom(bufsize[, flags])`

Receive data from the socket. The return value is a pair `(bytes, address)` where `bytes` is a bytes object representing the data received and `address` is the address of the socket sending the data.

See the `recv` function for an explanation of the optional `flags` argument.

`socket.setsockopt(level, optname, value)`

Set the value of the given socket option. The needed symbolic constants are defined in the `socket` module (`SO_*` etc.). The `value` can be an integer or a bytes-like object representing a buffer.

`socket.settimeout(value)`

Note: Not every port supports this method, see below.

Set a timeout on blocking socket operations. The value argument can be a nonnegative floating point number expressing seconds, or `None`. If a non-zero value is given, subsequent socket operations will raise an `OSError` exception if the timeout period value has elapsed before the operation has completed. If zero is given, the socket is put in non-blocking mode. If `None` is given, the socket is put in blocking mode.

Not every *MicroPython port* supports this method. A more portable and generic solution is to use `select.poll` object. This allows to wait on multiple objects at the same time (and not just on sockets, but on generic *stream* objects which support polling). Example:

```
# Instead of:
s.settimeout(1.0) # time in seconds
s.read(10) # may timeout

# Use:
poller = select.poll()
poller.register(s, select.POLLIN)
res = poller.poll(1000) # time in milliseconds
if not res:
    # s is still not ready for input, i.e. operation timed out
```

Difference to CPython

CPython raises a `socket.timeout` exception in case of timeout, which is an `OSError` subclass. MicroPython raises an `OSError` directly instead. If you use `except OSError:` to catch the exception, your code will work both in MicroPython and CPython.

`socket.setblocking(flag)`

Set blocking or non-blocking mode of the socket: if `flag` is false, the socket is set to non-blocking, else to blocking mode.

This method is a shorthand for certain `settimeout()` calls:

- `sock.setblocking(True)` is equivalent to `sock.settimeout(None)`
- `sock.setblocking(False)` is equivalent to `sock.settimeout(0)`

`socket.makefile(mode='rb', buffering=0, /)`

Return a file object associated with the socket. The exact returned type depends on the arguments given to `makefile()`. The support is limited to binary modes only ('rb', 'wb', and 'rwb'). CPython's arguments: *encoding*, *errors* and *newline* are not supported.

Difference to CPython

As MicroPython doesn't support buffered streams, values of *buffering* parameter is ignored and treated as if it was 0 (unbuffered).

Difference to CPython

Closing the file object returned by `makefile()` WILL close the original socket as well.

`socket.read([size])`

Read up to *size* bytes from the socket. Return a bytes object. If *size* is not given, it reads all data available from the socket until EOF; as such the method will not return until the socket is closed. This function tries to read as much data as requested (no "short reads"). This may be not possible with non-blocking socket though, and then less data will be returned.

`socket.readinto(buf[, nbytes])`

Read bytes into the *buf*. If *nbytes* is specified then read at most that many bytes. Otherwise, read at most `len(buf)` bytes. Just as `read()`, this method follows "no short reads" policy.

Return value: number of bytes read and stored into *buf*.

`socket.readline()`

Read a line, ending in a newline character.

Return value: the line read.

`socket.write(buf)`

Write the buffer of bytes to the socket. This function will try to write all data to a socket (no "short writes"). This may be not possible with a non-blocking socket though, and returned value will be less than the length of *buf*.

Return value: number of bytes written.

exception `socket.error`

MicroPython does NOT have this exception.

Difference to CPython

CPython used to have a `socket.error` exception which is now deprecated, and is an alias of `OSError`. In MicroPython, use `OSError` directly.

1.1.22 `ssl` – SSL/TLS module

This module implements a subset of the corresponding *CPython* module, as described below. For more information, refer to the original *CPython* documentation: `ssl`.

This module provides access to Transport Layer Security (previously and widely known as “Secure Sockets Layer”) encryption and peer authentication facilities for network sockets, both client-side and server-side.

Functions

`ssl.wrap_socket(sock, server_side=False, key=None, cert=None, cert_reqs=CERT_NONE, cadata=None, server_hostname=None, do_handshake=True)`

Wrap the given *sock* and return a new wrapped-socket object. The implementation of this function is to first create an *SSLContext* and then call the *SSLContext.wrap_socket* method on that context object. The arguments *sock*, *server_side* and *server_hostname* are passed through unchanged to the method call. The argument *do_handshake* is passed through as *do_handshake_on_connect*. The remaining arguments have the following behaviour:

- *cert_reqs* determines whether the peer (server or client) must present a valid certificate. Note that for mbedtls based ports, `ssl.CERT_NONE` and `ssl.CERT_OPTIONAL` will not validate any certificate, only `ssl.CERT_REQUIRED` will.
- *cadata* is a bytes object containing the CA certificate chain (in DER format) that will validate the peer’s certificate. Currently only a single DER-encoded certificate is supported.

Depending on the underlying module implementation in a particular *MicroPython port*, some or all keyword arguments above may be not supported.

class *SSLContext*

class `ssl.SSLContext(protocol, /)`

Create a new *SSLContext* instance. The *protocol* argument must be one of the `PROTOCOL_*` constants.

`SSLContext.load_cert_chain(certfile, keyfile)`

Load a private key and the corresponding certificate. The *certfile* is a string with the file path of the certificate. The *keyfile* is a string with the file path of the private key.

Difference to CPython

MicroPython extension: *certfile* and *keyfile* can be bytes objects instead of strings, in which case they are interpreted as the actual certificate/key data.

`SSLContext.load_verify_locations(cafile=None, cadata=None)`

Load the CA certificate chain that will validate the peer’s certificate. *cafile* is the file path of the CA certificates. *cadata* is a bytes object containing the CA certificates. Only one of these arguments should be provided.

`SSLContext.get_ciphers()`

Get a list of enabled ciphers, returned as a list of strings.

`SSLContext.set_ciphers(ciphers)`

Set the available ciphers for sockets created with this context. *ciphers* should be a list of strings in the [IANA cipher suite format](#).

`SSLContext.wrap_socket(sock, *, server_side=False, do_handshake_on_connect=True, server_hostname=None, client_id=None)`

Takes a *stream sock* (usually `socket.socket` instance of `SOCK_STREAM` type), and returns an instance of `ssl.SSLSocket`, wrapping the underlying stream. The returned object has the usual *stream* interface methods like `read()`, `write()`, etc.

- *server_side* selects whether the wrapped socket is on the server or client side. A server-side SSL socket should be created from a normal socket returned from `accept()` on a non-SSL listening server socket.
- *do_handshake_on_connect* determines whether the handshake is done as part of the `wrap_socket` or whether it is deferred to be done as part of the initial reads or writes. For blocking sockets doing the handshake immediately is standard. For non-blocking sockets (i.e. when the *sock* passed into `wrap_socket` is in non-blocking mode) the handshake should generally be deferred because otherwise `wrap_socket` blocks until it completes. Note that in AXTLS the handshake can be deferred until the first read or write but it then blocks until completion.
- *server_hostname* is for use as a client, and sets the hostname to check against the received server certificate. It also sets the name for Server Name Indication (SNI), allowing the server to present the proper certificate.
- *client_id* is a MicroPython-specific extension argument used only when implementing a DTLS Server. See [DTLS support](#) for details.

Warning

Some implementations of `ssl` module do NOT validate server certificates, which makes an SSL connection established prone to man-in-the-middle attacks.

CPython's `wrap_socket` returns an `SSLSocket` object which has methods typical for sockets, such as `send`, `recv`, etc. MicroPython's `wrap_socket` returns an object more similar to CPython's `SSLObject` which does not have these socket methods.

`SSLContext.verify_mode`

Set or get the behaviour for verification of peer certificates. Must be one of the `CERT_*` constants.

Note

`ssl.CERT_REQUIRED` requires the device's date/time to be properly set, e.g. using `mpremote rtc --set` or `ntptime`, and *server_hostname* must be specified when on the client side.

Exceptions

`ssl.SSLError`

This exception does NOT exist. Instead its base class, `OSError`, is used.

DTLS support

Difference to CPython

This is a MicroPython extension.

On most ports, this module supports DTLS in client and server mode via the `PROTOCOL_DTLS_CLIENT` and `PROTOCOL_DTLS_SERVER` constants that can be used as the *protocol* argument of `SSLContext`.

In this case the underlying socket is expected to behave as a datagram socket (i.e. like the socket opened with `socket.socket` with `socket.AF_INET` as `af` and `socket.SOCK_DGRAM` as `type`).

DTLS is only supported on ports that use mbedTLS, and it is enabled by default in most configurations but can be manually disabled by defining `MICROPY_PY_SSL_DTLS` to 0.

DTLS server support

MicroPython’s DTLS server support is configured with “Hello Verify” as required for DTLS 1.2. This is transparent for DTLS clients, but there are relevant considerations when implementing a DTLS server in MicroPython:

- The server should pass an additional argument *client_id* when calling `SSLContext.wrap_socket()`. This ID must be a *bytes* object (or similar) with a transport-specific identifier representing the client.

The simplest approach is to convert the tuple of `(client_ip, client_port)` returned from `socket.recv_from()` into a byte string, i.e.:

```
_, client_addr = sock.recvfrom(1, socket.MSG_PEEK)
sock.connect(client_addr) # Connect back to the client
sock = ssl_ctx.wrap_socket(sock, server_side=True,
                           client_id=repr(client_addr).encode())
```

- The first time a client connects, the server call to `wrap_socket` will fail with a `OSError` error “Hello Verify Required”. This is because the DTLS “Hello Verify” cookie is not yet known by the client. If the same client connects a second time then `wrap_socket` will succeed.
- DTLS cookies for “Hello Verify” are associated with the `SSLContext` object, so the same `SSLContext` object should be used to wrap a subsequent connection from the same client. The cookie implementation includes a timeout and has constant memory use regardless of how many clients connect, so it’s OK to reuse the same `SSLContext` object for the lifetime of the server.

Constants

`ssl.PROTOCOL_TLS_CLIENT`

`ssl.PROTOCOL_TLS_SERVER`

`ssl.PROTOCOL_DTLS_CLIENT` (when DTLS support is enabled)

`ssl.PROTOCOL_DTLS_SERVER` (when DTLS support is enabled)

Supported values for the *protocol* parameter.

`ssl.CERT_NONE`

`ssl.CERT_OPTIONAL`

`ssl.CERT_REQUIRED`

Supported values for *cert_reqs* parameter, and the `SSLContext.verify_mode` attribute.

1.1.23 struct – pack and unpack primitive data types

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `struct`.

The following byte orders are supported:

Character	Byte order	Size	Alignment
@	native	native	native
<	little-endian	standard	none
>	big-endian	standard	none
!	network (= big-endian)	standard	none

The following data types are supported:

Format	C Type	Python type	Standard size
b	signed char	integer	1
B	unsigned char	integer	1
h	short	integer	2
H	unsigned short	integer	2
i	int	integer (<i>I</i>)	4
I	unsigned int	integer (<i>I</i>)	4
l	long	integer (<i>I</i>)	4
L	unsigned long	integer (<i>I</i>)	4
q	long long	integer (<i>I</i>)	8
Q	unsigned long long	integer (<i>I</i>)	8
e	n/a (half-float)	float (<i>2</i>)	2
f	float	float (<i>2</i>)	4
d	double	float (<i>2</i>)	8
s	char[]	bytes	
P	void *	integer	

- (1) Requires long support when used with values larger than 30 bits.
- (2) Requires floating point support.

Difference to CPython

Whitespace is not supported in format strings.

Functions

`struct.calcsize(fmt)`

Return the number of bytes needed to store the given *fmt*.

`struct.pack(fmt, v1, v2, ...)`

Pack the values *v1*, *v2*, ... according to the format string *fmt*. The return value is a bytes object encoding the values.

`struct.pack_into(fmt, buffer, offset, v1, v2, ...)`

Pack the values *v1*, *v2*, ... according to the format string *fmt* into a *buffer* starting at *offset*. *offset* may be negative to count from the end of *buffer*.

`struct.unpack(fmt, data)`

Unpack from the *data* according to the format string *fmt*. The return value is a tuple of the unpacked values.

`struct.unpack_from(fmt, data, offset=0, /)`

Unpack from the *data* starting at *offset* according to the format string *fmt*. *offset* may be negative to count from the end of *data*. The return value is a tuple of the unpacked values.

1.1.24 `sys` – system specific functions

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `sys`.

Functions

`sys.exit (retval=0, /)`

Terminate current program with a given exit code. Underlyingly, this function raises a `SystemExit` exception. If an argument is given, its value given as an argument to `SystemExit`.

On embedded ports (i.e. all ports but Windows and Unix), an unhandled `SystemExit` currently causes a *Soft Reset* of MicroPython.

`sys.atexit (func)`

Register `func` to be called upon termination. `func` must be a callable that takes no arguments, or `None` to disable the call. The `atexit` function will return the previous value set by this function, which is initially `None`.

Difference to CPython

This function is a MicroPython extension intended to provide similar functionality to the `atexit` module in CPython.

`sys.print_exception (exc, file=sys.stdout, /)`

Print exception with a traceback to a file-like object `file` (or `sys.stdout` by default).

Difference to CPython

This is simplified version of a function which appears in the `traceback` module in CPython. Unlike `traceback.print_exception()`, this function takes just exception value instead of exception type, exception value, and traceback object; `file` argument should be positional; further arguments are not supported. CPython-compatible `traceback` module can be found in *micropython-lib*.

`sys.settrace (tracefunc)`

Enable tracing of bytecode execution. For details see the [CPython documentation](#).

This function requires a custom MicroPython build as it is typically not present in pre-built firmware (due to it affecting performance). The relevant configuration option is `MICROPY_PY_SYS_SETTRACE`.

Constants

`sys.argv`

A mutable list of arguments the current program was started with.

`sys.byteorder`

The byte order of the system ("little" or "big").

`sys.implementation`

Object with information about the current Python implementation. For MicroPython, it has following attributes:

- `name` - string "micropython"
- `version` - tuple (major, minor, micro, releaselevel), e.g. (1, 22, 0, " ")
- `_machine` - string describing the underlying machine

- `_mpy` - supported mpy file-format version (optional attribute)
- `_build` - string that can help identify the configuration that MicroPython was built with
- `_thread` - optional string attribute, exists if the target has threading and is either “GIL” or “unsafe”

This object is the recommended way to distinguish MicroPython from other Python implementations (note that it still may not exist in the very minimal ports).

Starting with version 1.22.0-preview, the fourth node *releaselevel* in *implementation.version* is either an empty string or “preview”.

The `_build` entry was added in version 1.25.0 and is a hyphen-separated set of elements. New elements may be appended in the future so it’s best to access this field using `sys.implementation._build.split("-")`. The elements that are currently used are:

- On the unix, webassembly and windows ports the first element is the variant name, for example ‘standard’.
- On microcontroller targets, the first element is the board name and the second element (if present) is the board variant, for example ‘RPI_PICO2-RISCV’

The `_thread` entry was added in version 1.26.0 and if it exists then the target has the `_thread` module. If the target enables the GIL (global interpreter lock) then this attribute is “GIL”. Otherwise the attribute is “unsafe” and the target has threading but does not enable the GIL, and mutable Python objects (such as *bytearray*, *list* and *dict*) that are shared amongst threads must be protected explicitly by locks such as `_thread.allocate_lock`.

Difference to CPython

CPython mandates more attributes for this object, but the actual useful bare minimum is implemented in MicroPython.

`sys.maxsize`

Maximum value which a native integer type can hold on the current platform, or maximum value representable by MicroPython integer type, if it’s smaller than platform max value (that is the case for MicroPython ports without long int support).

This attribute is useful for detecting “bitness” of a platform (32-bit vs 64-bit, etc.). It’s recommended to not compare this attribute to some value directly, but instead count number of bits in it:

```
bits = 0
v = sys.maxsize
while v:
    bits += 1
    v >>= 1
if bits > 32:
    # 64-bit (or more) platform
    ...
else:
    # 32-bit (or less) platform
    # Note that on 32-bit platform, value of bits may be less than 32
    # (e.g. 31) due to peculiarities described above, so use "> 16",
    # "> 32", "> 64" style of comparisons.
```

`sys.modules`

Dictionary of loaded modules. On some ports, it may not include builtin modules.

sys.path

A mutable list of directories to search for imported modules.

i Difference to CPython

On MicroPython, an entry with the value `".frozen"` will indicate that import should search *frozen modules* at that point in the search. If no frozen module is found then search will *not* look for a directory called `.frozen`, instead it will continue with the next entry in `sys.path`.

sys.platform

The platform that MicroPython is running on. For OS/RTOS ports, this is usually an identifier of the OS, e.g. `"linux"`. For baremetal ports it is an identifier of a board, e.g. `"pyboard"` for the original MicroPython reference board. It thus can be used to distinguish one board from another. If you need to check whether your program runs on MicroPython (vs other Python implementation), use `sys.implementation` instead.

sys.ps1**sys.ps2**

Mutable attributes holding strings, which are used for the REPL prompt. The defaults give the standard Python prompt of `>>>` and `...`.

sys.stderr

Standard error *stream*.

sys.stdin

Standard input *stream*.

sys.stdout

Standard output *stream*.

sys.tracebacklimit

A mutable attribute holding an integer value which is the maximum number of traceback entries to store in an exception. Set to 0 to disable adding tracebacks. Defaults to 1000.

Note: this is not available on all ports.

sys.version

Python language version that this implementation conforms to, as a string.

sys.version_info

Python language version that this implementation conforms to, as a tuple of ints.

i Difference to CPython

Only the first three version numbers (major, minor, micro) are supported and they can be referenced only by index, not by name.

1.1.25 time – time related functions

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: [time](#).

The `time` module provides functions for getting the current time and date, measuring time intervals, and for delays.

Time Epoch: The unix, windows, webassembly, alif, mimmxrt and rp2 ports use the standard for POSIX systems epoch of 1970-01-01 00:00:00 UTC. The other embedded ports use an epoch of 2000-01-01 00:00:00 UTC. Epoch year may be determined with `gmtime(0)[0]`.

Maintaining actual calendar date/time: This requires a Real Time Clock (RTC). On systems with underlying OS (including some RTOS), an RTC may be implicit. Setting and maintaining actual calendar time is responsibility of OS/RTOS and is done outside of MicroPython, it just uses OS API to query date/time. On baremetal ports however system time depends on `machine.RTC()` object. The current calendar time may be set using `machine.RTC().datetime(tuple)` function, and maintained by following means:

- By a backup battery (which may be an additional, optional component for a particular board).
- Using networked time protocol (requires setup by a port/user).
- Set manually by a user on each power-up (many boards then maintain RTC time across hard resets, though some may require setting it again in such case).

If actual calendar time is not maintained with a system/MicroPython RTC, functions below which require reference to current absolute time may behave not as expected.

Functions

`time.gmtime([secs])`

`time.localtime([secs])`

Convert the time *secs* expressed in seconds since the Epoch (see above) into an 8-tuple which contains: (*year*, *month*, *mday*, *hour*, *minute*, *second*, *weekday*, *yearday*) If *secs* is not provided or None, then the current time from the RTC is used.

The `gmtime()` function returns a date-time tuple in UTC, and `localtime()` returns a date-time tuple in local time.

The format of the entries in the 8-tuple are:

- year includes the century (for example 2014).
- month is 1-12
- mday is 1-31
- hour is 0-23
- minute is 0-59
- second is 0-59
- weekday is 0-6 for Mon-Sun
- yearday is 1-366

`time.mktime(date_time_tuple)`

This is inverse function of `localtime`. It's argument is a full 8-tuple which expresses a time as per `localtime`. It returns an integer which is the number of seconds since the time epoch.

`time.sleep(seconds)`

Sleep for the given number of seconds. Some boards may accept *seconds* as a floating-point number to sleep for a fractional number of seconds. Note that other boards may not accept a floating-point argument, for compatibility with them use `sleep_ms()` and `sleep_us()` functions.

`time.sleep_ms(ms)`

Delay for given number of milliseconds, should be positive or 0.

This function will delay for at least the given number of milliseconds, but may take longer than that if other processing must take place, for example interrupt handlers or other threads. Passing in 0 for *ms* will still allow this other processing to occur. Use `sleep_us()` for more precise delays.

`time.sleep_us(us)`

Delay for given number of microseconds, should be positive or 0.

This function attempts to provide an accurate delay of at least *us* microseconds, but it may take longer if the system has other higher priority processing to perform.

`time.ticks_ms()`

Returns an increasing millisecond counter with an arbitrary reference point, that wraps around after some value.

The wrap-around value is not explicitly exposed, but we will refer to it as *TICKS_MAX* to simplify discussion. Period of the values is *TICKS_PERIOD* = *TICKS_MAX* + 1. *TICKS_PERIOD* is guaranteed to be a power of two, but otherwise may differ from port to port. The same period value is used for all of `ticks_ms()`, `ticks_us()`, `ticks_cpu()` functions (for simplicity). Thus, these functions will return a value in range [0 .. *TICKS_MAX*], inclusive, total *TICKS_PERIOD* values. Note that only non-negative values are used. For the most part, you should treat values returned by these functions as opaque. The only operations available for them are `ticks_diff()` and `ticks_add()` functions described below.

Note: Performing standard mathematical operations (+, -) or relational operators (<, <=, >, >=) directly on these value will lead to invalid result. Performing mathematical operations and then passing their results as arguments to `ticks_diff()` or `ticks_add()` will also lead to invalid results from the latter functions.

`time.ticks_us()`

Just like `ticks_ms()` above, but in microseconds.

`time.ticks_cpu()`

Similar to `ticks_ms()` and `ticks_us()`, but with the highest possible resolution in the system. This is usually CPU clocks, and that's why the function is named that way. But it doesn't have to be a CPU clock, some other timing source available in a system (e.g. high-resolution timer) can be used instead. The exact timing unit (resolution) of this function is not specified on `time` module level, but documentation for a specific port may provide more specific information. This function is intended for very fine benchmarking or very tight real-time loops. Avoid using it in portable code.

Availability: Not every port implements this function.

`time.ticks_add(ticks, delta)`

Offset ticks value by a given number, which can be either positive or negative. Given a *ticks* value, this function allows to calculate ticks value *delta* ticks before or after it, following modular-arithmetic definition of tick values (see `ticks_ms()` above). *ticks* parameter must be a direct result of call to `ticks_ms()`, `ticks_us()`, or `ticks_cpu()` functions (or from previous call to `ticks_add()`). However, *delta* can be an arbitrary integer number or numeric expression. `ticks_add()` is useful for calculating deadlines for events/tasks. (Note: you must use `ticks_diff()` function to work with deadlines.)

Examples:

```
# Find out what ticks value there was 100ms ago
print(ticks_add(time.ticks_ms(), -100))

# Calculate deadline for operation and test for it
deadline = ticks_add(time.ticks_ms(), 200)
while ticks_diff(deadline, time.ticks_ms()) > 0:
    do_a_little_of_something()

# Find out TICKS_MAX used by this port
print(ticks_add(0, -1))
```

`time.ticks_diff(ticks1, ticks2)`

Measure ticks difference between values returned from `ticks_ms()`, `ticks_us()`, or `ticks_cpu()` functions, as a signed value which may wrap around.

The argument order is the same as for subtraction operator, `ticks_diff(ticks1, ticks2)` has the same meaning as `ticks1 - ticks2`. However, values returned by `ticks_ms()`, etc. functions may wrap around, so directly using subtraction on them will produce incorrect result. That is why `ticks_diff()` is needed, it implements modular (or more specifically, ring) arithmetic to produce correct result even for wrap-around values (as long as they not too distant in between, see below). The function returns **signed** value in the range `[-TICKS_PERIOD/2 .. TICKS_PERIOD/2-1]` (that's a typical range definition for two's-complement signed binary integers). If the result is negative, it means that `ticks1` occurred earlier in time than `ticks2`. Otherwise, it means that `ticks1` occurred after `ticks2`. This holds **only** if `ticks1` and `ticks2` are apart from each other for no more than `TICKS_PERIOD/2-1` ticks. If that does not hold, incorrect result will be returned. Specifically, if two tick values are apart for `TICKS_PERIOD/2-1` ticks, that value will be returned by the function. However, if `TICKS_PERIOD/2` of real-time ticks has passed between them, the function will return `-TICKS_PERIOD/2` instead, i.e. result value will wrap around to the negative range of possible values.

Informal rationale of the constraints above: Suppose you are locked in a room with no means to monitor passing of time except a standard 12-notch clock. Then if you look at dial-plate now, and don't look again for another 13 hours (e.g., if you fall for a long sleep), then once you finally look again, it may seem to you that only 1 hour has passed. To avoid this mistake, just look at the clock regularly. Your application should do the same. "Too long sleep" metaphor also maps directly to application behaviour: don't let your application run any single task for too long. Run tasks in steps, and do time-keeping in between.

`ticks_diff()` is designed to accommodate various usage patterns, among them:

- Polling with timeout. In this case, the order of events is known, and you will deal only with positive results of `ticks_diff()`:

```
# Wait for GPIO pin to be asserted, but at most 500us
start = time.ticks_us()
while pin.value() == 0:
    if time.ticks_diff(time.ticks_us(), start) > 500:
        raise TimeoutError
```

- Scheduling events. In this case, `ticks_diff()` result may be negative if an event is overdue:

```
# This code snippet is not optimized
now = time.ticks_ms()
scheduled_time = task.scheduled_time()
if ticks_diff(scheduled_time, now) > 0:
    print("Too early, let's nap")
    sleep_ms(ticks_diff(scheduled_time, now))
    task.run()
elif ticks_diff(scheduled_time, now) == 0:
    print("Right at time!")
    task.run()
elif ticks_diff(scheduled_time, now) < 0:
    print("Oops, running late, tell task to run faster!")
    task.run(run_faster=True)
```

Note: Do not pass `time()` values to `ticks_diff()`, you should use normal mathematical operations on them. But note that `time()` may (and will) also overflow. This is known as https://en.wikipedia.org/wiki/Year_2038_problem.

`time.time()`

Returns the number of seconds, as an integer, since the Epoch, assuming that underlying RTC is set and maintained

as described above. If an RTC is not set, this function returns number of seconds since a port-specific reference point in time (for embedded boards without a battery-backed RTC, usually since power up or reset). If you want to develop portable MicroPython application, you should not rely on this function to provide higher than second precision. If you need higher precision, absolute timestamps, use `time_ns()`. If relative times are acceptable then use the `ticks_ms()` and `ticks_us()` functions. If you need calendar time, `gmtime()` or `localtime()` without an argument is a better choice.

Difference to CPython

In CPython, this function returns number of seconds since Unix epoch, 1970-01-01 00:00 UTC, as a floating-point, usually having microsecond precision. With MicroPython, only Unix port uses the same Epoch, and if floating-point precision allows, returns sub-second precision. Embedded hardware usually doesn't have floating-point precision to represent both long time ranges and subsecond precision, so they use integer value with second precision. Some embedded hardware also lacks battery-powered RTC, so returns number of seconds since last power-up or from other relative, hardware-specific point (e.g. reset).

`time.time_ns()`

Similar to `time()` but returns nanoseconds since the Epoch, as an integer (usually a big integer, so will allocate on the heap).

1.1.26 `zlib` – zlib compression & decompression

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `zlib`.

This module allows compression and decompression of binary data with the [DEFLATE algorithm](#) (commonly used in the `zlib` library and `gzip` archiver).

Note

Prefer to use `deflate.DeflateIO` instead of the functions in this module as it provides a streaming interface to compression and decompression which is convenient and more memory efficient when working with reading or writing compressed data to a file, socket, or stream.

Availability:

- From MicroPython v1.21 onwards, this module may not be present by default on all MicroPython firmware as it duplicates functionality available in the `deflate` module.
- A copy of this module can be installed (or frozen) from [micropython-lib](#) (source). See [Package management](#) for more information. This documentation describes that module.
- Requires the built-in `deflate` module (available since MicroPython v1.21)
- Compression support will only be available if compression support is enabled in the built-in `deflate` module.

Functions

`zlib.decompress(data, wbits=15, /)`

Decompresses `data` into a bytes object.

The `wbits` parameter works the same way as for `zlib.compress()` with the following additional valid values:

- 0: Automatically determine the window size from the `zlib` header (`data` must be in `zlib` format).
- 35 to 47: Auto-detect either the `zlib` or `gzip` format.

As for `zlib.compress()`, see the CPython documentation for `zlib` for more information about the `wbits` parameter. As for `zlib.compress()`, MicroPython also supports smaller window sizes than CPython. See more *MicroPython-specific details* in the `deflate` module documentation.

If the data to be decompressed requires a larger window size, it will fail during decompression.

`zlib.compress(data, wbits=15, /)`

Compresses `data` into a bytes object.

`wbits` allows you to configure the DEFLATE dictionary window size and the output format. The window size allows you to trade-off memory usage for compression level. A larger window size will allow the compressor to reference fragments further back in the input. The output formats are “raw” DEFLATE (no header/footer), `zlib`, and `gzip`, where the latter two include a header and checksum.

The low four bits of the absolute value of `wbits` set the base-2 logarithm of the DEFLATE dictionary window size. So for example, `wbits=10`, `wbits=-10`, and `wbits=26` all set the window size to 1024 bytes. Valid window sizes are 5 to 15 inclusive (corresponding to 32 to 32k bytes).

Negative values of `wbits` between -5 and -15 correspond to “raw” output mode, positive values between 5 and 15 correspond to `zlib` output mode, and positive values between 21 and 31 correspond to `gzip` output mode.

See the CPython documentation for `zlib` for more information about the `wbits` parameter. Note that MicroPython allows for smaller window sizes, which is useful when memory is constrained while still achieving a reasonable level of compression. It also speeds up the compressor. See more *MicroPython-specific details* in the `deflate` module documentation.

1.1.27 `_thread` – multithreading support

This module implements a subset of the corresponding CPython module, as described below. For more information, refer to the original CPython documentation: `_thread`.

This module implements multithreading support.

This module is highly experimental and its API is not yet fully settled and not yet described in this documentation.

1.2 MicroPython-specific libraries

Functionality specific to the MicroPython implementation is available in the following libraries.

1.2.1 `bluetooth` — low-level Bluetooth

This module provides an interface to a Bluetooth controller on a board. Currently this supports Bluetooth Low Energy (BLE) in Central, Peripheral, Broadcaster, and Observer roles, as well as GATT Server and Client and L2CAP connection-oriented-channels. A device may operate in multiple roles concurrently. Pairing (and bonding) is supported on some ports.

This API is intended to match the low-level Bluetooth protocol and provide building-blocks for higher-level abstractions such as specific device types.

Note

For most applications, we recommend using the higher-level `aioble` library.

Note

This module is still under development and its classes, functions, methods and constants are subject to change.

class BLE

Constructor

```
class bluetooth.BLE
```

Returns the singleton BLE object.

Configuration

```
BLE.active([active, ]/)
```

Optionally changes the active state of the BLE radio, and returns the current state.

The radio must be made active before using any other methods on this class.

```
BLE.config('param', /)
```

```
BLE.config(*, param=value, ...)
```

Get or set configuration values of the BLE interface. To get a value the parameter name should be quoted as a string, and just one parameter is queried at a time. To set values use the keyword syntax, and one or more parameter can be set at a time.

Currently supported values are:

- 'mac': The current address in use, depending on the current address mode. This returns a tuple of (addr_type, addr).

See [gatts_write](#) for details about address type.

This may only be queried while the interface is currently active.

- 'addr_mode': Sets the address mode. Values can be:
 - 0x00 - PUBLIC - Use the controller's public address.
 - 0x01 - RANDOM - Use a generated static address.
 - 0x02 - RPA - Use resolvable private addresses.
 - 0x03 - NRPA - Use non-resolvable private addresses.

By default the interface mode will use a PUBLIC address if available, otherwise it will use a RANDOM address.

- 'gap_name': Get/set the GAP device name used by service 0x1800, characteristic 0x2a00. This can be set at any time and changed multiple times.
- 'rxbuf': Get/set the size in bytes of the internal buffer used to store incoming events. This buffer is global to the entire BLE driver and so handles incoming data for all events, including all characteristics. Increasing this allows better handling of bursty incoming data (for example scan results) and the ability to receive larger characteristic values.
- 'mtu': Get/set the MTU that will be used during a ATT MTU exchange. The resulting MTU will be the minimum of this and the remote device's MTU. ATT MTU exchange will not happen automatically (unless the remote device initiates it), and must be manually initiated with [gattc_exchange_mtu](#). Use the `_IRQ_MTU_EXCHANGED` event to discover the MTU for a given connection.
- 'bond': Sets whether bonding will be enabled during pairing. When enabled, pairing requests will set the "bond" flag and the keys will be stored by both devices.
- 'mitm': Sets whether MITM-protection is required for pairing.
- 'io': Sets the I/O capabilities of this device.

Available options are:

```

_IO_CAPABILITY_DISPLAY_ONLY = const(0)
_IO_CAPABILITY_DISPLAY_YESNO = const(1)
_IO_CAPABILITY_KEYBOARD_ONLY = const(2)
_IO_CAPABILITY_NO_INPUT_OUTPUT = const(3)
_IO_CAPABILITY_KEYBOARD_DISPLAY = const(4)

```

- `'le_secure'`: Sets whether “LE Secure” pairing is required. Default is false (i.e. allow “Legacy Pairing”).

Event Handling

`BLE.irq(handler, /)`

Registers a callback for events from the BLE stack. The *handler* takes two arguments, *event* (which will be one of the codes below) and *data* (which is an event-specific tuple of values).

Note: As an optimisation to prevent unnecessary allocations, the *addr*, *adv_data*, *char_data*, *notify_data*, and *uuid* entries in the tuples are read-only memoryview instances pointing to *bluetooth*’s internal ringbuffer, and are only valid during the invocation of the IRQ handler function. If your program needs to save one of these values to access after the IRQ handler has returned (e.g. by saving it in a class instance or global variable), then it needs to take a copy of the data, either by using `bytes()` or `bluetooth.UUID()`, like this:

```

connected_addr = bytes(addr) # equivalently: adv_data, char_data, or notify_data
matched_uuid = bluetooth.UUID(uuid)

```

For example, the IRQ handler for a scan result might inspect the *adv_data* to decide if it’s the correct device, and only then copy the address data to be used elsewhere in the program. And to print data from within the IRQ handler, `print(bytes(addr))` will be needed.

An event handler showing all possible events:

```

def bt_irq(event, data):
    if event == _IRQ_CENTRAL_CONNECT:
        # A central has connected to this peripheral.
        conn_handle, addr_type, addr = data
    elif event == _IRQ_CENTRAL_DISCONNECT:
        # A central has disconnected from this peripheral.
        conn_handle, addr_type, addr = data
    elif event == _IRQ_GATTS_WRITE:
        # A client has written to this characteristic or descriptor.
        conn_handle, attr_handle = data
    elif event == _IRQ_GATTS_READ_REQUEST:
        # A client has issued a read. Note: this is only supported on STM32.
        # Return a non-zero integer to deny the read (see below), or zero (or
        ↪ None)
        # to accept the read.
        conn_handle, attr_handle = data
    elif event == _IRQ_SCAN_RESULT:
        # A single scan result.
        addr_type, addr, adv_type, rssi, adv_data = data
    elif event == _IRQ_SCAN_DONE:
        # Scan duration finished or manually stopped.
        pass
    elif event == _IRQ_PERIPHERAL_CONNECT:
        # A successful gap_connect().
        conn_handle, addr_type, addr = data

```

(continues on next page)

(continued from previous page)

```

elif event == _IRQ_PERIPHERAL_DISCONNECT:
    # Connected peripheral has disconnected.
    conn_handle, addr_type, addr = data
elif event == _IRQ_GATTC_SERVICE_RESULT:
    # Called for each service found by gattc_discover_services().
    conn_handle, start_handle, end_handle, uuid = data
elif event == _IRQ_GATTC_SERVICE_DONE:
    # Called once service discovery is complete.
    # Note: Status will be zero on success, implementation-specific value_
↳ otherwise.
    conn_handle, status = data
elif event == _IRQ_GATTC_CHARACTERISTIC_RESULT:
    # Called for each characteristic found by gattc_discover_services().
    conn_handle, end_handle, value_handle, properties, uuid = data
elif event == _IRQ_GATTC_CHARACTERISTIC_DONE:
    # Called once service discovery is complete.
    # Note: Status will be zero on success, implementation-specific value_
↳ otherwise.
    conn_handle, status = data
elif event == _IRQ_GATTC_DESCRIPTOR_RESULT:
    # Called for each descriptor found by gattc_discover_descriptors().
    conn_handle, dsc_handle, uuid = data
elif event == _IRQ_GATTC_DESCRIPTOR_DONE:
    # Called once service discovery is complete.
    # Note: Status will be zero on success, implementation-specific value_
↳ otherwise.
    conn_handle, status = data
elif event == _IRQ_GATTC_READ_RESULT:
    # A gattc_read() has completed.
    conn_handle, value_handle, char_data = data
elif event == _IRQ_GATTC_READ_DONE:
    # A gattc_read() has completed.
    # Note: Status will be zero on success, implementation-specific value_
↳ otherwise.
    conn_handle, value_handle, status = data
elif event == _IRQ_GATTC_WRITE_DONE:
    # A gattc_write() has completed.
    # Note: Status will be zero on success, implementation-specific value_
↳ otherwise.
    conn_handle, value_handle, status = data
elif event == _IRQ_GATTC_NOTIFY:
    # A server has sent a notify request.
    conn_handle, value_handle, notify_data = data
elif event == _IRQ_GATTC_INDICATE:
    # A server has sent an indicate request.
    conn_handle, value_handle, notify_data = data
elif event == _IRQ_GATTS_INDICATE_DONE:
    # A client has acknowledged the indication.
    # Note: Status will be zero on successful acknowledgment, implementation-
↳ specific value otherwise.
    conn_handle, value_handle, status = data
elif event == _IRQ_MTU_EXCHANGED:

```

(continues on next page)

(continued from previous page)

```

    # ATT MTU exchange complete (either initiated by us or the remote device).
    conn_handle, mtu = data
    elif event == _IRQ_L2CAP_ACCEPT:
        # A new channel has been accepted.
        # Return a non-zero integer to reject the connection, or zero (or None)
        ↪to accept.
        conn_handle, cid, psm, our_mtu, peer_mtu = data
    elif event == _IRQ_L2CAP_CONNECT:
        # A new channel is now connected (either as a result of connecting or
        ↪accepting).
        conn_handle, cid, psm, our_mtu, peer_mtu = data
    elif event == _IRQ_L2CAP_DISCONNECT:
        # Existing channel has disconnected (status is zero), or a connection
        ↪attempt failed (non-zero status).
        conn_handle, cid, psm, status = data
    elif event == _IRQ_L2CAP_RECV:
        # New data is available on the channel. Use l2cap_recvinto to read.
        conn_handle, cid = data
    elif event == _IRQ_L2CAP_SEND_READY:
        # A previous l2cap_send that returned False has now completed and the
        ↪channel is ready to send again.
        # If status is non-zero, then the transmit buffer overflowed and the
        ↪application should re-send the data.
        conn_handle, cid, status = data
    elif event == _IRQ_CONNECTION_UPDATE:
        # The remote device has updated connection parameters.
        conn_handle, conn_interval, conn_latency, supervision_timeout, status =
        ↪data
    elif event == _IRQ_ENCRYPTION_UPDATE:
        # The encryption state has changed (likely as a result of pairing or
        ↪bonding).
        conn_handle, encrypted, authenticated, bonded, key_size = data
    elif event == _IRQ_GET_SECRET:
        # Return a stored secret.
        # If key is None, return the index'th value of this sec_type.
        # Otherwise return the corresponding value for this sec_type and key.
        sec_type, index, key = data
        return value
    elif event == _IRQ_SET_SECRET:
        # Save a secret to the store for this sec_type and key.
        sec_type, key, value = data
        return True
    elif event == _IRQ_PASSKEY_ACTION:
        # Respond to a passkey request during pairing.
        # See gap_passkey() for details.
        # action will be an action that is compatible with the configured "io"
        ↪config.
        # passkey will be non-zero if action is "numeric comparison".
        conn_handle, action, passkey = data

```

The event codes are:

```
from micropython import const
_IRQ_CENTRAL_CONNECT = const(1)
_IRQ_CENTRAL_DISCONNECT = const(2)
_IRQ_GATTS_WRITE = const(3)
_IRQ_GATTS_READ_REQUEST = const(4)
_IRQ_SCAN_RESULT = const(5)
_IRQ_SCAN_DONE = const(6)
_IRQ_PERIPHERAL_CONNECT = const(7)
_IRQ_PERIPHERAL_DISCONNECT = const(8)
_IRQ_GATTC_SERVICE_RESULT = const(9)
_IRQ_GATTC_SERVICE_DONE = const(10)
_IRQ_GATTC_CHARACTERISTIC_RESULT = const(11)
_IRQ_GATTC_CHARACTERISTIC_DONE = const(12)
_IRQ_GATTC_DESCRIPTOR_RESULT = const(13)
_IRQ_GATTC_DESCRIPTOR_DONE = const(14)
_IRQ_GATTC_READ_RESULT = const(15)
_IRQ_GATTC_READ_DONE = const(16)
_IRQ_GATTC_WRITE_DONE = const(17)
_IRQ_GATTC_NOTIFY = const(18)
_IRQ_GATTC_INDICATE = const(19)
_IRQ_GATTS_INDICATE_DONE = const(20)
_IRQ_MTU_EXCHANGED = const(21)
_IRQ_L2CAP_ACCEPT = const(22)
_IRQ_L2CAP_CONNECT = const(23)
_IRQ_L2CAP_DISCONNECT = const(24)
_IRQ_L2CAP_RECV = const(25)
_IRQ_L2CAP_SEND_READY = const(26)
_IRQ_CONNECTION_UPDATE = const(27)
_IRQ_ENCRYPTION_UPDATE = const(28)
_IRQ_GET_SECRET = const(29)
_IRQ_SET_SECRET = const(30)
```

For the `_IRQ_GATTS_READ_REQUEST` event, the available return codes are:

```
_GATTS_NO_ERROR = const(0x00)
_GATTS_ERROR_READ_NOT_PERMITTED = const(0x02)
_GATTS_ERROR_WRITE_NOT_PERMITTED = const(0x03)
_GATTS_ERROR_INSUFFICIENT_AUTHENTICATION = const(0x05)
_GATTS_ERROR_INSUFFICIENT_AUTHORIZATION = const(0x08)
_GATTS_ERROR_INSUFFICIENT_ENCRYPTION = const(0x0f)
```

For the `_IRQ_PASSKEY_ACTION` event, the available actions are:

```
_PASSKEY_ACTION_NONE = const(0)
_PASSKEY_ACTION_INPUT = const(2)
_PASSKEY_ACTION_DISPLAY = const(3)
_PASSKEY_ACTION_NUMERIC_COMPARISON = const(4)
```

In order to save space in the firmware, these constants are not included on the `bluetooth` module. Add the ones that you need from the list above to your program.

Broadcaster Role (Advertiser)

`BLE.gap_advertise(interval_us, adv_data=None, *, resp_data=None, connectable=True)`

Starts advertising at the specified interval (in **microseconds**). This interval will be rounded down to the nearest 625us. To stop advertising, set *interval_us* to `None`.

adv_data and *resp_data* can be any type that implements the buffer protocol (e.g. `bytes`, `bytearray`, `str`). *adv_data* is included in all broadcasts, and *resp_data* is send in reply to an active scan.

Note: if *adv_data* (or *resp_data*) is `None`, then the data passed to the previous call to `gap_advertise` will be reused. This allows a broadcaster to resume advertising with just `gap_advertise(interval_us)`. To clear the advertising payload pass an empty `bytes`, i.e. `b''`.

Observer Role (Scanner)

`BLE.gap_scan(duration_ms, interval_us=1280000, window_us=11250, active=False, /)`

Run a scan operation lasting for the specified duration (in **milliseconds**).

To scan indefinitely, set *duration_ms* to 0.

To stop scanning, set *duration_ms* to `None`.

Use *interval_us* and *window_us* to optionally configure the duty cycle. The scanner will run for *window_us* **microseconds** every *interval_us* **microseconds** for a total of *duration_ms* **milliseconds**. The default interval and window are 1.28 seconds and 11.25 milliseconds respectively (background scanning).

For each scan result the `_IRQ_SCAN_RESULT` event will be raised, with event data (*addr_type*, *addr*, *adv_type*, *rss_i*, *adv_data*).

addr_type values indicate public or random addresses:

- 0x00 - PUBLIC
- 0x01 - RANDOM (either static, RPA, or NRPA, the type is encoded in the address itself)

adv_type values correspond to the Bluetooth Specification:

- 0x00 - ADV_IND - connectable and scannable undirected advertising
- 0x01 - ADV_DIRECT_IND - connectable directed advertising
- 0x02 - ADV_SCAN_IND - scannable undirected advertising
- 0x03 - ADV_NONCONN_IND - non-connectable undirected advertising
- 0x04 - SCAN_RSP - scan response

active can be set `True` if you want to receive scan responses in the results.

When scanning is stopped (either due to the duration finishing or when explicitly stopped), the `_IRQ_SCAN_DONE` event will be raised.

Central Role

A central device can connect to peripherals that it has discovered using the observer role (see `gap_scan`) or with a known address.

`BLE.gap_connect(addr_type, addr, scan_duration_ms=2000, min_conn_interval_us=None, max_conn_interval_us=None, /)`

Connect to a peripheral.

See `gap_scan` for details about address types.

To cancel an outstanding connection attempt early, call `gap_connect(None)`.

On success, the `_IRQ_PERIPHERAL_CONNECT` event will be raised. If cancelling a connection attempt, the `_IRQ_PERIPHERAL_DISCONNECT` event will be raised.

The device will wait up to `scan_duration_ms` to receive an advertising payload from the device.

The connection interval can be configured in **microseconds** using either or both of `min_conn_interval_us` and `max_conn_interval_us`. Otherwise a default interval will be chosen, typically between 30000 and 50000 microseconds. A shorter interval will increase throughput, at the expense of power usage.

Peripheral Role

A peripheral device is expected to send connectable advertisements (see `gap_advertise`). It will usually be acting as a GATT server, having first registered services and characteristics using `gatts_register_services`.

When a central connects, the `_IRQ_CENTRAL_CONNECT` event will be raised.

Central & Peripheral Roles

`BLE.gap_disconnect(conn_handle, /)`

Disconnect the specified connection handle. This can either be a central that has connected to this device (if acting as a peripheral) or a peripheral that was previously connected to by this device (if acting as a central).

On success, the `_IRQ_PERIPHERAL_DISCONNECT` or `_IRQ_CENTRAL_DISCONNECT` event will be raised.

Returns `False` if the connection handle wasn't connected, and `True` otherwise.

GATT Server

A GATT server has a set of registered services. Each service may contain characteristics, which each have a value. Characteristics can also contain descriptors, which themselves have values.

These values are stored locally, and are accessed by their “value handle” which is generated during service registration. They can also be read from or written to by a remote client device. Additionally, a server can “notify” a characteristic to a connected client via a connection handle.

A device in either central or peripheral roles may function as a GATT server, however in most cases it will be more common for a peripheral device to act as the server.

Characteristics and descriptors have a default maximum size of 20 bytes. Anything written to them by a client will be truncated to this length. However, any local write will increase the maximum size, so if you want to allow larger writes from a client to a given characteristic, use `gatts_write` after registration. e.g. `gatts_write(char_handle, bytes(100))`.

`BLE.gatts_register_services(services_definition, /)`

Configures the server with the specified services, replacing any existing services.

`services_definition` is a list of **services**, where each **service** is a two-element tuple containing a UUID and a list of **characteristics**.

Each **characteristic** is a two-or-three-element tuple containing a UUID, a **flags** value, and optionally a list of **descriptors**.

Each **descriptor** is a two-element tuple containing a UUID and a **flags** value.

The **flags** are a bitwise-OR combination of the flags defined below. These set both the behaviour of the characteristic (or descriptor) as well as the security and privacy requirements.

The return value is a list (one element per service) of tuples (each element is a value handle). Characteristics and descriptor handles are flattened into the same tuple, in the order that they are defined.

The following example registers two services (Heart Rate, and Nordic UART):

```

HR_UUID = bluetooth.UUID(0x180D)
HR_CHAR = (bluetooth.UUID(0x2A37), bluetooth.FLAG_READ | bluetooth.FLAG_NOTIFY,)
HR_SERVICE = (HR_UUID, (HR_CHAR,))
UART_UUID = bluetooth.UUID('6E400001-B5A3-F393-E0A9-E50E24DCCA9E')
UART_TX = (bluetooth.UUID('6E400003-B5A3-F393-E0A9-E50E24DCCA9E'), bluetooth.FLAG_
↳READ | bluetooth.FLAG_NOTIFY,)
UART_RX = (bluetooth.UUID('6E400002-B5A3-F393-E0A9-E50E24DCCA9E'), bluetooth.FLAG_
↳WRITE,)
UART_SERVICE = (UART_UUID, (UART_TX, UART_RX,))
SERVICES = (HR_SERVICE, UART_SERVICE,)
( (hr,), (tx, rx,), ) = bt.gatts_register_services(SERVICES)

```

The three value handles (hr, tx, rx) can be used with `gatts_read`, `gatts_write`, `gatts_notify`, and `gatts_indicate`.

Note: Advertising must be stopped before registering services.

Available flags for characteristics and descriptors are:

```

from micropython import const
_FLAG_BROADCAST = const(0x0001)
_FLAG_READ = const(0x0002)
_FLAG_WRITE_NO_RESPONSE = const(0x0004)
_FLAG_WRITE = const(0x0008)
_FLAG_NOTIFY = const(0x0010)
_FLAG_INDICATE = const(0x0020)
_FLAG_AUTHENTICATED_SIGNED_WRITE = const(0x0040)

_FLAG_AUX_WRITE = const(0x0100)
_FLAG_READ_ENCRYPTED = const(0x0200)
_FLAG_READ_AUTHENTICATED = const(0x0400)
_FLAG_READ_AUTHORIZED = const(0x0800)
_FLAG_WRITE_ENCRYPTED = const(0x1000)
_FLAG_WRITE_AUTHENTICATED = const(0x2000)
_FLAG_WRITE_AUTHORIZED = const(0x4000)

```

As for the IRQs above, any required constants should be added to your Python code.

BLE.**gatts_read**(value_handle, /)

Reads the local value for this handle (which has either been written by `gatts_write` or by a remote client).

BLE.**gatts_write**(value_handle, data, send_update=False, /)

Writes the local value for this handle, which can be read by a client.

If `send_update` is `True`, then any subscribed clients will be notified (or indicated, depending on what they're subscribed to and which operations the characteristic supports) about this write.

BLE.**gatts_notify**(conn_handle, value_handle, data=None, /)

Sends a notification request to a connected client.

If `data` is `None` (the default), then the current local value (as set with `gatts_write`) will be sent.

Otherwise, if `data` is not `None`, then that value is sent to the client as part of the notification. The local value will not be modified.

Note: The notification will be sent regardless of the subscription status of the client to this characteristic.

`BLE.gatts_indicate(conn_handle, value_handle, data=None, /)`

Sends a indication request to a connected client.

If `data` is `None` (the default), then the current local value (as set with `gatts_write`) will be sent.

Otherwise, if `data` is not `None`, then that value is sent to the client as part of the indication. The local value will not be modified.

On acknowledgment (or failure, e.g. timeout), the `_IRQ_GATTS_INDICATE_DONE` event will be raised.

Note: The indication will be sent regardless of the subscription status of the client to this characteristic.

`BLE.gatts_set_buffer(value_handle, len, append=False, /)`

Sets the internal buffer size for a value in bytes. This will limit the largest possible write that can be received. The default is 20.

Setting `append` to `True` will make all remote writes append to, rather than replace, the current value. At most `len` bytes can be buffered in this way. When you use `gatts_read`, the value will be cleared after reading. This feature is useful when implementing something like the Nordic UART Service.

GATT Client

A GATT client can discover and read/write characteristics on a remote GATT server.

It is more common for a central role device to act as the GATT client, however it's also possible for a peripheral to act as a client in order to discover information about the central that has connected to it (e.g. to read the device name from the device information service).

`BLE.gattc_discover_services(conn_handle, uuid=None, /)`

Query a connected server for its services.

Optionally specify a service `uuid` to query for that service only.

For each service discovered, the `_IRQ_GATTC_SERVICE_RESULT` event will be raised, followed by `_IRQ_GATTC_SERVICE_DONE` on completion.

`BLE.gattc_discover_characteristics(conn_handle, start_handle, end_handle, uuid=None, /)`

Query a connected server for characteristics in the specified range.

Optionally specify a characteristic `uuid` to query for that characteristic only.

You can use `start_handle=1, end_handle=0xffff` to search for a characteristic in any service.

For each characteristic discovered, the `_IRQ_GATTC_CHARACTERISTIC_RESULT` event will be raised, followed by `_IRQ_GATTC_CHARACTERISTIC_DONE` on completion.

`BLE.gattc_discover_descriptors(conn_handle, start_handle, end_handle, /)`

Query a connected server for descriptors in the specified range.

For each descriptor discovered, the `_IRQ_GATTC_DESCRIPTOR_RESULT` event will be raised, followed by `_IRQ_GATTC_DESCRIPTOR_DONE` on completion.

`BLE.gattc_read(conn_handle, value_handle, /)`

Issue a remote read to a connected server for the specified characteristic or descriptor handle.

When a value is available, the `_IRQ_GATTC_READ_RESULT` event will be raised. Additionally, the `_IRQ_GATTC_READ_DONE` will be raised.

`BLE.gattc_write(conn_handle, value_handle, data, mode=0, /)`

Issue a remote write to a connected server for the specified characteristic or descriptor handle.

The argument `mode` specifies the write behaviour, with the currently supported values being:

- `mode=0` (default) is a write-without-response: the write will be sent to the remote server but no confirmation will be returned, and no event will be raised.
- `mode=1` is a write-with-response: the remote server is requested to send a response/acknowledgement that it received the data.

If a response is received from the remote server the `_IRQ_GATTC_WRITE_DONE` event will be raised.

`BLE.gattc_exchange_mtu(conn_handle, /)`

Initiate MTU exchange with a connected server, using the preferred MTU set using `BLE.config(mtu=value)`.

The `_IRQ_MTU_EXCHANGED` event will be raised when MTU exchange completes.

Note: MTU exchange is typically initiated by the central. When using the BlueKitchen stack in the central role, it does not support a remote peripheral initiating the MTU exchange. NimBLE works for both roles.

L2CAP connection-oriented-channels

This feature allows for socket-like data exchange between two BLE devices. Once the devices are connected via GAP, either device can listen for the other to connect on a numeric PSM (Protocol/Service Multiplexer).

Note: This is currently only supported when using the NimBLE stack on STM32 and Unix (not ESP32). Only one L2CAP channel may be active at a given time (i.e. you cannot connect while listening).

Active L2CAP channels are identified by the connection handle that they were established on and a CID (channel ID).

Connection-oriented channels have built-in credit-based flow control. Unlike ATT, where devices negotiate a shared MTU, both the listening and connecting devices each set an independent MTU which limits the maximum amount of outstanding data that the remote device can send before it is fully consumed in `l2cap_recvinto`.

`BLE.l2cap_listen(psm, mtu, /)`

Start listening for incoming L2CAP channel requests on the specified `psm` with the local MTU set to `mtu`.

When a remote device initiates a connection, the `_IRQ_L2CAP_ACCEPT` event will be raised, which gives the listening server a chance to reject the incoming connection (by returning a non-zero integer).

Once the connection is accepted, the `_IRQ_L2CAP_CONNECT` event will be raised, allowing the server to obtain the channel id (CID) and the local and remote MTU.

Note: It is not currently possible to stop listening.

`BLE.l2cap_connect(conn_handle, psm, mtu, /)`

Connect to a listening peer on the specified `psm` with local MTU set to `mtu`.

On successful connection, the `_IRQ_L2CAP_CONNECT` event will be raised, allowing the client to obtain the CID and the local and remote (peer) MTU.

An unsuccessful connection will raise the `_IRQ_L2CAP_DISCONNECT` event with a non-zero status.

`BLE.l2cap_disconnect(conn_handle, cid, /)`

Disconnect an active L2CAP channel with the specified `conn_handle` and `cid`.

`BLE.l2cap_send(conn_handle, cid, buf, /)`

Send the specified `buf` (which must support the buffer protocol) on the L2CAP channel identified by `conn_handle` and `cid`.

The specified buffer cannot be larger than the remote (peer) MTU, and no more than twice the size of the local MTU.

This will return `False` if the channel is now “stalled”, which means that `l2cap_send` must not be called again until the `_IRQ_L2CAP_SEND_READY` event is received (which will happen when the remote device grants more credits, typically after it has received and processed the data).

BLE. `l2cap_recvinto` (*conn_handle*, *cid*, *buf*, /)

Receive data from the specified *conn_handle* and *cid* into the provided *buf* (which must support the buffer protocol, e.g. bytearray or memoryview).

Returns the number of bytes read from the channel.

If *buf* is `None`, then returns the number of bytes available.

Note: After receiving the `_IRQ_L2CAP_RECV` event, the application should continue calling `l2cap_recvinto` until no more bytes are available in the receive buffer (typically up to the size of the remote (peer) MTU).

Until the receive buffer is empty, the remote device will not be granted more channel credits and will be unable to send any more data.

Pairing and bonding

Pairing allows a connection to be encrypted and authenticated via exchange of secrets (with optional MITM protection via passkey authentication).

Bonding is the process of storing those secrets into non-volatile storage. When bonded, a device is able to resolve a resolvable private address (RPA) from another device based on the stored identity resolving key (IRK). To support bonding, an application must implement the `_IRQ_GET_SECRET` and `_IRQ_SET_SECRET` events.

Note: This is currently only supported when using the NimBLE stack on ESP32, STM32 and Unix.

BLE. `gap_pair` (*conn_handle*, /)

Initiate pairing with the remote device.

Before calling this, ensure that the `io`, `mitm`, `le_secure`, and `bond` configuration options are set (via `config`).

On successful pairing, the `_IRQ_ENCRYPTION_UPDATE` event will be raised.

BLE. `gap_passkey` (*conn_handle*, *action*, *passkey*, /)

Respond to a `_IRQ_PASSKEY_ACTION` event for the specified *conn_handle* and *action*.

The *passkey* is a numeric value and will depend on on the *action* (which will depend on what I/O capability has been set):

- When the *action* is `_PASSKEY_ACTION_INPUT`, then the application should prompt the user to enter the passkey that is shown on the remote device.
- When the *action* is `_PASSKEY_ACTION_DISPLAY`, then the application should generate a random 6-digit passkey and show it to the user.
- When the *action* is `_PASSKEY_ACTION_NUMERIC_COMPARISON`, then the application should show the passkey that was provided in the `_IRQ_PASSKEY_ACTION` event and then respond with either 0 (cancel pairing), or 1 (accept pairing).

class UUID

Constructor

`class` `bluetooth.UUID` (*value*, /)

Creates a UUID instance with the specified **value**.

The **value** can be either:

- A 16-bit integer. e.g. 0x2908.
- An object with the buffer protocol and that is 2, 4 or 16 bytes long, e.g. `b'\x08\x29'`.
- A 128-bit UUID string. e.g. '6E400001-B5A3-F393-E0A9-E50E24DCCA9E'.

1.2.2 btree – simple BTree database

The `btree` module implements a simple key-value database using external storage (disk files, or in general case, a random-access `stream`). Keys are stored sorted in the database, and besides efficient retrieval by a key value, a database also supports efficient ordered range scans (retrieval of values with the keys in a given range). On the application interface side, BTree database work as close as possible to a way standard `dict` type works, one notable difference is that both keys and values must be *bytes*-like objects (so, if you want to store objects of other types, you need to first serialize them to `str` or `bytes` or another type that supports the buffer protocol).

The module is based on the well-known BerkelyDB library, version 1.xx.

Example:

```
import btree

# First, we need to open a stream which holds a database
# This is usually a file, but can be in-memory database
# using io.BytesIO, a raw flash partition, etc.
# Oftentimes, you want to create a database file if it doesn't
# exist and open if it exists. Idiom below takes care of this.
# DO NOT open database with "a+b" access mode.
try:
    f = open("mydb", "r+b")
except OSError:
    f = open("mydb", "w+b")

# Now open a database itself
db = btree.open(f)

# The keys you add will be sorted internally in the database
db[b"3"] = b"three"
db[b"1"] = b"one"
db[b"2"] = b"two"

# Assume that any changes are cached in memory unless
# explicitly flushed (or database closed). Flush database
# at the end of each "transaction".
db.flush()

# Prints b'two'
print(db[b"2"])

# Iterate over sorted keys in the database, starting from b"2"
# until the end of the database, returning only values.
# Mind that arguments passed to values() method are *key* values.
# Prints:
#   b'two'
#   b'three'
for word in db.values(b"2"):
```

(continues on next page)

(continued from previous page)

```
print(word)

del db[b"2"]

# No longer true, prints False
print(b"2" in db)

# Prints:
# b"1"
# b"3"
for key in db:
    print(key)

db.close()

# Don't forget to close the underlying stream!
f.close()
```

Functions

`btree.open(stream, *, flags=0, pagesize=0, cachesize=0, minkeypage=0)`

Open a database from a random-access *stream* (like an open file). All other parameters are optional and keyword-only, and allow to tweak advanced parameters of the database operation (most users will not need them):

- *flags* - Currently unused.
- *pagesize* - Page size used for the nodes in BTree. Acceptable range is 512-65536. If 0, a port-specific default will be used, optimized for port's memory usage and/or performance.
- *cachesize* - Suggested memory cache size in bytes. For a board with enough memory using larger values may improve performance. Cache policy is as follows: entire cache is not allocated at once; instead, accessing a new page in database will allocate a memory buffer for it, until value specified by *cachesize* is reached. Then, these buffers will be managed using LRU (least recently used) policy. More buffers may still be allocated if needed (e.g., if a database contains big keys and/or values). Allocated cache buffers aren't reclaimed.
- *minkeypage* - Minimum number of keys to store per page. Default value of 0 equivalent to 2.

Returns a BTree object, which implements a dictionary protocol (set of methods), and some additional methods described below.

Methods

`btree.close()`

Close the database. It's mandatory to close the database at the end of processing, as some unwritten data may be still in the cache. Note that this does not close underlying stream with which the database was opened, it should be closed separately (which is also mandatory to make sure that data flushed from buffer to the underlying storage).

`btree.flush()`

Flush any data in cache to the underlying stream.

`btree.__getitem__(key)`

`btree.get(key, default=None, /)`

`btree.__setitem__(key, val)`

`btree.__delitem__(key)`

`btree.__contains__(key)`

Standard dictionary methods.

`btree.__iter__()`

A BTree object can be iterated over directly (similar to a dictionary) to get access to all keys in order.

`btree.keys([start_key[, end_key[, flags]]])`

`btree.values([start_key[, end_key[, flags]]])`

`btree.items([start_key[, end_key[, flags]]])`

These methods are similar to standard dictionary methods, but also can take optional parameters to iterate over a key sub-range, instead of the entire database. Note that for all 3 methods, *start_key* and *end_key* arguments represent key values. For example, *values()* method will iterate over values corresponding to they key range given. None values for *start_key* means “from the first key”, no *end_key* or its value of None means “until the end of database”. By default, range is inclusive of *start_key* and exclusive of *end_key*, you can include *end_key* in iteration by passing *flags* of *btree.INCL*. You can iterate in descending key direction by passing *flags* of *btree.DESC*. The flags values can be ORed together.

Constants

`btree.INCL`

A flag for *btree.keys()*, *btree.values()*, *btree.items()* methods to specify that scanning should be inclusive of the end key.

`btree.DESC`

A flag for *btree.keys()*, *btree.values()*, *btree.items()* methods to specify that scanning should be in descending direction of keys.

1.2.3 cryptolib – cryptographic ciphers

Classes

`class cryptolib.aes`

`classmethod __init__(key, mode[, IV])`

Initialize cipher object, suitable for encryption/decryption. Note: after initialization, cipher object can be use only either for encryption or decryption. Running decrypt() operation after encrypt() or vice versa is not supported.

Parameters are:

- *key* is an encryption/decryption key (bytes-like).
- *mode* is:
 - 1 (or `cryptolib.MODE_ECB` if it exists) for Electronic Code Book (ECB).
 - 2 (or `cryptolib.MODE_CBC` if it exists) for Cipher Block Chaining (CBC).
 - 6 (or `cryptolib.MODE_CTR` if it exists) for Counter mode (CTR).
- *IV* is an initialization vector for CBC mode.
- For Counter mode, *IV* is the initial value for the counter.

`encrypt(in_buf[, out_buf])`

Encrypt *in_buf*. If no *out_buf* is given result is returned as a newly allocated *bytes* object. Otherwise, result is written into mutable buffer *out_buf*. *in_buf* and *out_buf* can also refer to the same mutable buffer, in which case data is encrypted in-place.

`decrypt(in_buf[, out_buf])`

Like `encrypt()`, but for decryption.

1.2.4 deflate – deflate compression & decompression

This module allows compression and decompression of binary data with the [DEFLATE algorithm](#) (commonly used in the zlib library and gzip archiver).

Availability:

- Added in MicroPython v1.21.
- Decompression: Enabled via the `MICROPY_PY_DEFLATE` build option, on by default on ports with the “extra features” level or higher (which is most boards).
- Compression: Enabled via the `MICROPY_PY_DEFLATE_COMPRESS` build option, on by default on ports with the “full features” level or higher (generally this means you need to build your own firmware to enable this).

Classes

class `deflate.DeflateIO(stream, format=AUTO, wbits=0, close=False, /)`

This class can be used to wrap a *stream* which is any *stream-like* object such as a file, socket, or stream (including `io.BytesIO`). It is itself a stream and implements the standard `read/readinto/write/close` methods.

The *stream* must be a blocking stream. Non-blocking streams are currently not supported.

The *format* can be set to any of the constants defined below, and defaults to `AUTO` which for decompressing will auto-detect gzip or zlib streams, and for compressing it will generate a raw stream.

The *wbits* parameter sets the base-2 logarithm of the DEFLATE dictionary window size. So for example, setting *wbits* to 10 sets the window size to 1024 bytes. Valid values are 5 to 15 inclusive (corresponding to window sizes of 32 to 32k bytes).

If *wbits* is set to 0 (the default), then for compression a window size of 256 bytes will be used (as if *wbits* was set to 8). For decompression, it depends on the format:

- `RAW` will use 256 bytes (corresponding to *wbits* set to 8).
- `ZLIB` (or `AUTO` with zlib detected) will use the value from the zlib header.
- `GZIP` (or `AUTO` with gzip detected) will use 32 kilobytes (corresponding to *wbits* set to 15).

See the [window size](#) notes below for more information about the window size, zlib, and gzip streams.

If *close* is set to `True` then the underlying stream will be closed automatically when the `deflate.DeflateIO` stream is closed. This is useful if you want to return a `deflate.DeflateIO` stream that wraps another stream and not have the caller need to know about managing the underlying stream.

If compression is enabled, a given `deflate.DeflateIO` instance supports both reading and writing. For example, a bidirectional stream like a socket can be wrapped, which allows for compression/decompression in both directions.

Constants

`deflate.AUTO`

`deflate.RAW`

`deflate.ZLIB`

`deflate.GZIP`

Supported values for the *format* parameter.

Examples

A typical use case for `deflate.DeflateIO` is to read or write a compressed file from storage:

```
import deflate

# Writing a zlib-compressed stream (uses the default window size of 256 bytes).
with open("data.gz", "wb") as f:
    with deflate.DeflateIO(f, deflate.ZLIB) as d:
        # Use d.write(...) etc

# Reading a zlib-compressed stream (auto-detect window size).
with open("data.z", "rb") as f:
    with deflate.DeflateIO(f, deflate.ZLIB) as d:
        # Use d.read(), d.readinto(), etc.
```

Because `deflate.DeflateIO` is a stream, it can be used for example with `json.dump()` and `json.load()` (and any other places streams can be used):

```
import deflate, json

# Write a dictionary as JSON in gzip format, with a
# small (64 byte) window size.
config = { ... }
with open("config.gz", "wb") as f:
    with deflate.DeflateIO(f, deflate.GZIP, 6) as f:
        json.dump(config, f)

# Read back that dictionary.
with open("config.gz", "rb") as f:
    with deflate.DeflateIO(f, deflate.GZIP, 6) as f:
        config = json.load(f)
```

If your source data is not in a stream format, you can use `io.BytesIO` to turn it into a stream suitable for use with `deflate.DeflateIO`:

```
import deflate, io

# Decompress a bytes/bytearray value.
compressed_data = get_data_z()
with deflate.DeflateIO(io.BytesIO(compressed_data), deflate.ZLIB) as d:
    decompressed_data = d.read()

# Compress a bytes/bytearray value.
uncompressed_data = get_data()
stream = io.BytesIO()
with deflate.DeflateIO(stream, deflate.ZLIB) as d:
    d.write(uncompressed_data)
compressed_data = stream.getvalue()
```

Deflate window size

The window size limits how far back in the stream the (de)compressor can reference. Increasing the window size will improve compression, but will require more memory and make the compressor slower.

If an input stream was compressed a given window size, then *DeflateIO* using a smaller window size will fail mid-way during decompression with *OSError*, but only if a back-reference actually refers back further than the decompressor's window size. This means it may be possible to decompress with a smaller window size. For example, this would trivially be the case if the original uncompressed data is shorter than the window size.

Decompression

The zlib format includes a header which specifies the window size that was used to compress the data. This indicates the maximum window size required to decompress this stream. If this header value is less than the specified *wbits* value (or if *wbits* is unset), then the header value will be used.

The gzip format does not include the window size in the header, and assumes that all gzip compressors (e.g. the *gzip* utility, or CPython's implementation of *gzip.GzipFile*) use the maximum window size of 32kiB. For this reason, if the *wbits* parameter is not set, the decompressor will use a 32 kiB window size (corresponding to *wbits* set to 15). This means that to be able to decompress an arbitrary gzip stream, you must have at least this much RAM available. If you control the source data, consider instead using the zlib format with a smaller window size.

The raw format has no header and therefore does not include any information about the window size. If *wbits* is not set, then it will default to a window size of 256 bytes, which may not be large enough for a given stream. Therefore it is recommended that you should always explicitly set *wbits* if using the raw format.

Compression

For compression, MicroPython will default to a window size of 256 bytes for all formats. This provides a reasonable amount of compression with minimal memory usage and fast compression time, and will generate output that will work with any decompressor.

1.2.5 framebuffer — frame buffer manipulation

This module provides a general frame buffer which can be used to create bitmap images, which can then be sent to a display.

class FrameBuffer

The *FrameBuffer* class provides a pixel buffer which can be drawn upon with pixels, lines, rectangles, ellipses, polygons, text and even other *FrameBuffers*. It is useful when generating output for displays.

For example:

```
import framebuffer

# FrameBuffer needs 2 bytes for every RGB565 pixel
fbuf = framebuffer.FrameBuffer(bytearray(100 * 10 * 2), 100, 10, framebuffer.RGB565)

fbuf.fill(0)
fbuf.text('MicroPython!', 0, 0, 0xffff)
fbuf.hline(0, 9, 96, 0xffff)
```

Constructors

`class framebuffer.FrameBuffer (buffer, width, height, format, stride=width, /)`

Construct a FrameBuffer object. The parameters are:

- *buffer* is an object with a buffer protocol which must be large enough to contain every pixel defined by the width, height and format of the FrameBuffer.
- *width* is the width of the FrameBuffer in pixels
- *height* is the height of the FrameBuffer in pixels
- *format* specifies the type of pixel used in the FrameBuffer; permissible values are listed under Constants below. These set the number of bits used to encode a color value and the layout of these bits in *buffer*. Where a color value *c* is passed to a method, *c* is a small integer with an encoding that is dependent on the format of the FrameBuffer.
- *stride* is the number of pixels between each horizontal line of pixels in the FrameBuffer. This defaults to *width* but may need adjustments when implementing a FrameBuffer within another larger FrameBuffer or screen. The *buffer* size must accommodate an increased step size.

One must specify valid *buffer*, *width*, *height*, *format* and optionally *stride*. Invalid *buffer* size or dimensions may lead to unexpected errors.

Drawing primitive shapes

The following methods draw shapes onto the FrameBuffer.

`FrameBuffer.fill (c)`

Fill the entire FrameBuffer with the specified color.

`FrameBuffer.pixel (x, y[, c])`

If *c* is not given, get the color value of the specified pixel. If *c* is given, set the specified pixel to the given color.

`FrameBuffer.hline (x, y, w, c)`

`FrameBuffer.vline (x, y, h, c)`

`FrameBuffer.line (x1, y1, x2, y2, c)`

Draw a line from a set of coordinates using the given color and a thickness of 1 pixel. The *line* method draws the line up to a second set of coordinates whereas the *hline* and *vline* methods draw horizontal and vertical lines respectively up to a given length.

`FrameBuffer.rect (x, y, w, h, c[, f])`

Draw a rectangle at the given location, size and color.

The optional *f* parameter can be set to `True` to fill the rectangle. Otherwise just a one pixel outline is drawn.

`FrameBuffer.ellipse (x, y, xr, yr, c[, f, m])`

Draw an ellipse at the given location. Radii *xr* and *yr* define the geometry; equal values cause a circle to be drawn. The *c* parameter defines the color.

The optional *f* parameter can be set to `True` to fill the ellipse. Otherwise just a one pixel outline is drawn.

The optional *m* parameter enables drawing to be restricted to certain quadrants of the ellipse. The LS four bits determine which quadrants are to be drawn, with bit 0 specifying Q1, b1 Q2, b2 Q3 and b3 Q4. Quadrants are numbered counterclockwise with Q1 being top right.

`Framebuffer.poly(x, y, coords, c[, f])`

Given a list of coordinates, draw an arbitrary (convex or concave) closed polygon at the given x, y location using the given color.

The *coords* must be specified as a `array` of integers, e.g. `array('h', [x0, y0, x1, y1, ... xn, yn])`.

The optional *f* parameter can be set to `True` to fill the polygon. Otherwise just a one pixel outline is drawn.

Drawing text

`Framebuffer.text(s, x, y[, c])`

Write text to the `Framebuffer` using the coordinates as the upper-left corner of the text. The color of the text can be defined by the optional argument but is otherwise a default value of 1. All characters have dimensions of 8x8 pixels and there is currently no way to change the font.

Other methods

`Framebuffer.scroll(xstep, ystep)`

Shift the contents of the `Framebuffer` by the given vector. This may leave a footprint of the previous colors in the `Framebuffer`.

`Framebuffer.blit(fbuf, x, y, key=-1, palette=None)`

Draw another `Framebuffer` on top of the current one at the given coordinates. If *key* is specified then it should be a color integer and the corresponding color will be considered transparent: all pixels with that color value will not be drawn. (If the *palette* is specified then the *key* is compared to the value from *palette*, not to the value directly from *fbuf*.)

fbuf can be another `Framebuffer` instance, or a tuple or list of the form:

`(buffer, width, height, format)`

or:

`(buffer, width, height, format, stride)`

This matches the signature of the `Framebuffer` constructor, and the elements of the tuple/list are the same as the arguments to the constructor except that the *buffer* here can be read-only.

The *palette* argument enables blitting between `FrameBuffers` with differing formats. Typical usage is to render a monochrome or grayscale glyph/icon to a color display. The *palette* is a `Framebuffer` instance whose format is that of the current `Framebuffer`. The *palette* height is one pixel and its pixel width is the number of colors in the source `Framebuffer`. The *palette* for an N-bit source needs 2*N pixels; the *palette* for a monochrome source would have 2 pixels representing background and foreground colors. The application assigns a color to each pixel in the *palette*. The color of the current pixel will be that of that *palette* pixel whose x position is the color of the corresponding source pixel.

Constants

`framebuf.MONO_VLSB`

Monochrome (1-bit) color format This defines a mapping where the bits in a byte are vertically mapped with bit 0 being nearest the top of the screen. Consequently each byte occupies 8 vertical pixels. Subsequent bytes appear at successive horizontal locations until the rightmost edge is reached. Further bytes are rendered at locations starting at the leftmost edge, 8 pixels lower.

`framebuf.MONO_HLSB`

Monochrome (1-bit) color format This defines a mapping where the bits in a byte are horizontally mapped. Each byte occupies 8 horizontal pixels with bit 7 being the leftmost. Subsequent bytes appear at successive horizontal locations until the rightmost edge is reached. Further bytes are rendered on the next row, one pixel lower.

`framebuf.MONO_HMSB`

Monochrome (1-bit) color format This defines a mapping where the bits in a byte are horizontally mapped. Each byte occupies 8 horizontal pixels with bit 0 being the leftmost. Subsequent bytes appear at successive horizontal locations until the rightmost edge is reached. Further bytes are rendered on the next row, one pixel lower.

`framebuf.RGB565`

Red Green Blue (16-bit, 5+6+5) color format

`framebuf.GS2_HMSB`

Grayscale (2-bit) color format

`framebuf.GS4_HMSB`

Grayscale (4-bit) color format

`framebuf.GS8`

Grayscale (8-bit) color format

1.2.6 `machine` — functions related to the hardware

The `machine` module contains specific functions related to the hardware on a particular board. Most functions in this module allow to achieve direct and unrestricted access to and control of hardware blocks on a system (like CPU, timers, buses, etc.). Used incorrectly, this can lead to malfunction, lockups, crashes of your board, and in extreme cases, hardware damage.

A note of callbacks used by functions and class methods of `machine` module: all these callbacks should be considered as executing in an interrupt context. This is true for both physical devices with IDs ≥ 0 and “virtual” devices with negative IDs like -1 (these “virtual” devices are still thin shims on top of real hardware and real hardware interrupts). See [Writing interrupt handlers](#).

Memory access

The module exposes three objects used for raw memory access.

`machine.mem8`

Read/write 8 bits of memory.

`machine.mem16`

Read/write 16 bits of memory.

`machine.mem32`

Read/write 32 bits of memory.

Use subscript notation `[...]` to index these objects with the address of interest. Note that the address is the byte address, regardless of the size of memory being accessed.

Example use (registers are specific to an stm32 microcontroller):

```
import machine
from micropython import const

GPIOA = const(0x48000000)
GPIO_BSRR = const(0x18)
```

(continues on next page)

(continued from previous page)

```
GPIO_IDR = const(0x10)

# set PA2 high
machine.mem32[GPIOA + GPIO_BSRR] = 1 << 2

# read PA3
value = (machine.mem32[GPIOA + GPIO_IDR] >> 3) & 1
```

Reset related functions

`machine.reset()`

Hard resets the device in a manner similar to pushing the external RESET button.

`machine.soft_reset()`

Performs a *soft reset* of the interpreter, deleting all Python objects and resetting the Python heap.

`machine.reset_cause()`

Get the reset cause. See *constants* for the possible return values.

`machine.bootloader([value])`

Reset the device and enter its bootloader. This is typically used to put the device into a state where it can be programmed with new firmware.

Some ports support passing in an optional *value* argument which can control which bootloader to enter, what to pass to it, or other things.

Interrupt related functions

The following functions allow control over interrupts. Some systems require interrupts to operate correctly so disabling them for long periods may compromise core functionality, for example watchdog timers may trigger unexpectedly. Interrupts should only be disabled for a minimum amount of time and then re-enabled to their previous state. For example:

```
import machine

# Disable interrupts
state = machine.disable_irq()

# Do a small amount of time-critical work here

# Enable interrupts
machine.enable_irq(state)
```

`machine.disable_irq()`

Disable interrupt requests. Returns the previous IRQ state which should be considered an opaque value. This return value should be passed to the `enable_irq()` function to restore interrupts to their original state, before `disable_irq()` was called.

`machine.enable_irq(state)`

Re-enable interrupt requests. The *state* parameter should be the value that was returned from the most recent call to the `disable_irq()` function.

Power related functions

`machine.freq([hz])`

Returns the CPU frequency in hertz.

On some ports this can also be used to set the CPU frequency by passing in *hz*.

`machine.idle()`

Gates the clock to the CPU, useful to reduce power consumption at any time during short or long periods. Peripherals continue working and execution resumes as soon as any interrupt is triggered, or at most one millisecond after the CPU was paused.

It is recommended to call this function inside any tight loop that is continuously checking for an external change (i.e. polling). This will reduce power consumption without significantly impacting performance. To reduce power consumption further then see the `lightsleep()`, `time.sleep()` and `time.sleep_ms()` functions.

`machine.sleep()`

Note

This function is deprecated, use `lightsleep()` instead with no arguments.

`machine.lightsleep([time_ms])`

`machine.deepsleep([time_ms])`

Stops execution in an attempt to enter a low power state.

If *time_ms* is specified then this will be the maximum time in milliseconds that the sleep will last for. Otherwise the sleep can last indefinitely.

With or without a timeout, execution may resume at any time if there are events that require processing. Such events, or wake sources, should be configured before sleeping, like *Pin* change or *RTC* timeout.

The precise behaviour and power-saving capabilities of `lightsleep` and `deepsleep` is highly dependent on the underlying hardware, but the general properties are:

- A `lightsleep` has full RAM and state retention. Upon wake execution is resumed from the point where the sleep was requested, with all subsystems operational.
- A `deepsleep` may not retain RAM or any other state of the system (for example peripherals or network interfaces). Upon wake execution is resumed from the main script, similar to a hard or power-on reset. The `reset_cause()` function will return `machine.DEEPSLEEP` and this can be used to distinguish a `deepsleep` wake from other resets.

`machine.wake_reason()`

Get the wake reason. See *constants* for the possible return values.

Availability: ESP32, WiPy.

Miscellaneous functions

`machine.unique_id()`

Returns a byte string with a unique identifier of a board/SoC. It will vary from a board/SoC instance to another, if underlying hardware allows. Length varies by hardware (so use substring of a full value if you expect a short ID). In some MicroPython ports, ID corresponds to the network MAC address.

```
machine.time_pulse_us(pin, pulse_level, timeout_us=1000000, /)
```

Time a pulse on the given *pin*, and return the duration of the pulse in microseconds. The *pulse_level* argument should be 0 to time a low pulse or 1 to time a high pulse.

If the current input value of the pin is different to *pulse_level*, the function first (*) waits until the pin input becomes equal to *pulse_level*, then (**) times the duration that the pin is equal to *pulse_level*. If the pin is already equal to *pulse_level* then timing starts straight away.

The function will return -2 if there was timeout waiting for condition marked (*) above, and -1 if there was timeout during the main measurement, marked (**) above. The timeout is the same for both cases and given by *timeout_us* (which is in microseconds).

```
machine.bitstream(pin, encoding, timing, data, /)
```

Transmits *data* by bit-banging the specified *pin*. The *encoding* argument specifies how the bits are encoded, and *timing* is an encoding-specific timing specification.

The supported encodings are:

- 0 for “high low” pulse duration modulation. This will transmit 0 and 1 bits as timed pulses, starting with the most significant bit. The *timing* must be a four-tuple of nanoseconds in the format (*high_time_0*, *low_time_0*, *high_time_1*, *low_time_1*). For example, (400, 850, 800, 450) is the timing specification for WS2812 RGB LEDs at 800kHz.

The accuracy of the timing varies between ports. On Cortex M0 at 48MHz, it is at best +/- 120ns, however on faster MCUs (ESP8266, ESP32, STM32, Pyboard), it will be closer to +/-30ns.

Note

For controlling WS2812 / NeoPixel strips, see the [neopixel](#) module for a higher-level API.

```
machine.rng()
```

Return a 24-bit software generated random number.

Availability: WiPy.

Constants

```
machine.IDLE
```

```
machine.SLEEP
```

```
machine.DEEPSLEEP
```

IRQ wake values.

```
machine.PWRON_RESET
```

```
machine.HARD_RESET
```

```
machine.WDT_RESET
```

```
machine.DEEPSLEEP_RESET
```

```
machine.SOFT_RESET
```

Reset causes.

```
machine.WLAN_WAKE
```

```
machine.PIN_WAKE
```

```
machine.RTC_WAKE
```

Wake-up reasons.

Classes

class Pin – control I/O pins

A pin object is used to control I/O pins (also known as GPIO - general-purpose input/output). Pin objects are commonly associated with a physical pin that can drive an output voltage and read input voltages. The pin class has methods to set the mode of the pin (IN, OUT, etc) and methods to get and set the digital logic level. For analog control of a pin, see the [ADC](#) class.

A pin object is constructed by using an identifier which unambiguously specifies a certain I/O pin. The allowed forms of the identifier and the physical pin that the identifier maps to are port-specific. Possibilities for the identifier are an integer, a string or a tuple with port and pin number.

Usage Model:

```
from machine import Pin

# create an output pin on pin #0
p0 = Pin(0, Pin.OUT)

# set the value low then high
p0.value(0)
p0.value(1)

# create an input pin on pin #2, with a pull up resistor
p2 = Pin(2, Pin.IN, Pin.PULL_UP)

# read and print the pin value
print(p2.value())

# reconfigure pin #0 in input mode with a pull down resistor
p0.init(p0.IN, p0.PULL_DOWN)

# configure an irq callback
p0.irq(lambda p: print(p))
```

Constructors

class machine.Pin(*id*, *mode=-1*, *pull=-1*, *, *value=None*, *drive=0*, *alt=-1*)

Access the pin peripheral (GPIO pin) associated with the given *id*. If additional arguments are given in the constructor then they are used to initialise the pin. Any settings that are not specified will remain in their previous state.

The arguments are:

- *id* is mandatory and can be an arbitrary object. Among possible value types are: int (an internal Pin identifier), str (a Pin name), and tuple (pair of [port, pin]).
- *mode* specifies the pin mode, which can be one of:
 - Pin.IN - Pin is configured for input. If viewed as an output the pin is in high-impedance state.
 - Pin.OUT - Pin is configured for (normal) output.
 - Pin.OPEN_DRAIN - Pin is configured for open-drain output. Open-drain output works in the following way: if the output value is set to 0 the pin is active at a low level; if the output value is 1 the pin is in a high-impedance state. Not all ports implement this mode, or some might only on certain pins.

- `Pin.ALT` - Pin is configured to perform an alternative function, which is port specific. For a pin configured in such a way any other Pin methods (except `Pin.init()`) are not applicable (calling them will lead to undefined, or a hardware-specific, result). Not all ports implement this mode.
- `Pin.ALT_OPEN_DRAIN` - The Same as `Pin.ALT`, but the pin is configured as open-drain. Not all ports implement this mode.
- `Pin.ANALOG` - Pin is configured for analog input, see the [ADC](#) class.
- `pull` specifies if the pin has a (weak) pull resistor attached, and can be one of:
 - `None` - No pull up or down resistor.
 - `Pin.PULL_UP` - Pull up resistor enabled.
 - `Pin.PULL_DOWN` - Pull down resistor enabled.
- `value` is valid only for `Pin.OUT` and `Pin.OPEN_DRAIN` modes and specifies initial output pin value if given, otherwise the state of the pin peripheral remains unchanged.
- `drive` specifies the output power of the pin and can be one of: `Pin.DRIVE_0`, `Pin.DRIVE_1`, etc., increasing in drive strength. The actual current driving capabilities are port dependent. Not all ports implement this argument.
- `alt` specifies an alternate function for the pin and the values it can take are port dependent. This argument is valid only for `Pin.ALT` and `Pin.ALT_OPEN_DRAIN` modes. It may be used when a pin supports more than one alternate function. If only one pin alternate function is supported the this argument is not required. Not all ports implement this argument.

As specified above, the `Pin` class allows to set an alternate function for a particular pin, but it does not specify any further operations on such a pin. Pins configured in alternate-function mode are usually not used as GPIO but are instead driven by other hardware peripherals. The only operation supported on such a pin is re-initialising, by calling the constructor or `Pin.init()` method. If a pin that is configured in alternate-function mode is re-initialised with `Pin.IN`, `Pin.OUT`, or `Pin.OPEN_DRAIN`, the alternate function will be removed from the pin.

Methods

`Pin.init(mode=-1, pull=-1, *, value=None, drive=0, alt=-1)`

Re-initialise the pin using the given parameters. Only those arguments that are specified will be set. The rest of the pin peripheral state will remain unchanged. See the constructor documentation for details of the arguments.

Returns `None`.

`Pin.value([x])`

This method allows to set and get the value of the pin, depending on whether the argument `x` is supplied or not.

If the argument is omitted then this method gets the digital logic level of the pin, returning 0 or 1 corresponding to low and high voltage signals respectively. The behaviour of this method depends on the mode of the pin:

- `Pin.IN` - The method returns the actual input value currently present on the pin.
- `Pin.OUT` - The behaviour and return value of the method is undefined.
- `Pin.OPEN_DRAIN` - If the pin is in state '0' then the behaviour and return value of the method is undefined. Otherwise, if the pin is in state '1', the method returns the actual input value currently present on the pin.

If the argument is supplied then this method sets the digital logic level of the pin. The argument `x` can be anything that converts to a boolean. If it converts to `True`, the pin is set to state '1', otherwise it is set to state '0'. The behaviour of this method depends on the mode of the pin:

- `Pin.IN` - The value is stored in the output buffer for the pin. The pin state does not change, it remains in the high-impedance state. The stored value will become active on the pin as soon as it is changed to `Pin.OUT` or `Pin.OPEN_DRAIN` mode.

- `Pin.OUT` - The output buffer is set to the given value immediately.
- `Pin.OPEN_DRAIN` - If the value is '0' the pin is set to a low voltage state. Otherwise the pin is set to high-impedance state.

When setting the value this method returns `None`.

`Pin.__call__([x])`

Pin objects are callable. The call method provides a (fast) shortcut to set and get the value of the pin. It is equivalent to `Pin.value([x])`. See [Pin.value\(\)](#) for more details.

`Pin.on()`

Set pin to “1” output level.

`Pin.off()`

Set pin to “0” output level.

`Pin.irq(handler=None, trigger=Pin.IRQ_FALLING | Pin.IRQ_RISING, *, priority=1, wake=None, hard=False)`

Configure an interrupt handler to be called when the trigger source of the pin is active. If the pin mode is `Pin.IN` then the trigger source is the external value on the pin. If the pin mode is `Pin.OUT` then the trigger source is the output buffer of the pin. Otherwise, if the pin mode is `Pin.OPEN_DRAIN` then the trigger source is the output buffer for state '0' and the external pin value for state '1'.

The arguments are:

- `handler` is an optional function to be called when the interrupt triggers. The handler must take exactly one argument which is the `Pin` instance.
- `trigger` configures the event which can generate an interrupt. Possible values are:
 - `Pin.IRQ_FALLING` interrupt on falling edge.
 - `Pin.IRQ_RISING` interrupt on rising edge.
 - `Pin.IRQ_LOW_LEVEL` interrupt on low level.
 - `Pin.IRQ_HIGH_LEVEL` interrupt on high level.

These values can be OR'ed together to trigger on multiple events.

- `priority` sets the priority level of the interrupt. The values it can take are port-specific, but higher values always represent higher priorities.
- `wake` selects the power mode in which this interrupt can wake up the system. It can be `machine.IDLE`, `machine.SLEEP` or `machine.DEEPSLEEP`. These values can also be OR'ed together to make a pin generate interrupts in more than one power mode.
- `hard` if true a hardware interrupt is used. This reduces the delay between the pin change and the handler being called. Hard interrupt handlers may not allocate memory; see [Writing interrupt handlers](#). Not all ports support this argument.

This method returns a callback object.

The following methods are not part of the core Pin API and only implemented on certain ports.

`Pin.low()`

Set pin to “0” output level.

Availability: mimxrt, nrf, renesas-ra, rp2, samd, stm32 ports.

`Pin.high()`

Set pin to “1” output level.

Availability: mimxrt, nrf, renesas-ra, rp2, samd, stm32 ports.

`Pin.mode([mode])`

Get or set the pin mode. See the constructor documentation for details of the `mode` argument.

Availability: cc3200, stm32 ports.

`Pin.pull([pull])`

Get or set the pin pull state. See the constructor documentation for details of the `pull` argument.

Availability: cc3200, stm32 ports.

`Pin.drive([drive])`

Get or set the pin drive strength. See the constructor documentation for details of the `drive` argument.

Availability: cc3200 port.

`Pin.toggle()`

Toggle output pin from “0” to “1” or vice-versa.

Availability: cc3200, esp32, esp8266, mimxrt, rp2, samd ports.

Constants

The following constants are used to configure the pin objects. Note that not all constants are available on all ports.

`Pin.IN`

`Pin.OUT`

`Pin.OPEN_DRAIN`

`Pin.ALT`

`Pin.ALT_OPEN_DRAIN`

`Pin.ANALOG`

Selects the pin mode.

`Pin.PULL_UP`

`Pin.PULL_DOWN`

`Pin.PULL_HOLD`

Selects whether there is a pull up/down resistor. Use the value `None` for no pull.

`Pin.DRIVE_0`

`Pin.DRIVE_1`

`Pin.DRIVE_2`

Selects the pin drive strength. A port may define additional drive constants with increasing number corresponding to increasing drive strength.

`Pin.IRQ_FALLING`

`Pin.IRQ_RISING`

`Pin.IRQ_LOW_LEVEL`

`Pin.IRQ_HIGH_LEVEL`

Selects the IRQ trigger type.

class `Signal` – control and sense external I/O devices

The `Signal` class is a simple extension of the `Pin` class. Unlike `Pin`, which can be only in “absolute” 0 and 1 states, a `Signal` can be in “asserted” (on) or “deasserted” (off) states, while being inverted (active-low) or not. In other words, it adds logical inversion support to `Pin` functionality. While this may seem a simple addition, it is exactly what is needed to support wide array of simple digital devices in a way portable across different boards, which is one of the major

MicroPython goals. Regardless of whether different users have an active-high or active-low LED, a normally open or normally closed relay - you can develop a single, nicely looking application which works with each of them, and capture hardware configuration differences in few lines in the config file of your app.

Example:

```
from machine import Pin, Signal

# Suppose you have an active-high LED on pin 0
led1_pin = Pin(0, Pin.OUT)
# ... and active-low LED on pin 1
led2_pin = Pin(1, Pin.OUT)

# Now to light up both of them using Pin class, you'll need to set
# them to different values
led1_pin.value(1)
led2_pin.value(0)

# Signal class allows to abstract away active-high/active-low
# difference
led1 = Signal(led1_pin, invert=False)
led2 = Signal(led2_pin, invert=True)

# Now lighting up them looks the same
led1.value(1)
led2.value(1)

# Even better:
led1.on()
led2.on()
```

Following is the guide when Signal vs Pin should be used:

- Use Signal: If you want to control a simple on/off (including software PWM!) devices like LEDs, multi-segment indicators, relays, buzzers, or read simple binary sensors, like normally open or normally closed buttons, pulled high or low, Reed switches, moisture/flame detectors, etc. etc. Summing up, if you have a real physical device/sensor requiring GPIO access, you likely should use a Signal.
- Use Pin: If you implement a higher-level protocol or bus to communicate with more complex devices.

The split between Pin and Signal come from the use cases above and the architecture of MicroPython: Pin offers the lowest overhead, which may be important when bit-banging protocols. But Signal adds additional flexibility on top of Pin, at the cost of minor overhead (much smaller than if you implemented active-high vs active-low device differences in Python manually!). Also, Pin is a low-level object which needs to be implemented for each support board, while Signal is a high-level object which comes for free once Pin is implemented.

If in doubt, give the Signal a try! Once again, it is offered to save developers from the need to handle unexciting differences like active-low vs active-high signals, and allow other users to share and enjoy your application, instead of being frustrated by the fact that it doesn't work for them simply because their LEDs or relays are wired in a slightly different way.

Constructors

```
class machine.Signal(pin_obj, invert=False)
class machine.Signal(pin_arguments..., *, invert=False)
```

Create a Signal object. There're two ways to create it:

- By wrapping existing Pin object - universal method which works for any board.

- By passing required Pin parameters directly to Signal constructor, skipping the need to create intermediate Pin object. Available on many, but not all boards.

The arguments are:

- `pin_obj` is existing Pin object.
- `pin_arguments` are the same arguments as can be passed to Pin constructor.
- `invert` - if True, the signal will be inverted (active low).

Methods

`Signal.value([x])`

This method allows to set and get the value of the signal, depending on whether the argument `x` is supplied or not.

If the argument is omitted then this method gets the signal level, 1 meaning signal is asserted (active) and 0 - signal inactive.

If the argument is supplied then this method sets the signal level. The argument `x` can be anything that converts to a boolean. If it converts to `True`, the signal is active, otherwise it is inactive.

Correspondence between signal being active and actual logic level on the underlying pin depends on whether signal is inverted (active-low) or not. For non-inverted signal, active status corresponds to logical 1, inactive - to logical 0. For inverted/active-low signal, active status corresponds to logical 0, while inactive - to logical 1.

`Signal.on()`

Activate signal.

`Signal.off()`

Deactivate signal.

class ADC – analog to digital conversion

The ADC class provides an interface to analog-to-digital converters, and represents a single endpoint that can sample a continuous voltage and convert it to a discretised value.

For extra control over ADC sampling see [*machine.ADCBlock*](#).

Example usage:

```
from machine import ADC

adc = ADC(pin)           # create an ADC object acting on a pin
val = adc.read_u16()     # read a raw analog value in the range 0-65535
val = adc.read_uv()      # read an analog value in microvolts
```

Constructors

class `machine.ADC(id, *, sample_ns, atten)`

Access the ADC associated with a source identified by `id`. This `id` may be an integer (usually specifying a channel number), a [*Pin*](#) object, or other value supported by the underlying machine.

If additional keyword-arguments are given then they will configure various aspects of the ADC. If not given, these settings will take previous or default values. The settings are:

- `sample_ns` is the sampling time in nanoseconds.
- `atten` specifies the input attenuation.

Methods

`ADC.init(*, sample_ns, atten)`

Apply the given settings to the ADC. Only those arguments that are specified will be changed. See the ADC constructor above for what the arguments are.

`ADC.block()`

Return the *ADCBlock* instance associated with this ADC object.

This method only exists if the port supports the *ADCBlock* class.

`ADC.read_u16()`

Take an analog reading and return an integer in the range 0-65535. The return value represents the raw reading taken by the ADC, scaled such that the minimum value is 0 and the maximum value is 65535.

`ADC.read_uv()`

Take an analog reading and return an integer value with units of microvolts. It is up to the particular port whether or not this value is calibrated, and how calibration is done.

class ADCBlock – control ADC peripherals

The ADCBlock class provides access to an ADC peripheral which has a number of channels that can be used to sample analog values. It allows finer control over configuration of *machine.ADC* objects, which do the actual sampling.

This class is not always available.

Example usage:

```
from machine import ADCBlock

block = ADCBlock(id, bits=12)  # create an ADCBlock with 12-bit resolution
adc = block.connect(4, pin)    # connect channel 4 to the given pin
val = adc.read_uv()            # read an analog value
```

Constructors

`class machine.ADCBlock(id, *, bits)`

Access the ADC peripheral identified by *id*, which may be an integer or string.

The *bits* argument, if given, sets the resolution in bits of the conversion process. If not specified then the previous or default resolution is used.

Methods

`ADCBlock.init(*, bits)`

Configure the ADC peripheral. *bits* will set the resolution of the conversion process.

`ADCBlock.connect(channel, *, ...)`

`ADCBlock.connect(source, *, ...)`

`ADCBlock.connect(channel, source, *, ...)`

Connect up a channel on the ADC peripheral so it is ready for sampling, and return an *ADC* object that represents that connection.

The *channel* argument must be an integer, and *source* must be an object (for example a *Pin*) which can be connected up for sampling.

If only *channel* is given then it is configured for sampling.

If only *source* is given then that object is connected to a default channel ready for sampling.

If both *channel* and *source* are given then they are connected together and made ready for sampling.

Any additional keyword arguments are used to configure the returned ADC object, via its *init* method.

class PWM – pulse width modulation

This class provides pulse width modulation output.

Example usage:

```
from machine import PWM

pwm = PWM(pin, freq=50, duty_u16=8192)  # create a PWM object on a pin
                                       # and set freq 50 Hz and duty 12.5%
pwm.duty_u16(32768)                    # set duty to 50%

# reinitialise with a period of 200us, duty of 5us
pwm.init(freq=5000, duty_ns=5000)

pwm.duty_ns(3000)                      # set pulse width to 3us

pwm.deinit()
```

Constructors

class machine.**PWM**(*dest*, *, *freq*, *duty_u16*, *duty_ns*, *invert=False*)

Construct and return a new PWM object using the following parameters:

- *dest* is the entity on which the PWM is output, which is usually a *machine.Pin* object, but a port may allow other values, like integers.
- *freq* should be an integer which sets the frequency in Hz for the PWM cycle.
- *duty_u16* sets the duty cycle as a ratio `duty_u16 / 65535`.
- *duty_ns* sets the pulse width in nanoseconds.
- *invert* inverts the respective output if the value is True

Setting *freq* may affect other PWM objects if the objects share the same underlying PWM generator (this is hardware specific). Only one of *duty_u16* and *duty_ns* should be specified at a time. *invert* is available only on the esp32, mimxrt, nrf, rp2, samd and zephyr ports.

Methods

PWM.**init**(*, *freq*, *duty_u16*, *duty_ns*)

Modify settings for the PWM object. See the above constructor for details about the parameters.

PWM.**deinit**()

Disable the PWM output.

PWM.**freq**([*value*])

Get or set the current frequency of the PWM output.

With no arguments the frequency in Hz is returned.

With a single *value* argument the frequency is set to that value in Hz. The method may raise a `ValueError` if the frequency is outside the valid range.

`PWM.duty_u16([value])`

Get or set the current duty cycle of the PWM output, as an unsigned 16-bit value in the range 0 to 65535 inclusive.

With no arguments the duty cycle is returned.

With a single *value* argument the duty cycle is set to that value, measured as the ratio `value / 65535`.

`PWM.duty_ns([value])`

Get or set the current pulse width of the PWM output, as a value in nanoseconds.

With no arguments the pulse width in nanoseconds is returned.

With a single *value* argument the pulse width is set to that value.

Specific PWM class implementations

The following concrete class(es) implement enhancements to the PWM class.

pyb.Timer for PyBoard

Limitations of PWM

- Not all frequencies can be generated with absolute accuracy due to the discrete nature of the computing hardware. Typically the PWM frequency is obtained by dividing some integer base frequency by an integer divider. For example, if the base frequency is 80MHz and the required PWM frequency is 300kHz the divider must be a non-integer number $80000000 / 300000 = 266.67$. After rounding the divider is set to 267 and the PWM frequency will be $80000000 / 267 = 299625.5$ Hz, not 300kHz. If the divider is set to 266 then the PWM frequency will be $80000000 / 266 = 300751.9$ Hz, but again not 300kHz.

Some ports like the RP2040 one use a fractional divider, which allow a finer granularity of the frequency at higher frequencies by switching the PWM pulse duration between two adjacent values, such that the resulting average frequency is more close to the intended one, at the cost of spectral purity.

- The duty cycle has the same discrete nature and its absolute accuracy is not achievable. On most hardware platforms the duty will be applied at the next frequency period. Therefore, you should wait more than “1/frequency” before measuring the duty.
- The frequency and the duty cycle resolution are usually interdependent. The higher the PWM frequency the lower the duty resolution which is available, and vice versa. For example, a 300kHz PWM frequency can have a duty cycle resolution of 8 bit, not 16-bit as may be expected. In this case, the lowest 8 bits of *duty_u16* are insignificant. So:

```
pwm=PWM(Pin(13), freq=300_000, duty_u16=65536//2)
```

and:

```
pwm=PWM(Pin(13), freq=300_000, duty_u16=65536//2 + 255)
```

will generate PWM with the same 50% duty cycle.

class UART – duplex serial communication bus

UART implements the standard UART/USART duplex serial communications protocol. At the physical level it consists of 2 lines: RX and TX. The unit of communication is a character (not to be confused with a string character) which can be 8 or 9 bits wide.

UART objects can be created and initialised using:

```
from machine import UART

uart = UART(1, 9600) # init with given baudrate
uart.init(9600, bits=8, parity=None, stop=1) # init with given parameters
```

Supported parameters differ on a board:

Pyboard: Bits can be 7, 8 or 9. Stop can be 1 or 2. With *parity=None*, only 8 and 9 bits are supported. With parity enabled, only 7 and 8 bits are supported.

WiPy/CC3200: Bits can be 5, 6, 7, 8. Stop can be 1 or 2.

A UART object acts like a *stream* object and reading and writing is done using the standard stream methods:

```
uart.read(10) # read 10 characters, returns a bytes object
uart.read()   # read all available characters
uart.readline() # read a line
uart.readinto(buf) # read and store into the given buffer
uart.write('abc') # write the 3 characters
```

Constructors

```
class machine.UART(id, ...)
```

Construct a UART object of the given id.

Methods

```
UART.init(baudrate=9600, bits=8, parity=None, stop=1, *, ...)
```

Initialise the UART bus with the given parameters:

- *baudrate* is the clock rate.
- *bits* is the number of bits per character, 7, 8 or 9.
- *parity* is the parity, `None`, 0 (even) or 1 (odd).
- *stop* is the number of stop bits, 1 or 2.

Additional keyword-only parameters that may be supported by a port are:

- *tx* specifies the TX pin to use.
- *rx* specifies the RX pin to use.
- *rts* specifies the RTS (output) pin to use for hardware receive flow control.
- *cts* specifies the CTS (input) pin to use for hardware transmit flow control.
- *txbuf* specifies the length in characters of the TX buffer.
- *rxbuf* specifies the length in characters of the RX buffer.
- *timeout* specifies the time to wait for the first character (in ms).
- *timeout_char* specifies the time to wait between characters (in ms).
- *invert* specifies which lines to invert.
 - 0 will not invert lines (idle state of both lines is logic high).
 - `UART.INV_TX` will invert TX line (idle state of TX line now logic low).
 - `UART.INV_RX` will invert RX line (idle state of RX line now logic low).

- `UART.INV_TX` | `UART.INV_RX` will invert both lines (idle state at logic low).
- *flow* specifies which hardware flow control signals to use. The value is a bitmask.
 - 0 will ignore hardware flow control signals.
 - `UART.RTS` will enable receive flow control by using the RTS output pin to signal if the receive FIFO has sufficient space to accept more data.
 - `UART.CTS` will enable transmit flow control by pausing transmission when the CTS input pin signals that the receiver is running low on buffer space.
 - `UART.RTS` | `UART.CTS` will enable both, for full hardware flow control.

On the WiPy only the following keyword-only parameter is supported:

- *pins* is a 4 or 2 item list indicating the TX, RX, RTS and CTS pins (in that order). Any of the pins can be `None` if one wants the UART to operate with limited functionality. If the RTS pin is given the RX pin must be given as well. The same applies to CTS. When no pins are given, then the default set of TX and RX pins is taken, and hardware flow control will be disabled. If *pins* is `None`, no pin assignment will be made.

Note

It is possible to call `init()` multiple times on the same object in order to reconfigure UART on the fly. That allows using single UART peripheral to serve different devices attached to different GPIO pins. Only one device can be served at a time in that case. Also do not call `deinit()` as it will prevent calling `init()` again.

`UART.deinit()`

Turn off the UART bus.

Note

You will not be able to call `init()` on the object after `deinit()`. A new instance needs to be created in that case.

`UART.any()`

Returns an integer counting the number of characters that can be read without blocking. It will return 0 if there are no characters available and a positive number if there are characters. The method may return 1 even if there is more than one character available for reading.

For more sophisticated querying of available characters use `select.poll`:

```
poll = select.poll()
poll.register(uart, select.POLLIN)
poll.poll(timeout)
```

`UART.read([nbytes])`

Read characters. If *nbytes* is specified then read at most that many bytes, otherwise read as much data as possible. It may return sooner if a timeout is reached. The timeout is configurable in the constructor.

Return value: a bytes object containing the bytes read in. Returns `None` on timeout.

`UART.readinto(buf[, nbytes])`

Read bytes into the *buf*. If *nbytes* is specified then read at most that many bytes. Otherwise, read at most `len(buf)` bytes. It may return sooner if a timeout is reached. The timeout is configurable in the constructor.

Return value: number of bytes read and stored into *buf* or `None` on timeout.

`UART.readline()`

Read a line, ending in a newline character. It may return sooner if a timeout is reached. The timeout is configurable in the constructor.

Return value: the line read or `None` on timeout.

`UART.write(buf)`

Write the buffer of bytes to the bus.

Return value: number of bytes written or `None` on timeout.

`UART.sendbreak()`

Send a break condition on the bus. This drives the bus low for a duration longer than required for a normal transmission of a character.

`UART.flush()`

Waits until all data has been sent. In case of a timeout, an exception is raised. The timeout duration depends on the tx buffer size and the baud rate. Unless flow control is enabled, a timeout should not occur.

Note

For the esp8266 and nrf ports the call returns while the last byte is sent. If required, a one character wait time has to be added in the calling script.

Availability: rp2, esp32, esp8266, mimxrt, cc3200, stm32, nrf ports, renesas-ra

`UART.txdone()`

Tells whether all data has been sent or no data transfer is happening. In this case, it returns `True`. If a data transmission is ongoing it returns `False`.

Note

For the esp8266 and nrf ports the call may return `True` even if the last byte of a transfer is still being sent. If required, a one character wait time has to be added in the calling script.

Availability: rp2, esp32, esp8266, mimxrt, cc3200, stm32, nrf ports, renesas-ra

`UART irq(handler=None, trigger=0, hard=False)`

Configure an interrupt handler to be called when a UART event occurs.

The arguments are:

- *handler* is an optional function to be called when the interrupt event triggers. The handler must take exactly one argument which is the `UART` instance.
- *trigger* configures the event(s) which can generate an interrupt. Possible values are a mask of one or more of the following:
 - `UART.IRQ_RXIDLE` interrupt after receiving at least one character and then the RX line goes idle.
 - `UART.IRQ_RX` interrupt after each received character.
 - `UART.IRQ_TXIDLE` interrupt after or while the last character(s) of a message are or have been sent.
 - `UART.IRQ_BREAK` interrupt when a break state is detected at RX
- *hard* if true a hardware interrupt is used. This reduces the delay between the pin change and the handler being called. Hard interrupt handlers may not allocate memory; see [Writing interrupt handlers](#).

Returns an irq object.

Due to limitations of the hardware not all trigger events are available on all ports.

Table 1: Availability of triggers

Port / Trigger	IRQ_RXIDLE	IRQ_RX	IRQ_TXIDLE	IRQ_BREAK
CC3200		yes		
ESP32	yes	yes		yes
MIMXRT	yes		yes	
NRF		yes	yes	
RENESAS-RA	yes	yes		
RP2	yes		yes	yes
SAMD	yes	yes	yes	
STM32	yes	yes		

Note

- The ESP32 port does not support the option `hard=True`. It uses `Timer(0)` for `UART.IRQ_RXIDLE`, so this timer cannot be used for other means.
- The rp2 port's `UART.IRQ_TXIDLE` is only triggered when the message is longer than 5 characters and the trigger happens when still 5 characters are to be sent.
- The rp2 port's `UART.IRQ_BREAK` needs receiving valid characters for triggering again.
- The SAMD port's `UART.IRQ_TXIDLE` is triggered while the last character is sent.
- On STM32F4xx MCU's, using the trigger `UART.IRQ_RXIDLE` the handler will be called once after the first character and then after the end of the message, when the line is idle.

Availability: cc3200, esp32, mimxrt, nrf, renesas-ra, rp2, samd, stm32.

Constants

`UART.RTS`

`UART.CTS`

Flow control options.

Availability: esp32, mimxrt, renesas-ra, rp2, stm32.

`UART.IRQ_RXIDLE`

`UART.IRQ_RX`

`UART.IRQ_TXIDLE`

`UART.IRQ_BREAK`

IRQ trigger sources.

Availability: renesas-ra, stm32, esp32, rp2040, mimxrt, samd, cc3200.

class SPI – a Serial Peripheral Interface bus protocol (controller side)

SPI is a synchronous serial protocol that is driven by a controller. At the physical level, a bus consists of 3 lines: SCK, MOSI, MISO. Multiple devices can share the same bus. Each device should have a separate, 4th signal, CS (Chip Select),

to select a particular device on a bus with which communication takes place. Management of a CS signal should happen in user code (via `machine.Pin` class).

Both hardware and software SPI implementations exist via the `machine.SPI` and `machine.SoftSPI` classes. Hardware SPI uses underlying hardware support of the system to perform the reads/writes and is usually efficient and fast but may have restrictions on which pins can be used. Software SPI is implemented by bit-banging and can be used on any pin but is not as efficient. These classes have the same methods available and differ primarily in the way they are constructed.

Example usage:

```
from machine import SPI, Pin

spi = SPI(0, baudrate=400000)           # Create SPI peripheral 0 at frequency of 400kHz.
# Depending on the use case, extra parameters may be required
# to select the bus characteristics and/or pins to use.
cs = Pin(4, mode=Pin.OUT, value=1)     # Create chip-select on pin 4.

try:
    cs(0)                               # Select peripheral.
    spi.write(b"12345678")              # Write 8 bytes, and don't care about received data.
finally:
    cs(1)                               # Deselect peripheral.

try:
    cs(0)                               # Select peripheral.
    rxdata = spi.read(8, 0x42)          # Read 8 bytes while writing 0x42 for each byte.
finally:
    cs(1)                               # Deselect peripheral.

rxdata = bytearray(8)
try:
    cs(0)                               # Select peripheral.
    spi.readinto(rxdata, 0x42)          # Read 8 bytes inplace while writing 0x42 for each byte.
finally:
    cs(1)                               # Deselect peripheral.

txdata = b"12345678"
rxdata = bytearray(len(txdata))
try:
    cs(0)                               # Select peripheral.
    spi.write_readinto(txdata, rxdata)  # Simultaneously write and read bytes.
finally:
    cs(1)                               # Deselect peripheral.
```

Constructors

class `machine.SPI` (*id*, ...)

Construct an SPI object on the given bus, *id*. Values of *id* depend on a particular port and its hardware. Values 0, 1, etc. are commonly used to select hardware SPI block #0, #1, etc.

With no additional parameters, the SPI object is created but not initialised (it has the settings from the last initialisation of the bus, if any). If extra arguments are given, the bus is initialised. See `init` for parameters of initialisation.

class `machine.SoftSPI` (*baudrate*=500000, *, *polarity*=0, *phase*=0, *bits*=8, *firstbit*=`MSB`, *sck*=None, *mosi*=None, *miso*=None)

Construct a new software SPI object. Additional parameters must be given, usually at least *sck*, *mosi* and *miso*, and these are used to initialise the bus. See `SPI.init` for a description of the parameters.

Methods

`SPI.init` (*baudrate*=1000000, *, *polarity*=0, *phase*=0, *bits*=8, *firstbit*=`SPI.MSB`, *sck*=None, *mosi*=None, *miso*=None, *pins*=(`SCK`, `MOSI`, `MISO`))

Initialise the SPI bus with the given parameters:

- *baudrate* is the SCK clock rate.
- *polarity* can be 0 or 1, and is the level the idle clock line sits at.
- *phase* can be 0 or 1 to sample data on the first or second clock edge respectively.
- *bits* is the width in bits of each transfer. Only 8 is guaranteed to be supported by all hardware.
- *firstbit* can be `SPI.MSB` or `SPI.LSB`.
- *sck*, *mosi*, *miso* are pins (`machine.Pin`) objects to use for bus signals. For most hardware SPI blocks (as selected by *id* parameter to the constructor), pins are fixed and cannot be changed. In some cases, hardware blocks allow 2-3 alternative pin sets for a hardware SPI block. Arbitrary pin assignments are possible only for a bitbanging SPI driver (*id* = -1).
- *pins* - WiPy port doesn't *sck*, *mosi*, *miso* arguments, and instead allows to specify them as a tuple of *pins* parameter.

In the case of hardware SPI the actual clock frequency may be lower than the requested baudrate. This is dependent on the platform hardware. The actual rate may be determined by printing the SPI object.

`SPI.deinit` ()

Turn off the SPI bus.

`SPI.read` (*nbytes*, *write*=0x00)

Read a number of bytes specified by *nbytes* while continuously writing the single byte given by *write*. Returns a `bytes` object with the data that was read.

`SPI.readinto` (*buf*, *write*=0x00)

Read into the buffer specified by *buf* while continuously writing the single byte given by *write*. Returns `None`.

Note: on WiPy this function returns the number of bytes read.

`SPI.write` (*buf*)

Write the bytes contained in *buf*. Returns `None`.

Note: on WiPy this function returns the number of bytes written.

`SPI.write_readinto(write_buf, read_buf)`

Write the bytes from `write_buf` while reading into `read_buf`. The buffers can be the same or different, but both buffers must have the same length. Returns `None`.

Note: on WiPy this function returns the number of bytes written.

Constants

`SPI.CONTROLLER`

for initialising the SPI bus to controller; this is only used for the WiPy

`SPI.MSB`

`SoftSPI.MSB`

set the first bit to be the most significant bit

`SPI.LSB`

`SoftSPI.LSB`

set the first bit to be the least significant bit

class I2C – a two-wire serial protocol

I2C is a two-wire protocol for communicating between devices. At the physical level it consists of 2 wires: SCL and SDA, the clock and data lines respectively.

I2C objects are created attached to a specific bus. They can be initialised when created, or initialised later on.

Printing the I2C object gives you information about its configuration.

Both hardware and software I2C implementations exist via the `machine.I2C` and `machine.SoftI2C` classes. Hardware I2C uses underlying hardware support of the system to perform the reads/writes and is usually efficient and fast but may have restrictions on which pins can be used. Software I2C is implemented by bit-banging and can be used on any pin but is not as efficient. These classes have the same methods available and differ primarily in the way they are constructed.

Note

The I2C bus requires pull-up circuitry on both SDA and SCL for its operation. Usually these are resistors in the range of 1 - 10 kOhm, connected from each SDA/SCL to Vcc. Without these, the behaviour is undefined and may range from blocking, unexpected watchdog reset to just wrong values. Often, this pull-up circuitry is built-in already to the MCU board or sensor breakout boards, but there is no rule for that. So please check in case of trouble. See also this excellent [learning guide](#) by Adafruit about I2C wiring.

Example usage:

```
from machine import I2C

i2c = I2C(freq=400000)           # create I2C peripheral at frequency of 400kHz
                                # depending on the port, extra parameters may be
                                # required
                                # to select the peripheral and/or pins to use

i2c.scan()                       # scan for peripherals, returning a list of 7-bit
                                # addresses

i2c.writeto(42, b'123')          # write 3 bytes to peripheral with 7-bit address 42
```

(continues on next page)

(continued from previous page)

```

i2c.readfrom(42, 4)           # read 4 bytes from peripheral with 7-bit address 42

i2c.readfrom_mem(42, 8, 3)    # read 3 bytes from memory of peripheral 42,
                              #   starting at memory-address 8 in the peripheral

i2c.writeto_mem(42, 2, b'\x10') # write 1 byte to memory of peripheral 42
                              #   starting at address 2 in the peripheral

```

Constructors

class `machine.I2C(id, *, scl, sda, freq=400000, timeout=50000)`

Construct and return a new I2C object using the following parameters:

- *id* identifies a particular I2C peripheral. Allowed values for depend on the particular port/board
- *scl* should be a pin object specifying the pin to use for SCL.
- *sda* should be a pin object specifying the pin to use for SDA.
- *freq* should be an integer which sets the maximum frequency for SCL.
- *timeout* is the maximum time in microseconds to allow for I2C transactions. This parameter is not allowed on some ports.

Note that some ports/boards will have default values of *scl* and *sda* that can be changed in this constructor. Others will have fixed values of *scl* and *sda* that cannot be changed.

class `machine.SoftI2C(scl, sda, *, freq=400000, timeout=50000)`

Construct a new software I2C object. The parameters are:

- *scl* should be a pin object specifying the pin to use for SCL.
- *sda* should be a pin object specifying the pin to use for SDA.
- *freq* should be an integer which sets the maximum frequency for SCL.
- *timeout* is the maximum time in microseconds to wait for clock stretching (SCL held low by another device on the bus), after which an `OSError(ETIMEDOUT)` exception is raised.

General Methods

`I2C.init(scl, sda, *, freq=400000)`

Initialise the I2C bus with the given arguments:

- *scl* is a pin object for the SCL line
- *sda* is a pin object for the SDA line
- *freq* is the SCL clock rate

In the case of hardware I2C the actual clock frequency may be lower than the requested frequency. This is dependent on the platform hardware. The actual rate may be determined by printing the I2C object.

`I2C.deinit()`

Turn off the I2C bus.

Availability: WiPy.

`I2C.scan()`

Scan all I2C addresses between 0x08 and 0x77 inclusive and return a list of those that respond. A device responds if it pulls the SDA line low after its address (including a write bit) is sent on the bus.

Primitive I2C operations

The following methods implement the primitive I2C controller bus operations and can be combined to make any I2C transaction. They are provided if you need more control over the bus, otherwise the standard methods (see below) can be used.

These methods are only available on the `machine.SoftI2C` class.

`I2C.start()`

Generate a START condition on the bus (SDA transitions to low while SCL is high).

`I2C.stop()`

Generate a STOP condition on the bus (SDA transitions to high while SCL is high).

`I2C.readinto(buf, nack=True, /)`

Reads bytes from the bus and stores them into *buf*. The number of bytes read is the length of *buf*. An ACK will be sent on the bus after receiving all but the last byte. After the last byte is received, if *nack* is true then a NACK will be sent, otherwise an ACK will be sent (and in this case the peripheral assumes more bytes are going to be read in a later call).

`I2C.write(buf)`

Write the bytes from *buf* to the bus. Checks that an ACK is received after each byte and stops transmitting the remaining bytes if a NACK is received. The function returns the number of ACKs that were received.

Standard bus operations

The following methods implement the standard I2C controller read and write operations that target a given peripheral device.

`I2C.readfrom(addr, nbytes, stop=True, /)`

Read *nbytes* from the peripheral specified by *addr*. If *stop* is true then a STOP condition is generated at the end of the transfer. Returns a `bytes` object with the data read.

`I2C.readfrom_into(addr, buf, stop=True, /)`

Read into *buf* from the peripheral specified by *addr*. The number of bytes read will be the length of *buf*. If *stop* is true then a STOP condition is generated at the end of the transfer.

The method returns `None`.

`I2C.writeto(addr, buf, stop=True, /)`

Write the bytes from *buf* to the peripheral specified by *addr*. If a NACK is received following the write of a byte from *buf* then the remaining bytes are not sent. If *stop* is true then a STOP condition is generated at the end of the transfer, even if a NACK is received. The function returns the number of ACKs that were received.

`I2C.writevto(addr, vector, stop=True, /)`

Write the bytes contained in *vector* to the peripheral specified by *addr*. *vector* should be a tuple or list of objects with the buffer protocol. The *addr* is sent once and then the bytes from each object in *vector* are written out sequentially. The objects in *vector* may be zero bytes in length in which case they don't contribute to the output.

If a NACK is received following the write of a byte from one of the objects in *vector* then the remaining bytes, and any remaining objects, are not sent. If *stop* is true then a STOP condition is generated at the end of the transfer, even if a NACK is received. The function returns the number of ACKs that were received.

Memory operations

Some I2C devices act as a memory device (or set of registers) that can be read from and written to. In this case there are two addresses associated with an I2C transaction: the peripheral address and the memory address. The following methods are convenience functions to communicate with such devices.

`I2C.readfrom_mem(addr, memaddr, nbytes, *, addrsize=8)`

Read *nbytes* from the peripheral specified by *addr* starting from the memory address specified by *memaddr*. The argument *addrsize* specifies the address size in bits. Returns a *bytes* object with the data read.

`I2C.readfrom_mem_into(addr, memaddr, buf, *, addrsize=8)`

Read into *buf* from the peripheral specified by *addr* starting from the memory address specified by *memaddr*. The number of bytes read is the length of *buf*. The argument *addrsize* specifies the address size in bits (on ESP8266 this argument is not recognised and the address size is always 8 bits).

The method returns `None`.

`I2C.writeto_mem(addr, memaddr, buf, *, addrsize=8)`

Write *buf* to the peripheral specified by *addr* starting from the memory address specified by *memaddr*. The argument *addrsize* specifies the address size in bits (on ESP8266 this argument is not recognised and the address size is always 8 bits).

The method returns `None`.

class I2CTarget – an I2C target device

An I2C target is a device which connects to an I2C bus and is controlled by an I2C controller. I2C targets can take many forms. The `machine.I2CTarget` class implements an I2C target that can be configured as a memory/register device, or as an arbitrary I2C device by using callbacks (if supported by the port).

Example usage for the case of a memory device:

```
from machine import I2CTarget

# Create the backing memory for the I2C target.
mem = bytearray(8)

# Create an I2C target. Depending on the port, extra parameters
# may be required to select the peripheral and/or pins to use.
i2c = I2CTarget(addr=67, mem=mem)

# At this point an I2C controller can read and write `mem`.
...

# Deinitialise the I2C target.
i2c.deinit()
```

Note that some ports require an *id*, and maybe *scl* and *sda* pins, to be passed to the `I2CTarget` constructor, to select the hardware I2C instance and pins that it connects to.

When configured as a memory device, it's also possible to register to receive events. For example to be notified when the memory is read/written:

```
from machine import I2CTarget

# Define an IRQ handler, for I2C events.
def irq_handler(i2c_target):
    flags = i2c_target.irq().flags()
    if flags & I2CTarget.IRQ_END_READ:
        print("controller read target at addr", i2c_target.memaddr)
    if flags & I2CTarget.IRQ_END_WRITE:
        print("controller wrote target at addr", i2c_target.memaddr)
```

(continues on next page)

(continued from previous page)

```
# Create the I2C target and register to receive default events.
mem = bytearray(8)
i2c = I2CTarget(addr=67, mem=mem)
i2c.irq(irq_handler)
```

More complicated I2C devices can be implemented using the full set of events. For example, to see the raw events as they are triggered:

```
from machine import I2CTarget

# Define an IRQ handler that prints the event id and responds to reads/writes.
def irq_handler(i2c_target, buf=bytearray(1)):
    flags = i2c_target.irq().flags()
    print(flags)
    if flags & I2CTarget.IRQ_READ_REQ:
        i2c_target.write(buf)
    if flags & I2CTarget.IRQ_WRITE_REQ:
        i2c_target.readinto(buf)

# Create the I2C target and register to receive all events.
i2c = I2CTarget(addr=67)
all_triggers = (
    I2CTarget.IRQ_ADDR_MATCH_READ
    | I2CTarget.IRQ_ADDR_MATCH_WRITE
    | I2CTarget.IRQ_READ_REQ
    | I2CTarget.IRQ_WRITE_REQ
    | I2CTarget.IRQ_END_READ
    | I2CTarget.IRQ_END_WRITE
)
i2c.irq(irq_handler, trigger=all_triggers, hard=True)
```

Constructors

class machine.**I2CTarget** (*id*, *addr*, *, *addrsz*=7, *mem*=None, *mem_addrsz*=8, *scl*=None, *sda*=None)

Construct and return a new I2CTarget object using the following parameters:

- *id* identifies a particular I2C peripheral. Allowed values depend on the particular port/board. Some ports have a default in which case this parameter can be omitted.
- *addr* is the I2C address of the target.
- *addrsz* is the number of bits in the I2C target address. Valid values are 7 and 10.
- *mem* is an object with the buffer protocol that is writable. If not specified then there is no backing memory and data must be read/written using the `I2CTarget.readinto()` and `I2CTarget.write()` methods.
- *mem_addrsz* is the number of bits in the memory address. Valid values are 0, 8, 16, 24 and 32.
- *scl* is a pin object specifying the pin to use for SCL.
- *sda* is a pin object specifying the pin to use for SDA.

Note that some ports/boards will have default values of *scl* and *sda* that can be changed in this constructor. Others will have fixed values of *scl* and *sda* that cannot be changed.

General Methods

`I2CTarget.deinit()`

Deinitialise the I2C target. After this method is called the hardware will no longer respond to requests on the I2C bus, and no other methods can be called.

`I2CTarget.readinto(buf)`

Read into the given buffer any pending bytes written by the I2C controller. Returns the number of bytes read.

`I2CTarget.write(buf)`

Write out the bytes from the given buffer, to be passed to the I2C controller after it sends a read request. Returns the number of bytes written. Most ports only accept one byte at a time to this method.

`I2CTarget.irq(handler=None, trigger=IRQ_END_READ | IRQ_END_WRITE, hard=False)`

Configure an IRQ *handler* to be called when an event occurs. The possible events are given by the following constants, which can be or'd together and passed to the *trigger* argument:

- `IRQ_ADDR_MATCH_READ` indicates that the target was addressed by a controller for a read transaction.
- `IRQ_ADDR_MATCH_WRITE` indicates that the target was addressed by a controller for a write transaction.
- `IRQ_READ_REQ` indicates that the controller is requesting data, and this request must be satisfied by calling `I2CTarget.write` with the data to be passed back to the controller.
- `IRQ_WRITE_REQ` indicates that the controller has written data, and the data must be read by calling `I2CTarget.readinto`.
- `IRQ_END_READ` indicates that the controller has finished a read transaction.
- `IRQ_END_WRITE` indicates that the controller has finished a write transaction.

Not all triggers are available on all ports. If a port has the constant then that event is available.

Note the following restrictions:

- `IRQ_ADDR_MATCH_READ`, `IRQ_ADDR_MATCH_WRITE`, `IRQ_READ_REQ` and `IRQ_WRITE_REQ` must be handled by a hard IRQ callback (with the *hard* argument set to `True`). This is because these events have very strict timing requirements and must usually be satisfied synchronously with the hardware event.
- `IRQ_END_READ` and `IRQ_END_WRITE` may be handled by either a soft or hard IRQ callback (although note that all events must be registered with the same handler, so if any events need a hard callback then all events must be hard).
- If a memory buffer has been supplied in the constructor then `IRQ_END_WRITE` is not emitted for the transaction that writes the memory address. This is to allow `IRQ_END_READ` and `IRQ_END_WRITE` to function correctly as soft IRQ callbacks, where the IRQ handler may be called quite some time after the actual hardware event.

`I2CTarget.memaddr`

The integer value of the most recent memory address that was selected by the I2C controller (only valid if `mem` was specified in the constructor).

Constants

`I2CTarget.IRQ_ADDR_MATCH_READ`

`I2CTarget.IRQ_ADDR_MATCH_WRITE`

`I2CTarget.IRQ_READ_REQ`

`I2CTarget.IRQ_WRITE_REQ`

`I2CTarget.IRQ_END_READ`

I2CTarget.IRQ_END_WRITE

IRQ trigger sources.

class I2S – Inter-IC Sound bus protocol

I2S is a synchronous serial protocol used to connect digital audio devices. At the physical level, a bus consists of 3 lines: SCK, WS, SD. The I2S class supports controller operation. Peripheral operation is not supported.

The I2S class is currently available as a Technical Preview. During the preview period, feedback from users is encouraged. Based on this feedback, the I2S class API and implementation may be changed.

I2S objects can be created and initialized using:

```
from machine import I2S
from machine import Pin

# ESP32
sck_pin = Pin(14)    # Serial clock output
ws_pin = Pin(13)     # Word clock output
sd_pin = Pin(12)     # Serial data output

or

# PyBoards
sck_pin = Pin("Y6")  # Serial clock output
ws_pin = Pin("Y5")   # Word clock output
sd_pin = Pin("Y8")   # Serial data output

audio_out = I2S(2,
                 sck=sck_pin, ws=ws_pin, sd=sd_pin,
                 mode=I2S.TX,
                 bits=16,
                 format=I2S.MONO,
                 rate=44100,
                 ibuf=20000)

audio_in = I2S(2,
               sck=sck_pin, ws=ws_pin, sd=sd_pin,
               mode=I2S.RX,
               bits=32,
               format=I2S.STEREO,
               rate=22050,
               ibuf=20000)
```

3 modes of operation are supported:

- blocking
- non-blocking
- asyncio

blocking:

```
num_written = audio_out.write(buf) # blocks until buf emptied

num_read = audio_in.readinto(buf) # blocks until buf filled
```

non-blocking:

```
audio_out.irq(i2s_callback)      # i2s_callback is called when buf is emptied
num_written = audio_out.write(buf) # returns immediately

audio_in.irq(i2s_callback)      # i2s_callback is called when buf is filled
num_read = audio_in.readinto(buf) # returns immediately
```

asyncio:

```
swriter = asyncio.StreamWriter(audio_out)
swriter.write(buf)
await swriter.drain()

sreader = asyncio.StreamReader(audio_in)
num_read = await sreader.readinto(buf)
```

Some codec devices like the WM8960 or SGTL5000 require separate initialization before they can operate with the I2S class. For these, separate drivers are supplied, which also offer methods for controlling volume, audio processing and other things. For these drivers see:

- [WM8960 – Driver for the WM8960 codec](#)

Constructor

class machine.I2S (*id*, *, *sck*, *ws*, *sd*, *mck*=None, *mode*, *bits*, *format*, *rate*, *ibuf*)

Construct an I2S object of the given id:

- *id* identifies a particular I2S bus; it is board and port specific

Keyword-only parameters that are supported on all ports:

- *sck* is a pin object for the serial clock line
- *ws* is a pin object for the word select line
- *sd* is a pin object for the serial data line
- *mck* is a pin object for the master clock line; master clock frequency is sampling rate * 256
- *mode* specifies receive or transmit
- *bits* specifies sample size (bits), 16 or 32
- *format* specifies channel format, STEREO or MONO
- *rate* specifies audio sampling rate (Hz); this is the frequency of the *ws* signal
- *ibuf* specifies internal buffer length (bytes)

For all ports, DMA runs continuously in the background and allows user applications to perform other operations while sample data is transferred between the internal buffer and the I2S peripheral unit. Increasing the size of the internal buffer has the potential to increase the time that user applications can perform non-I2S operations before underflow (e.g. `write` method) or overflow (e.g. `readinto` method).

Methods

I2S.**init** (*sck*, ...)

see Constructor for argument descriptions

I2S.deinit()

Deinitialize the I2S bus

I2S.readinto(buf)

Read audio samples into the buffer specified by `buf`. `buf` must support the buffer protocol, such as bytearray or array. “buf” byte ordering is little-endian. For Stereo format, left channel sample precedes right channel sample. For Mono format, the left channel sample data is used. Returns number of bytes read

I2S.write(buf)

Write audio samples contained in `buf`. `buf` must support the buffer protocol, such as bytearray or array. “buf” byte ordering is little-endian. For Stereo format, left channel sample precedes right channel sample. For Mono format, the sample data is written to both the right and left channels. Returns number of bytes written

I2S.irq(handler)

Set a callback. `handler` is called when `buf` is emptied (`write` method) or becomes full (`readinto` method). Setting a callback changes the `write` and `readinto` methods to non-blocking operation. `handler` is called in the context of the MicroPython scheduler.

static I2S.shift(*, buf, bits, shift)

bitwise shift of all samples contained in `buf`. `bits` specifies sample size in bits. `shift` specifies the number of bits to shift each sample. Positive for left shift, negative for right shift. Typically used for volume control. Each bit shift changes sample volume by 6dB.

Constants

I2S.RX

for initialising the I2S bus mode to receive

I2S.TX

for initialising the I2S bus mode to transmit

I2S.STEREO

for initialising the I2S bus format to stereo

I2S.MONO

for initialising the I2S bus format to mono

class RTC – real time clock

The RTC is an independent clock that keeps track of the date and time.

Example usage:

```
rtc = machine.RTC()
rtc.datetime((2020, 1, 21, 2, 10, 32, 36, 0))
print(rtc.datetime())
```

Constructors

class machine.RTC(id=0, ...)

Create an RTC object. See `init` for parameters of initialization.

Methods

`RTC.datetime([datetimeuple])`

Get or set the date and time of the RTC.

With no arguments, this method returns an 8-tuple with the current date and time. With 1 argument (being an 8-tuple) it sets the date and time.

The 8-tuple has the following format:

(year, month, day, weekday, hours, minutes, seconds, subseconds)

The meaning of the `subseconds` field is hardware dependent.

`RTC.init(datetime)`

Initialise the RTC. Datetime is a tuple of the form:

(year, month, day, hour, minute, second, microsecond, tzinfo)

All eight arguments must be present. The `microsecond` and `tzinfo` values are currently ignored but might be used in the future.

Availability: CC3200, ESP32, MIMXRT, SAMD. The `rtc.init()` method on the `stm32` and `renesas-ra` ports just (re-)starts the RTC and does not accept arguments.

`RTC.now()`

Get get the current datetime tuple.

Availability: WiPy.

`RTC.deinit()`

Resets the RTC to the time of January 1, 2015 and starts running it again.

`RTC.alarm(id, time, *, repeat=False)`

Set the RTC alarm. Time might be either a millisecond value to program the alarm to current time + `time_in_ms` in the future, or a `datetimeuple`. If the time passed is in milliseconds, `repeat` can be set to `True` to make the alarm periodic.

`RTC.alarm_left(alarm_id=0)`

Get the number of milliseconds left before the alarm expires.

`RTC.alarm_cancel(alarm_id=0)`

Cancel a running alarm.

The `mimxrt` port also exposes this function as `RTC.cancel(alarm_id=0)`, but this is scheduled to be removed in MicroPython 2.0.

`RTC.irq(*, trigger, handler=None, wake=machine.IDLE)`

Create an `irq` object triggered by a real time clock alarm.

- `trigger` must be `RTC.ALARM0`
- `handler` is the function to be called when the callback is triggered.
- `wake` specifies the sleep mode from where this interrupt can wake up the system.

`RTC.memory([data])`

`RTC.memory(data)` will write `data` to the RTC memory, where `data` is any object which supports the buffer protocol (including `bytes`, `bytearray`, `memoryview` and `array.array`). `RTC.memory()` reads RTC memory and returns a `bytes` object.

Data written to RTC user memory is persistent across restarts, including *Soft Reset* and `machine.deepsleep()`.

The maximum length of RTC user memory is 2048 bytes by default on esp32, and 492 bytes on esp8266.

Availability: esp32, esp8266 ports.

Constants

RTC.**ALARM0**

irq trigger source

class Timer – control hardware timers

Hardware timers deal with timing of periods and events. Timers are perhaps the most flexible and heterogeneous kind of hardware in MCUs and SoCs, differently greatly from a model to a model. MicroPython's Timer class defines a baseline operation of executing a callback with a given period (or once after some delay), and allow specific boards to define more non-standard behaviour (which thus won't be portable to other boards).

See discussion of *important constraints* on Timer callbacks.

Note

Memory can't be allocated inside irq handlers (an interrupt) and so exceptions raised within a handler don't give much information. See `micropython.alloc_emergency_exception_buf()` for how to get around this limitation.

If you are using a WiPy board please refer to `machine.TimerWiPy` instead of this class.

Constructors

class `machine.Timer(id, /, ...)`

Construct a new timer object of the given `id`. `id` of -1 constructs a virtual timer (if supported by a board). `id` shall not be passed as a keyword argument.

See `init` for parameters of initialisation.

Methods

`Timer.init(*, mode=Timer.PERIODIC, freq=-1, period=-1, callback=None)`

Initialise the timer. Example:

```
def mycallback(t):
    pass

# periodic at 1kHz
tim.init(mode=Timer.PERIODIC, freq=1000, callback=mycallback)

# periodic with 100ms period
tim.init(period=100, callback=mycallback)

# one shot firing after 1000ms
tim.init(mode=Timer.ONE_SHOT, period=1000, callback=mycallback)
```

Keyword arguments:

- `mode` can be one of:
 - `Timer.ONE_SHOT` - The timer runs once until the configured period of the channel expires.

- `Timer.PERIODIC` - The timer runs periodically at the configured frequency of the channel.
- `freq` - The timer frequency, in units of Hz. The upper bound of the frequency is dependent on the port. When both the `freq` and `period` arguments are given, `freq` has a higher priority and `period` is ignored.
- `period` - The timer period, in milliseconds.
- `callback` - The callable to call upon expiration of the timer period. The callback must take one argument, which is passed the `Timer` object. The `callback` argument shall be specified. Otherwise an exception will occur upon timer expiration: `TypeError: 'NoneType' object isn't callable`

`Timer.deinit()`

Deinitialises the timer. Stops the timer, and disables the timer peripheral.

Constants

`Timer.ONE_SHOT`

`Timer.PERIODIC`

Timer operating mode.

class Counter – pulse counter

Counter implements pulse counting by monitoring an input signal and counting rising or falling edges.

Minimal example usage:

```
from machine import Pin, Counter

counter = Counter(0, Pin(0, Pin.IN)) # create Counter for pin 0 and begin counting
value = counter.value()             # retrieve current pulse count
```

Availability: **ESP32**

Constructors

class `machine.Counter` (*id*, ...)

Returns the singleton Counter object for the the given *id*. Values of *id* depend on a particular port and its hardware. Values 0, 1, etc. are commonly used to select hardware block #0, #1, etc.

Additional arguments are passed to the `init()` method described below, and will cause the Counter instance to be re-initialised and reset.

On ESP32, the *id* corresponds to a *PCNT unit*.

Methods

`Counter.init` (*src*, *, ...)

Initialise and reset the Counter with the given parameters:

- *src* specifies the input pin as a *machine.Pin* object. May be omitted on ports that have a predefined pin for a given hardware block.

Additional keyword-only parameters that may be supported by a port are:

- *edge* specifies the edge to count. Either `Counter.RISING` (the default) or `Counter.FALLING`. (*Supported on ESP32*)
- *direction* specifies the direction to count. Either `Counter.UP` (the default) or `Counter.DOWN`. (*Supported on ESP32*)

- *filter_ns* specifies a minimum period of time in nanoseconds that the source signal needs to be stable for a pulse to be counted. Implementations should use the longest filter supported by the hardware that is less than or equal to this value. The default is 0 (no filter). (*Supported on ESP32*)

`Counter.deinit()`

Stops the Counter, disabling any interrupts and releasing hardware resources. A Soft Reset should deinitialize all Counter objects.

`Counter.value([value])`

Get, and optionally set, the counter value as a signed integer. Implementations must aim to do the get and set atomically (i.e. without leading to skipped counts).

This counter value could exceed the range of a *small integer*, which means that calling `Counter.value()` could cause a heap allocation, but implementations should aim to ensure that internal state only uses small integers and therefore will not allocate until the user calls `Counter.value()`.

For example, on ESP32, the internal state counts overflows of the hardware counter (every 32000 counts), which means that it will not exceed the small integer range until $2^{30} * 32000$ counts (slightly over 1 year at 1MHz).

In general, it is recommended that you should use `Counter.value(0)` to reset the counter (i.e. to measure the counts since the last call), and this will avoid this problem.

Constants

`Counter.RISING`

`Counter.FALLING`

Select the pulse edge.

`Counter.UP`

`Counter.DOWN`

Select the counting direction.

class Encoder – quadrature decoding

Encoder implements decoding of quadrature signals as commonly output from rotary encoders, by counting either up or down depending on the order of two input pulses.

Minimal example usage:

```
from machine import Pin, Encoder

counter = Counter(0, Pin(0, Pin.IN), Pin(1, Pin.IN))  # create Encoder for pins 0, 1
↪and begin counting
value = counter.value()                               # retrieve current count
```

Availability: **ESP32**

Constructors

`class machine.Encoder(id, ...)`

Returns the singleton Encoder object for the the given *id*. Values of *id* depend on a particular port and its hardware. Values 0, 1, etc. are commonly used to select hardware block #0, #1, etc.

Additional arguments are passed to the `init()` method described below, and will cause the Encoder instance to be re-initialised and reset.

On ESP32, the *id* corresponds to a *PCNT unit*.

Methods

`Encoder.init(phase_a, phase_b, *, ...)`

Initialise and reset the Encoder with the given parameters:

- *phase_a* specifies the first input pin as a *machine.Pin* object.
- *phase_b* specifies the second input pin as a *machine.Pin* object.

These pins may be omitted on ports that have predefined pins for a given hardware block.

Additional keyword-only parameters that may be supported by a port are:

- *filter_ns* specifies a minimum period of time in nanoseconds that the source signal needs to be stable for a pulse to be counted. Implementations should use the longest filter supported by the hardware that is less than or equal to this value. The default is 0 (no filter). (*Supported on ESP32*)
- *phases* specifies the number of signal edges to count and thus the granularity of the decoding. e.g. 4 phases corresponds to “4x quadrature decoding”, and will result in four counts per pulse. Ports may support either 1, 2, or 4 phases and the default is 1 phase. (*Supported on ESP32*)

`Encoder.deinit()`

Stops the Encoder, disabling any interrupts and releasing hardware resources. A Soft Reset should deinitialize all Encoder objects.

`Encoder.value([value])`

Get, and optionally set, the encoder value as a signed integer. Implementations should aim to do the get and set atomically.

See `machine.Counter.value()` for details about overflow of this value.

class WDT – watchdog timer

The WDT is used to restart the system when the application crashes and ends up into a non recoverable state. Once started it cannot be stopped or reconfigured in any way. After enabling, the application must “feed” the watchdog periodically to prevent it from expiring and resetting the system.

Example usage:

```
from machine import WDT
wdt = WDT(timeout=2000) # enable it with a timeout of 2s
wdt.feed()
```

Availability of this class: pyboard, WiPy, esp8266, esp32, rp2040, mimmxt.

Constructors

`class machine.WDT(id=0, timeout=5000)`

Create a WDT object and start it. The timeout must be given in milliseconds. Once it is running the timeout cannot be changed and the WDT cannot be stopped either.

Notes: On the esp8266 a timeout cannot be specified, it is determined by the underlying system. On rp2040 devices, the maximum timeout is 8388 ms.

Methods

`WDT.feed()`

Feed the WDT to prevent it from resetting the system. The application should place this call in a sensible place ensuring that the WDT is only fed after verifying that everything is functioning correctly.

class SD – secure digital memory card (cc3200 port only)

Warning

This is a non-standard class and is only available on the cc3200 port.

The SD card class allows to configure and enable the memory card module of the WiPy and automatically mount it as `/sd` as part of the file system. There are several pin combinations that can be used to wire the SD card socket to the WiPy and the pins used can be specified in the constructor. Please check the [pinout and alternate functions table](#) for more info regarding the pins which can be remapped to be used with a SD card.

Example usage:

```
from machine import SD
import vfs
# clk cmd and dat0 pins must be passed along with
# their respective alternate functions
sd = machine.SD(pins=('GP10', 'GP11', 'GP15'))
vfs.mount(sd, '/sd')
# do normal file operations
```

Constructors

class `machine.SD(id, ...)`

Create a SD card object. See `init()` for parameters if initialization.

Methods

`SD.init(id=0, pins=('GP10', 'GP11', 'GP15'))`

Enable the SD card. In order to initialize the card, give it a 3-tuple: (`clk_pin`, `cmd_pin`, `dat0_pin`).

`SD.deinit()`

Disable the SD card.

class SDCard – secure digital memory card

SD cards are one of the most common small form factor removable storage media. SD cards come in a variety of sizes and physical form factors. MMC cards are similar removable storage devices while eMMC devices are electrically similar storage devices designed to be embedded into other systems. All three form share a common protocol for communication with their host system and high-level support looks the same for them all. As such in MicroPython they are implemented in a single class called `machine.SDCard`.

Both SD and MMC interfaces support being accessed with a variety of bus widths. When being accessed with a 1-bit wide interface they can be accessed using the SPI protocol. Different MicroPython hardware platforms support different widths and pin configurations but for most platforms there is a standard configuration for any given hardware. In general constructing an `SDCard` object with without passing any parameters will initialise the interface to the default card slot for the current hardware. The arguments listed below represent the common arguments that might need to be set in order to use either a non-standard slot or a non-standard pin assignment. The exact subset of arguments supported will vary from platform to platform.

`SDCard(slot=1, width=1, cd=None, wp=None, sck=None, miso=None, mosi=None,`

```
cs=None, cmd=None, data=None, freq=20000000)
```

This class provides access to SD or MMC storage cards using either a dedicated SD/MMC interface hardware or through an SPI channel. The class implements the block protocol defined by `vfs.AbstractBlockDev`. This allows the mounting of an SD card to be as simple as:

```
vfs.mount(machine.SDCard(), "/sd")
```

The constructor takes the following parameters:

- *slot* selects which of the available interfaces to use. Leaving this unset will select the default interface.
- *width* selects the bus width for the SD/MMC interface. This many data pins must be connected to the SD card.
- *cd* can be used to specify a card-detect pin.
- *wp* can be used to specify a write-protect pin.
- *sck* can be used to specify an SPI clock pin.
- *miso* can be used to specify an SPI miso pin.
- *mosi* can be used to specify an SPI mosi pin.
- *cs* can be used to specify an SPI chip select pin.

The following additional parameters are only present on ESP32 port:

- *cmd* can be used to specify the SD CMD pin (ESP32-S3 only).
- *data* can be used to specify a list or tuple of SD data bus pins (ESP32-S3 only).
- *freq* selects the SD/MMC interface frequency in Hz.

Implementation-specific details

Different implementations of the `SDCard` class on different hardware support varying subsets of the options above.

PyBoard

The standard PyBoard has just one slot. No arguments are necessary or supported.

ESP32

SD cards support access in both SD/MMC mode and the simpler (but slower) SPI mode.

SPI mode makes use of a [SPI](#) host peripheral, which cannot concurrently be used for other SPI interactions.

The `slot` argument determines which mode is used. Different values are supported on different chips:

Chip	Slot 0	Slot 1	Slot 2	Slot 3
ESP32		SD/MMC	SPI (id=1)	SPI (id=0)
ESP32-C3			SPI (id=0)	
ESP32-C6			SPI (id=0)	
ESP32-S2			SPI (id=1)	SPI (id=0)
ESP32-S3	SD/MMC	SD/MMC	SPI (id=1)	SPI (id=0)

Different slots support different data bus widths (number of data pins):

Slot	Type	Supported data widths
0	SD/MMC	1, 4, 8
1	SD/MMC	1, 4
2	SPI	1
3	SPI	1

Note

Most ESP32 modules that provide an SD card slot using the dedicated hardware only wire up 1 data pin, so the default value for `width` is 1.

Additional details depend on which ESP32 family chip is in use:

Original ESP32

In SD/MMC mode (slot 1), pin assignments in SD/MMC mode are fixed on the original ESP32. The SPI mode slots (2 & 3) allow pins to be set to different values in the constructor.

The default pin assignments are as follows:

Slot	1	2	3	Can be set
Signal	Pin	Pin	Pin	
CLK	14			No
CMD	15			No
D0	2			No
D1	4			No
D2	12			No
D3	13			No
sck		18	14	Yes
cs		5	15	Yes
miso		19	12	Yes
mosi		23	13	Yes

The `cd` and `wp` pins are not fixed in either mode and default to disabled, unless set.

ESP32-S3

The ESP32-S3 chip allows pins to be set to different values for both SD/MMC and SPI mode access.

If not set, default pin assignments are as follows:

Slot Signal	0 Pin	1 Pin	2 Pin	3 Pin
CLK	14	14		
CMD	15	15		
D0	2	2		
D1	4	4		
D2	12	12		
D3	13	13		
D4	33*			
D5	34*			
D6	35*			
D7	36*			
sck			37*	14
cs			34*	13
miso			37*	2
mosi			35*	15

Note

Slots 0 and 1 cannot both be in use at the same time.

Note

Pins marked with an asterisk * in the table must be changed from the default if the ESP32-S3 board is configured for Octal SPI Flash or PSRAM.

To access a card in SD/MMC mode, set `slot` parameter value 0 or 1 and parameters `sck` (for CLK), `cmd` and `data` as needed to assign pins. If the `data` argument is passed then it should be a list or tuple of data pins or pin numbers with length equal to the `width` argument. For example:

```
sd = SDCard(slot=0, width=4, sck=8, cmd=9, data=(10, 11, 12, 13))
```

To access a card in SPI mode, set `slot` parameter value 2 or 3 and pass parameters `sck`, `cs`, `miso`, `mosi` as needed to assign pins.

In either mode the `cd` and `wp` pins default to disabled, unless set in the constructor.

Other ESP32 chips

Other ESP32 family chips do not have hardware SD/MMC host controllers and can only access SD cards in SPI mode.

To access a card in SPI mode, set `slot` parameter value 2 or 3 and pass parameters `sck`, `cs`, `miso`, `mosi` to assign pins.

Note

ESP32-C3 and ESP32-C6 only have one available *SPI* bus, so the only valid `slot` parameter value is 2. Using this bus for the SD card will prevent also using it for *machine.SPI*.

cc3200

You can set the pins used for SPI access by passing a tuple as the *pins* argument.

Note: The current cc3200 SD card implementation names the this class `machine.SD` rather than `machine.SDCard`.

mimxrt

The SDCard module for the mimxrt port only supports access via dedicated SD/MMC peripheral (USDHC) in 4-bit mode with 50MHz clock frequency exclusively. Unfortunately the MIMXRT1011 controller does not support the USDHC peripheral. Hence this controller does not feature the `machine.SDCard` module.

Due to the decision to only support 4-bit mode with 50MHz clock frequency the interface has been simplified, and the constructor signature is:

```
class machine.SDCard(slot=1)
```

The pins used for the USDHC peripheral have to be configured in `mpconfigboard.h`. Most of the controllers supported by the mimxrt port provide up to two USDHC peripherals. Therefore the pin configuration is performed using the macro `MICROPY_USDHCx` with *x* being 1 or 2 respectively.

The following shows an example configuration for USDHC1:

```
#define MICROPY_USDHC1 \
{ \
    .cmd    = { GPIO_SD_B0_02_USDHC1_CMD }, \
    .clk    = { GPIO_SD_B0_03_USDHC1_CLK }, \
    .cd_b   = { GPIO_SD_B0_06_USDHC1_CD_B }, \
    .data0  = { GPIO_SD_B0_04_USDHC1_DATA0 }, \
    .data1  = { GPIO_SD_B0_05_USDHC1_DATA1 }, \
    .data2  = { GPIO_SD_B0_00_USDHC1_DATA2 }, \
    .data3  = { GPIO_SD_B0_01_USDHC1_DATA3 }, \
}
```

If the card detect pin is not used (cb_b pin) then the respective entry has to be filled with the following dummy value:

```
#define USDHC_DUMMY_PIN NULL, 0
```

Based on the definition of macro `MICROPY_USDHC1` and/or `MICROPY_USDHC2` the `machine.SDCard` module either supports one or two slots. If only one of the defines is provided, calling `machine.SDCard()` or `machine.SDCard(1)` will return an instance using the respective USDHC peripheral. When both macros are defined, calling `machine.SDCard(2)` returns an instance using USDHC2.

class USBDevice – USB Device driver

Note

`machine.USBDevice` is currently only supported for esp32, rp2 and samd ports. Native USB support is also required, and not every board supports native USB.

USBDevice provides a low-level Python API for implementing USB device functions using Python code.

Warning

This low-level API assumes familiarity with the USB standard. There are high-level [usb driver modules](#) in [micropython-lib](#) which provide a simpler interface and more built-in functionality.

Terminology

- A “Runtime” USB device interface or driver is one which is defined using this Python API after MicroPython initially starts up.
- A “Built-in” USB device interface or driver is one that is compiled into the MicroPython firmware, and is always available. Examples are USB-CDC (serial port) which is usually enabled by default. Built-in USB-MSC (Mass Storage) is an option on some ports.

Lifecycle

Managing a runtime USB interface can be tricky, especially if you are communicating with MicroPython over a built-in USB-CDC serial port that’s part of the same USB device.

- A MicroPython [soft reset](#) will always clear all runtime USB interfaces, which results in the entire USB device disconnecting from the host. If MicroPython is also providing a built-in USB-CDC serial port then this will re-appear after the soft reset.

This means some functions (like `mpremote run`) that target the USB-CDC serial port will immediately fail if a runtime USB interface is active, because the port goes away when `mpremote` triggers a soft reset. The operation should succeed on the second try, as after the soft reset there is no more runtime USB interface.

- To configure a runtime USB device on every boot, it’s recommended to place the configuration code in the [boot.py](#) file on the [device VFS](#). On each reset this file is executed before the USB subsystem is initialised (and before [main.py](#)), so it allows the board to come up with the runtime USB device immediately.
- For development or debugging, it may be convenient to connect a hardware serial REPL and disable the built-in USB-CDC serial port entirely. Not all ports support this (currently only `rp2`). The custom build should be configured with `#define MICROPY_HW_USB_CDC (0)` and `#define MICROPY_HW_ENABLE_UART_REPL (1)`.

Constructors

`class machine.USBDevice`

Construct a USBDevice object.

Note

This object is a singleton, each call to this constructor returns the same object reference.

Methods

`USBDevice.config(desc_dev, desc_cfg, desc_strs=None, open_itf_cb=None, reset_cb=None, control_xfer_cb=None, xfer_cb=None)`

Configures the `USBDevice` singleton object with the USB runtime device state and callback functions:

- `desc_dev` - A bytes-like object containing the new USB device descriptor.
- `desc_cfg` - A bytes-like object containing the new USB configuration descriptor.

- **desc_strs** - Optional object holding strings or bytes objects

containing USB string descriptor values. Can be a list, a dict, or any object which supports subscript indexing with integer keys (USB string descriptor index).

Strings are an optional USB feature, and this parameter can be unset (default) if no strings are referenced in the device and configuration descriptors, or if only built-in strings should be used.

Apart from index 0, all the string values should be plain ASCII. Index 0 is the special “languages” USB descriptor, represented as a bytes object with a custom format defined in the USB standard. `None` can be returned at index 0 in order to use a default “English” language descriptor.

To fall back to providing a built-in string value for a given index, a subscript lookup can return `None`, raise `KeyError`, or raise `IndexError`.

- **open_intf_cb** - This callback is called once for each interface or Interface Association Descriptor in response to a Set Configuration request from the USB Host (the final stage before the USB device is available to the host).

The callback takes a single argument, which is a memoryview of the interface or IAD descriptor that the host is accepting (including all associated descriptors). It is a view into the same `desc_cfg` object that was provided as a separate argument to this function. The memoryview is only valid until the callback function returns.

- **reset_cb** - This callback is called when the USB host performs a bus reset. The callback takes no arguments. Any in-progress transfers will never complete. The USB host will most likely proceed to re-enumerate the USB device by calling the descriptor callbacks and then `open_intf_cb()`.
- **control_xfer_cb** - This callback is called one or more times for each USB control transfer (device Endpoint 0). It takes two arguments.

The first argument is the control transfer stage. It is one of:

- 1 for SETUP stage.
- 2 for DATA stage.
- 3 for ACK stage.

Second argument is a memoryview to read the USB control request data for this stage. The memoryview is only valid until the callback function returns. Data in this memoryview will be the same across each of the three stages of a single transfer.

A successful transfer consists of this callback being called in sequence for the three stages. Generally speaking, if a device wants to do something in response to a control request then it’s best to wait until the ACK stage to confirm the host controller completed the transfer as expected.

The callback should return one of the following values:

- `False` to stall the endpoint and reject the transfer. It won’t proceed to any remaining stages.
- `True` to continue the transfer to the next stage.
- A buffer object can be returned at the SETUP stage when the transfer will send or receive additional data. Typically this is the case when the `wLength` field in the request has a non-zero value. This should be a writable buffer for an `OUT` direction transfer, or a readable buffer with data for an `IN` direction transfer.

- **xfer_cb** - This callback is called whenever a non-control transfer submitted by calling `USBDevice.submit_xfer()` completes.

The callback has three arguments:

1. The Endpoint number for the completed transfer.
2. Result value: `True` if the transfer succeeded, `False` otherwise.